Unique raw Optimised Pattern values: 3735

1. 0-1 BFS (Deque)
2. 0-1 BFS (Deque) or Dijkstra's Algorithm with a 3D State Space `(row, col, direction/momentum)`.
3. 0-1 BFS / Dijkstra's Algorithm
4. 0-1 BFS or Dijkstra
5. 0-1 BFS or Dijkstra's Algorithm on a modified graph where every directed edge (u -> v) with weight 0 has a "ghost" edge (v -> u) with weight 1.
6. 0-1 Trie (Binary Trie).
7. 0/1 Knapsack DP (1D) Use a boolean array `dp` where `dp[i]` is true if sum `i` is possible.
8. 0/1 Knapsack Dynamic Programming
9. 0/1 Knapsack Dynamic Programming (Inverse Problem). Find subsets summing to strictly less than K, and subtract from total subsets.
10. 1D Array Direct Indexing (1-based to 0-based mapping)
11. 1D Array Extraction & Modular Rotation
12. 1D Array tracking with consecutive free-block counting and slice assignment.
13. 1D DP (Character Bucket State)
14. 1D DP (Fibonacci) + Math Independence
15. 1D DP + Bitmasking (State tracking per bit).
16. 1D DP / Two Pointers If arr[i]-arr[i-1] == arr[i-1]-arr[i-2], current slices = prev slices + 1. Add to total sum.
17. 1D Dynamic Programming
18. 1D Dynamic Programming (2 States)
19. 1D Dynamic Programming (Boolean State)
20. 1D Dynamic Programming (Bottom-Up)
21. 1D Dynamic Programming (Climbing Stairs Variant)
22. 1D Dynamic Programming (Fibonacci-like)
23. 1D Dynamic Programming (Iterative Tabulation).
24. 1D Dynamic Programming (Knapsack Variation)
25. 1D Dynamic Programming (Knapsack)
26. 1D Dynamic Programming (Memoization)
27. 1D Dynamic Programming (Right to Left).
28. 1D Dynamic Programming (Rolling Array) calculated strictly from right-to-left.
29. 1D Dynamic Programming (State Machine).
30. 1D Dynamic Programming (Suffix/Prefix Array)
31. 1D Dynamic Programming (Tabulation).
32. 1D Dynamic Programming (Two States)
33. 1D Dynamic Programming (Unbounded Knapsack).
34. 1D Dynamic Programming + GCD Check
35. 1D Dynamic Programming / Greedy
36. 1D Dynamic Programming / Sliding Window
37. 1D Dynamic Programming with 2D Palindrome Pre-processing.
38. 1D Dynamic Programming with Frequency Map optimization.

39. 1D Greedy / Sorting
40. 1D Index Mapping (Modulo Arithmetic)
41. 1D Projection (Interval Overlap).
42. 1D Rolling DP (4 Stages)
43. 26x26 Frequency Map Simulation.
44. 2D Binary Indexed Tree (Fenwick Tree)
45. 2D Bitset / Hash Set of Pairs
46. 2D Diagonal Prefix Sums (Prefix Sum on two diagonal directions).
47. 2D Difference Array (Prefix Sums).
48. 2D DP (Knapsack Variant)
49. 2D DP (Prefix Minimum)
50. 2D DP (Subarray XOR Triangle + Range Max)
51. 2D DP + 2D Prefix Sum (Suffix Sum)
52. 2D DP with Binary Search OR "Moves to Floors" Inverse DP.
53. 2D DP: Longest Common Subsequence (LCS)
54. 2D Dynamic Programming
55. 2D Dynamic Programming (Bottom-Up Tabulation).
56. 2D Dynamic Programming (Bottom-Up)
57. 2D Dynamic Programming (Bottom-Up).
58. 2D Dynamic Programming (dp[i][j])
59. 2D Dynamic Programming (HashMap/Array).
60. 2D Dynamic Programming (Interval DP)
61. 2D Dynamic Programming (Layered)
62. 2D Dynamic Programming (LCS Variant)
63. 2D Dynamic Programming (LCS).
64. 2D Dynamic Programming (LPS on word1 + word2).
65. 2D Dynamic Programming (Memoization).
66. 2D Dynamic Programming (Palindrome Table).
67. 2D Dynamic Programming (Path Counting)
68. 2D Dynamic Programming (Source Idx vs Pattern Idx)
69. 2D Dynamic Programming (Space Optimized)
70. 2D Dynamic Programming (State Switching)
71. 2D Dynamic Programming (State: Index, Last Value)
72. 2D Dynamic Programming (Suffix-Sum style)
73. 2D Dynamic Programming (Tracking mod pairs)
74. 2D Dynamic Programming + Hash Set.
75. 2D Dynamic Programming + Prefix Sums.
76. 2D Dynamic Programming / Memoization.
77. 2D Dynamic Programming with Memoization + Math Pruning (Probability limit).
78. 2D Dynamic Programming with Prefix Sum Optimization
79. 2D Dynamic Programming with Suffix Max Optimization (State: dp[num][diff]). (O(N * M))
80. 2D Dynamic Programming.
81. 2D Frequency Matrix (Counting)
82. 2D Interval Dynamic Programming

83. 2D Knapsack DP `dp[i][j]` is max strings using `i` zeros and `j` ones. Update backwards to save space.
84. 2D Prefix Sum (DP) + Quickselect/Min-Heap.
85. 2D Prefix Sum (In-Place)
86. 2D Prefix Sum (Inclusion-Exclusion Principle)
87. 2D Prefix Sum + 1D TreeSet Range Search
88. 2D Prefix Sum + 2D Difference Array.
89. 2D Prefix Sum + Binary Search (or Greedy Side Length)
90. 2D Prefix Sum Array
91. 2D Prefix Sum Matrix
92. 2D Prefix Sums (Factors of 2 and 5)
93. 2D Prefix Sums (Inclusion-Exclusion).
94. 2D Prefix Sums (Left and Up counts)
95. 2D Prefix Sums (Rows, Cols, and Diagonals).
96. 2D Prefix Sums + Binary Search for Radius, treating the "spiral rings" as concentric square boundaries.
97. 2D Prefix Sums + Hash Map / Binning.
98. 2D Prefix Sums / Precomputation of "Smaller" and "Greater" counts.
99. 2D Prefix/Suffix Max + Greedy
100. 2D Sliding Window / Submatrix Hashing
101. 2D-to-1D Reduction + Subarray Sum Logic.
102. 3-Way Dijkstra's Algorithm
103. 3D Dynamic Programming
104. 3D Dynamic Programming (Grid + Modulo State)
105. 3D Dynamic Programming (Index, Last Move, Score Diff)
106. 3D Dynamic Programming (Knapsack Variant)
107. 3D Dynamic Programming (Knapsack variant).
108. 3D Dynamic Programming (Knapsack with Sign State).
109. 3D Dynamic Programming (Memoization).
110. 3D Dynamic Programming (Simultaneous Path)
111. 3D Dynamic Programming (Spatial State).
112. 3D Dynamic Programming (State Machine)
113. 3D Dynamic Programming (State: i, j, kRemain). O(n^2 Â· k)
114. 3D Dynamic Programming (Top-Down with Memoization).
115. 3D Dynamic Programming (Top-Down)
116. 3D Dynamic Programming / Grid DP (State: row, col, neutralizations_left). O(M Â· N Â· K)
117. 3D Dynamic Programming / Memoization
118. 3D Dynamic Programming.
119. 5D Dynamic Programming + Sorting
120. 8-Directional Raycasting from Move
121. `CASE` statement or `IF` function + `MOD()` + `LIKE`.
122. Accumulating Closure Scopes (Each timeout creates a new closure, potentially keeping old variables in memory longer).
123. Adjacency List + BFS (Graph Conversion)

124. Adjacency Matrix + Degree Precomputation.
125. Adjacency Set Intersection (Triangle Counting)
126. Advanced Combinatorics + Inclusion-Exclusion
127. Advanced Digit DP + Precomputed Combinatorics Modulo 10^9+7. Find valid popcounts up to Length L, then use nCr to place bits.
128. Advanced Digit DP with Modulo/State constraints
129. Aggregate Comparison (HAVING Clause)
130. Aggregate Math (`MAX` - `MIN`)
131. Aggregated Subquery / IFNULL
132. Aho-Corasick + Greedy Jump / Segment Tree
133. Aho-Corasick / Trie + 1D Dynamic Programming
134. Algebraic Factorization (N = kx + frac{k(k-1)}{2}).
135. Algebraic Simplification + Binary Search / Fenwick Tree. Re-write the property as a range: arr[j] must be in [arr[i]/2, 2*arr[i]] .
136. Amortized Linear Scan / Precomputation
137. Angular Sweep / Fixed Circle Centers
138. Argument Object Cloning (Visualizing the memory used to copy the arguments list for every call).
139. Arithmetic Computation
140. Arithmetic Digit Extraction
141. Arithmetic Digit Summation
142. Arithmetic Progression (AP) Formula
143. Arithmetic Series Segmentation (Modulo Math)
144. Arithmetic Snapping / Modulo Math
145. Array Grouping by Set Bits + Min/Max Tracking
146. Array Indexing (Direct Access)
147. Array Length Property (`this.length`)
148. Array Math (Grade School Multiplication)
149. Array modification tracking (snapshotting the entire array state at each simulated step).
150. Array of 26 Stacks / Deques mapping available indices. (O(N))
151. Array of Linked Lists (Chaining)
152. ASCII Bit Manipulation / Offset
153. AWK Column Accumulation.
154. Backtracking (Constrained DFS)
155. Backtracking (Counter Only)
156. Backtracking (DFS + In-Place Mark)
157. Backtracking (DFS + Lexicographic)
158. Backtracking (DFS for DAG)
159. Backtracking (DFS)
160. Backtracking (DFS) / Pick & Skip State Tracking
161. Backtracking (DFS) with Time Pruning
162. Backtracking (Half-String Construction)
163. Backtracking (In-Place Swap)
164. Backtracking (Include/Exclude)

165. Backtracking (Keep or Abbreviate)
166. Backtracking (Row-by-Row + Bitsets)
167. Backtracking (Warnsdorffâs Rule optional)
168. Backtracking + Frequency Array.
169. Backtracking + Greedy (Place largest available number first).
170. Backtracking + Memoization (Top-Down DP)
171. Backtracking + Memoization (Top-Down DP).
172. Backtracking + Recursive State Validation
173. Backtracking + Sorting Groups.
174. Backtracking / DFS
175. Backtracking / Divide and Conquer.
176. Backtracking / Greedy (Small N)
177. Backtracking / Set
178. Backtracking DFS with Column-by-Column Pruning
179. Backtracking Iterate through the 10 LEDs, choosing to turn them on/off until `turnedOn` is met.
180. Backtracking on Net Balances Flatten everyone into their net balance. Try to settle one person's balance with any other person of the opposite sign.
181. Backtracking with Bi-directional Mapping
182. Backtracking with Coordinate Set Maintain a `visited` Set of `(row, col)`. Always try to clean, move, recurse, then return and face original direction.
183. Backtracking with DP/Memoization (pre-checking palindromes).
184. Backtracking with Early Pruning
185. Backtracking with Evaluation-on-the-fly
186. Backtracking with Fibonacci Pruning & Integer Overflow check.
187. Backtracking with Frequency State Management
188. Backtracking with Memoization (Bitmasking)
189. Backtracking with Memoization (DFS + Map).
190. Backtracking with Path Tracking
191. Backtracking with Pre-calculated Jump Table
192. Backtracking with Pruning
193. Backtracking with Pruning & Skyline State
194. Backtracking with Pruning + State Compression.
195. Backtracking with Pruning + Visited Array (or Bitmask).
196. Backtracking with Pruning / BFS for State Space
197. Backtracking with Pruning / Iterative Prefix Checking
198. Backtracking with Pruning / Network Flow.
199. Backtracking with Pruning.
200. Backtracking with State (Hamiltonian Path)
201. Backtracking with State Pruning (Bitmask)
202. Backtracking with Value Pruning.
203. Backtracking/Recursion or Bit Manipulation (Math: result is OR(all_elements) x 2^{n-1}).
204. Backward Pointer Scan
205. Backward Pointer Scan.

206. Backward Scan + Boolean Frequency Array
207. Backwards Traversal (Target to Start)
208. Balance Counter (Greedy)
209. Balanced Binary Search Tree (TreeSet) + Sliding Pointer
210. Balanced BST / TreeMap (Ordered Map)
211. Base 3 Representation (Ternary Logic).
212. Base-26 Conversion (with Offset Correction).
213. Base-9 Conversion.
214. Bellman-Ford / BFS (Level-based)
215. BFS (Breadth First Search) for Unweighted Shortest Path.
216. BFS (Breadth-First Search)
217. BFS (Breadth-First Search) + Visited Array (0-1000)
218. BFS (Digit Permutation Generation)
219. BFS (Level Order Traversal)
220. BFS (Level Order Traversal) but strictly Right-to-Left.
221. BFS (Level Order Traversal) tracking max per level.
222. BFS (Level Order Traversal) with an early exit on the first leaf node encountered.
223. BFS (Level Order Traversal).
224. BFS (Level-Order Expansion) or Bucket Sort.
225. BFS (Level-Order Traversal)
226. BFS (Level-Order Traversal).
227. BFS (Queue) Use a Queue. Process all nodes in the current queue (one level) before moving to the next.
228. BFS (Queue-based Level Order)
229. BFS (Reverse Level Order) + Hash Set
230. BFS (Shortest Path) Use a Queue. For each mutation, check if it's in the "bank". Use a visited set to avoid cycles.
231. BFS (Shortest Path) using a Queue and a Distance 2D Array / Dijkstra.
232. BFS (to find shortest distances) + DFS/Backtracking (to reconstruct paths).
233. BFS + Cycle Decomposition (Minimum Swaps)
234. BFS + Ordered Sets (Parity sets)
235. BFS + Priority Queue (Min-Heap / Max-Heap).
236. BFS + Priority Queue / Monotonic Stack
237. BFS / DFS (State Space Generation)
238. BFS / DFS Digit Construction
239. BFS / DFS with Edge Direction Tracking
240. BFS / DFS with Positional Indexing.
241. BFS / DFS with Visited State Set
242. BFS / Dijkstra (Day-by-Day)
243. BFS / Dijkstra on Grid State.
244. BFS / Dijkstra on Sorted Checkpoints.
245. BFS / Dijkstra on State Space (Grid + Bitmask).
246. BFS from Buildings (Multi-Source Distance Accumulation)
247. BFS from every node (or Edge Removal)
248. BFS on (CurrentNode, VisitedMask) State Space.

249. BFS on 3D State (row, col, k)
250. BFS on Busses/Routes (Bipartite-like Graph).
251. BFS on Matrix State (Bitmasking)
252. BFS on State (x, y, bitmask_of_keys).
253. BFS Precomputation + Bitmask DP
254. BFS Queue (Visualizing the search space widening with every time step).
255. BFS using a Queue.
256. BFS with a Double-ended Queue (Deque) and a boolean direction flag.
257. BFS with a Queue + Prepending to a Deque/Linked List for the result (or standard array reversal, which is practically optimal in many languages).
258. BFS with Character Mutation (a to z).
259. BFS with Multi-State Tracking
260. BFS with Multi-Value Jump Optimization
261. BFS with State Pruning (Shortest Path in String Graph).
262. BFS with Symmetry + Pruning (or Math)
263. BFS with Two-Condition Entry (Keys + Possession)
264. BFS/DFS with Hostname Filtering
265. BFS/DFS with Sliding Logic From a cell, move in one direction using a `while` loop until hitting a wall. Treat the *stop point* as a graph node.
266. Bi-directional Dynamic Programming (Meet-in-the-Middle) / Graph BFS. State = (u, v) .
267. Bi-directional Linear Scan with Last-Seen Map. O(N + Q)
268. Binary Exponentiation
269. Binary Exponentiation (Fast Power)
270. Binary Exponentiation (Modular Pow)
271. Binary Indexed Tree (Fenwick Tree) / Segment Tree
272. Binary Lifting (Sparse Table)
273. Binary Offset Mapping
274. Binary Search
275. Binary Search (bisect) on a sorted copy of nums1 to find the closest value to nums2[i].
276. Binary Search (Flattened Index)
277. Binary Search (if sorted) or Single Pass Min-Tracker.
278. Binary Search (Iterative Descent)
279. Binary Search (Left-most target)
280. Binary Search (Leftmost Boundary)
281. Binary Search (O(log N)).
282. Binary Search (on 1D projections of the 2D grid)
283. Binary Search (Range [0, x])
284. Binary Search (Range [1, num])
285. Binary Search (Range-based Gap Detection)
286. Binary Search (Range-Based)
287. Binary Search (Range-based).
288. Binary Search (Shrinking Bound)
289. Binary Search (Sorted Heaters) Sort heaters. For each house, binary search for

the two closest heaters (one left, one right).
290. Binary Search (Ternary Splitting)
291. Binary Search (Triple Pass).
292. Binary Search (Two Bounds)
293. Binary Search + Bit Counting Math
294. Binary Search + Greedy Priority Queue
295. Binary Search + Inclusion-Exclusion (PIE)
296. Binary Search + Max-Heap (Priority Queue)
297. Binary Search + Sort Store all `start` values in a sorted list with original indices. For each `end`, binary search the list.
298. Binary Search + Two Pointers.
299. Binary Search / Jump Search
300. Binary Search / Math Search for k such that frac{k(k+1)}{2} le N or use the quadratic formula directly.
301. Binary Search on Answer
302. Binary Search on Answer (k) + Two Pointers.
303. Binary Search on Answer (Range [1, max(piles)]).
304. Binary Search on Answer (Range)
305. Binary Search on Answer (Y-coordinate). O(N log(Range/Epsilon))
306. Binary Search on Answer + Bit Digit DP (Math)
307. Binary Search on Answer + Difference Array
308. Binary Search on Answer + Disjoint Set Union (DSU) / Connected Components Check.
309. Binary Search on Answer + Edge Cases
310. Binary Search on Answer + Fenwick Tree (BIT) / Merge Sort for Inversions.
311. Binary Search on Answer + Greedy
312. Binary Search on Answer + Greedy Math (Arithmetic Progression sum).
313. Binary Search on Answer + Greedy Traversal. O(N log(Max_Score))
314. Binary Search on Answer + Greedy Verification
315. Binary Search on Answer + Greedy with Monotonic Queue (Deque)
316. Binary Search on Answer + Greedy.
317. Binary Search on Answer + Greedy/Flow.
318. Binary Search on Answer + Inclusion-Exclusion Principle.
319. Binary Search on Answer + Intersection Checks
320. Binary Search on Answer + Pruned Backtracking.
321. Binary Search on Answer + Reversed Graph Traversal. O(E log W)
322. Binary Search on Answer + Sliding Window
323. Binary Search on Answer + Sliding Window / Greedy Check.
324. Binary Search on Answer + Sliding Window / Segment Tree.
325. Binary Search on Answer + Sliding Window.
326. Binary Search on Answer + Validation (Greedy/Bipartite)
327. Binary Search on Answer / Math
328. Binary Search on Answer / Priority Queue
329. Binary Search on Answer Binary search the "max sum" (Low: max(arr), High: sum(arr)). Greedily count chunks needed for each guess.

330. Binary Search on Answer Range
331. Binary Search on Answer Range + Inclusion-Exclusion Principle
332. Binary Search on Answer Range + Two Pointers/Binary Search for Counting
333. Binary Search on Answer Range.
334. Binary Search on Answer Space (Time).
335. Binary Search on Answer Space + Greedy Counting.
336. Binary Search on Answer Space + Number Theory (Inclusion-Exclusion Principle using LCM).
337. Binary Search on Answer.
338. Binary Search on Candidate Points
339. Binary Search on Columns/Rows
340. Binary Search on Cubic Formula
341. Binary Search on Days + BFS/DFS
342. Binary Search on Distance + Sliding Window
343. Binary Search on Even Indices.
344. Binary Search on H-Boundary
345. Binary Search on Leaf Nodes + Tree Height Properties
346. Binary Search on Length + Rolling Hash (Rabin-Karp).
347. Binary Search on Length + Rolling Hash (Rolling Hash)
348. Binary Search on Length The number of '1's can only be between 2 and 64. For each fixed length m, binary search for base k.
349. Binary Search on Monotonic Property
350. Binary Search on Partitions
351. Binary Search on Range + Upper Bound
352. Binary Search on Slope Direction.
353. Binary Search on Slope.
354. Binary Search on the Answer Space + Greedy Sliding Window.
355. Binary Search on the Answer.
356. Binary Search on the Answer. Guess a "Finish Time" T. Write a greedy O(N) checker to see if all rides can be completed within time T.
357. Binary Search on Value or Min-Heap Merge
358. Binary Search on Value Range
359. Binary Search on Value Threshold / Greedy Math
360. Binary Search with Duplicate Handling (O(log N) average, O(N) worst).
361. Binary Search.
362. Binary Tree Symmetry / Recursion
363. Bipartite Check + Component BFS Max Depth
364. Bipartite Coloring (Parity Counting)
365. Bipartite Graph Matching / Min Vertex Cover.
366. Bipartite Graph Maximum Matching (Kuhn's Algorithm / DFS with Backtracking).
367. Bit contribution counting array tracking 1s, coupled with right-shift carry propagation logic.
368. Bit Count Frequency + Inclusion-Exclusion
369. Bit Manipulation (bitCount) + Constant Set Check
370. Bit Manipulation (Bitmasking)

371. Bit Manipulation (Column-wise Bit Counting)
372. Bit Manipulation (Common Prefix Match)
373. Bit Manipulation (Exponential Subtraction)
374. Bit Manipulation (Extract and Shift).
375. Bit Manipulation (Flipping Trailing Ones)
376. Bit Manipulation (Greedy Bit-by-Bit)
377. Bit Manipulation (Left Shifting)
378. Bit Manipulation (Modulo 3 Summing).
379. Bit Manipulation (Popcount + Bit Length)
380. Bit Manipulation (Popcount) / Math
381. Bit Manipulation (Power of 2 - 1)
382. Bit Manipulation (Right Shift) + Distance Tracker.
383. Bit Manipulation (Shift and Add)
384. Bit Manipulation (XOR + Rightmost Set Bit)
385. Bit Manipulation (XOR Masking).
386. Bit Manipulation (XOR Shift).
387. Bit Manipulation (XOR) or Character Summation
388. Bit Manipulation (XOR).
389. Bit Manipulation (XOR, AND)
390. Bit Manipulation (Zero-Bit Filling)
391. Bit Manipulation + Masking
392. Bit Manipulation / Math
393. Bit Manipulation Group every 4 bits using bitwise AND (num & 15), then right shift by 4 (num >>> 4).
394. Bit Manipulation: (n > 0) and (n & (n-1) == 0)
395. Bit-by-Bit Counting / Mathematical Combinatorics.
396. Bit-by-Bit Counting For each bit position (0-31), count how many numbers have a '1' (k) and how many have a '0' (n-k). Distance contribution is k x (n-k).
397. Bit-by-Bit Reconstruction
398. Bit-Counting + Greedy Reconstruction
399. Bit-Position Counting
400. Bitmask + Backtracking Find character differences as bitmasks. Use DFS to find the shortest abbreviation mask that covers all difference constraints.
401. Bitmask + Prefix XOR
402. Bitmask + Prefix XOR (First Occurrence)
403. Bitmask / Binary Enumeration
404. Bitmask / Profile DP
405. Bitmask DP
406. Bitmask DP (Submask Iteration)
407. Bitmask DP (Ternary/Base-3 or Base-2 with double slots).
408. Bitmask DP (TSP Variant)
409. Bitmask DP + Floyd-Warshall
410. Bitmask DP + Sorting Values
411. Bitmask DP + State Compression.
412. Bitmask DP + Submask Iteration

413. Bitmask DP / Backtracking with State Compression.
414. Bitmask DP / State Compression.
415. Bitmask DP over Target Array + LCM. O(N x 2^|target|)
416. Bitmask Dynamic Programming (DP on Subsets).
417. Bitmask Dynamic Programming (Inverted)
418. Bitmask Dynamic Programming (Knapsack Variant)
419. Bitmask Dynamic Programming (Primes as Bits).
420. Bitmask Dynamic Programming + GCD Precomputation.
421. Bitmask Dynamic Programming.
422. Bitmask Enumeration + Precomputed Palindromes
423. Bitmasking (Bitwise AND Comparison)
424. Bitmasking (Brute Force is the only way, but Bitmasking is the production-ready implementation).
425. Bitmasking + Array.
426. Bitmasking + Backtracking (Pruned)
427. Bitmasking + Disjoint Set Union (DSU).
428. Bitmasking + Gosper's Hack
429. Bitmasking + Hash Map / Advanced State Tracking
430. Bitmasking + Prefix XOR + Frequency Hash Map
431. Bitmasking + Subset Enumeration
432. Bitmasking / Backtracking Iterator
433. Bitmasking / Set Hashing
434. Bitmasking / Sorting + Hash Set.
435. Bitmasking / State Space Search
436. Bitmasking for character presence + Hash Map frequency counting.
437. Bitmasking Subset Enumeration + DFS / Floyd-Warshall
438. Bitwise AND (&) Operator in SELECT/WHERE
439. Bitwise AND Properties / Hash Set
440. Bitwise Evaluation / Prefix Counts.
441. Bitwise Logic (Total XOR Property).
442. Bitwise Logic / CASE statement with ID manipulation.
443. Bitwise Mask XOR Find the highest set bit. Create a mask of all 1s of that length. Result is `num ^ mask`.
444. Bitwise Operations & Masking
445. Bitwise OR (Math / Bit Manipulation Insight)
446. Bitwise Parity Counting
447. Bitwise Parity Math / Combinatorics
448. Bitwise Powers of 2 Math.
449. Bitwise Prefix XOR & Frequency Counting.
450. Bitwise Shift & Subtract
451. Bitwise Shift and Masking.
452. Bitwise Simulation (Carry Manipulation).
453. Bitwise Trie Insert all numbers into a Binary Trie. For each number, traverse the trie trying to pick the opposite bit.
454. Bitwise XOR (Sum) & AND-Shift (Carry) Loop

455. Bitwise XOR + Bit Counting XOR the two numbers (x oplus y). Count the set bits using Brian Kernighanâs algorithm.
456. Bitwise XOR + Brian Kernighan's Alg
457. Bitwise XOR Properties (`A ^ B = C` implies `A ^ C = B`).
458. Bitwise XOR Properties + Hamming Distance
459. Blocking Execution Context (One task occupies the stack while others wait in the queue).
460. Blocking Main Thread (Visualizing the CPU stuck in a loop waiting for the async task).
461. Boolean Array + Step Increment
462. Boolean Array Masking
463. Boolean Frequency Array / Hash Set
464. Boolean Indexing: df[df['id'] == 101]
465. Boolean Mask + Interval Merging.
466. Border-to-Interior DFS (Inverse Flood Fill).
467. Bottom-Up DFS (Post-order Traversal)
468. Bottom-Up DFS (Post-order Traversal) with a Global Max tracker.
469. Bottom-Up DFS (Post-order Traversal).
470. Bottom-Up DFS (returning pairs).
471. Bottom-Up DFS. Return the height of the subtree if it is balanced. If it is ever unbalanced, return -1 immediately to short-circuit.
472. Bottom-Up Dynamic Programming (In-place or rolling 1D array).
473. Bottom-Up Dynamic Programming (Reverse Tabulation).
474. Bottom-Up Greedy (Post-order Traversal)
475. Bottom-Up Post-Order DFS
476. Bottom-Up Post-Order DFS (Tree DP)
477. Bottom-Up Tree DP (Post-order Traversal).
478. Boundary Analysis (Sorting).
479. Boundary Contraction
480. Boundary Shrinking
481. Bounded BFS / Geometric Limitation.
482. Bounded Knapsack Dynamic Programming.
483. Boyer-Moore Voting + Random Sampling / Segment Tree
484. Boyer-Moore Voting Algorithm (Extended)
485. Boyer-Moore Voting Algorithm.
486. Brain Teaser / Math (Length Comparison).
487. Brain Teaser / Math (Parity).
488. Brainteaser / Array Math (Ghost Ants)
489. Brainteaser / Logical Deduction
490. Breadth-First Search (BFS)
491. Breadth-First Search (BFS) + Mathematical Formula
492. Breadth-First Search (BFS) / Bidirectional BFS
493. Breadth-First Search (BFS) / Level-order traversal with a Visited HashSet.
494. Breadth-First Search (BFS) / Shortest Path
495. Breadth-First Search (BFS) Shortest Path on a 3D State Graph

496. Breadth-First Search (BFS) String Building + Frequency Pruning
497. Breadth-First Search (BFS) with Column Tracking (Hash Map)
498. Brian Kernighanâs Algorithm (n & (n-1)).
499. Brute Force (Small Constraints)
500. Brute Force (Small Constraints) / Trie (Optimal)
501. Brute Force / Constraint Optimization
502. Brute Force Path Memory (Storing every conceivable route configuration simultaneously).
503. Brute Force with Early Exit (Small Constraints)
504. BST Property Navigation (Binary Search).
505. BST Property Navigation (Succ = smallest of right subtree)
506. BST Property Traversal (Value Comparison)
507. BST Traversal (Iterative)
508. BST Traversal leveraging Parent Pointers.
509. Bucket Sort (Counting Citations)
510. Bucket Sort (Pigeonhole Principle).
511. Bucket Sort / Counting Tally
512. Bucket Sort / Frequency Map + Greedy Flattening
513. Bucket Sort / Greedy with Priority.
514. Bucket Sort / Prefix Sum
515. Bucket Sort / Sliding Window
516. Bucket Sort of Iterators / Waiting Lists
517. Bucket Sort or Min-Heap
518. Bucket Sort Use an array where the index is the frequency. Put characters into the bucket at `frequency[char]` index.
519. Buffer Scan (Visualizing the database engine reading the text column repeatedly for each word).
520. Buffered Chunk Processing with Bound Checking.
521. Built-in Bit Manipulation (popcount)
522. Built-in O(N log N) Sort with a strictly defined Custom Comparator function returning -1, 0, or 1 based on a prioritized rule cascade.
523. Calculate average, throw array into a Hash Set, and start checking from ceil(avg) + 1 .
524. Call Stack & Arguments Object frame allocation.
525. Call Stack & Redundant Object Allocation (Draw stacked boxes for deep recursion + identical clone structures in heap).
526. Canonical Hashing (DFS + Sort)
527. Cantor's Diagonalization
528. Cascading / Bitmasking
529. Case When + Correlated Subquery (or In/Not In)
530. CASE WHEN + CROSS JOIN (Static Ranges)
531. CASE WHEN + SUM (Pivoting)
532. CASE WHEN / Aggregate Filtering
533. Case When Logic (Geometric Constraints)
534. CASE WHEN with IN clause (Subquery).

535. Case-Based Boundary Analysis. O(n)
536. Case-Based Comparison (Counter + Mismatch Pointer).
537. CEIL + SUM + LEFT JOIN
538. Center Point + Diagonal Property
539. Chained Operations: df.loc[...].sort_values(...)[['name']]
540. Chaining (Array of Linked Lists / BSTs)
541. Character Frequency Counting + Greedy Symmetric Placement.
542. Character Frequency Signatures (Even/Odd split).
543. Chebyshev Distance (Math/Geometry)
544. Chebyshev Distance (Max of ÎX, ÎY)
545. Chunked Transfer (Length + Delimiter)
546. Circular Buffer (Fixed-size Array)
547. Circular Buffer / Fixed-size Array with Head/Tail Pointers.
548. Closure + clearTimeout / setTimeout
549. Closure + Hash Map (Caching)
550. Closure + setTimeout + Stored Arguments
551. Closure / Higher-Order Function
552. Closures / Factory Pattern
553. Column-Wise Linear Scan
554. Column-Wise Max Pre-calculation
555. Combinatorial Construction. Generate palindromes directly by mirroring the "first half" of the binary bits.
556. Combinatorial Counting (Pascal's Triangle logic)
557. Combinatorial Counting + Digit-by-Digit Construction. O(n)
558. Combinatorial Row Storage (Visualizing all row combinations in memory).
559. Combinatorics (Cantor Expansion) + Modular Multiplicative Inverse + Precomputed Factorials.
560. Combinatorics (Contribution of Each Element)
561. Combinatorics (Handshake Lemma)
562. Combinatorics (Inclusion-Exclusion Principle)
563. Combinatorics (nCr mod10) using Lucas Theorem & CRT. O(N)
564. Combinatorics (nCr) + Bit-by-Bit Construction.
565. Combinatorics (nCr) + Fast Power Math. (O(N))
566. Combinatorics (Pascal's Triangle / Math)
567. Combinatorics (Permutations with Repetition) + Modular Multiplicative Inverse.
568. Combinatorics (Permutations)
569. Combinatorics (Stars and Bars)
570. Combinatorics + Fenwick Tree (BIT)
571. Combinatorics + Frequency Counting.
572. Combinatorics + Modular Inverse
573. Combinatorics + Prefix/Suffix Maps
574. Combinatorics / 4-State DP (Markov Chain approach)
575. Combinatorics / DP (Parity Switch)
576. Combinatorics / Gap Counting
577. Combinatorics / Greedy Math.

578. Combinatorics / Hash Map
579. Combinatorics / Math (DP)
580. Common Denominator (LCM) + Final GCD.
581. Common Table Expression (CTE) + LEFT JOIN filtering.
582. Common Table Expression (CTE) / GROUP BY
583. Common Table Expressions (CTE) + Aggregation
584. Common Table Expressions (CTE) + Window Functions (RANK/DENSE_RANK)
585. Common Table Expressions (CTEs) + GROUP BY aggregation + Self JOIN.
586. Common Table Expressions (CTEs) to group by employee_id, year, week , aggregate sums, filter > 20 , then self-join.
587. Compare Current vs Next
588. Complement Counting (Total - Good)
589. Complementary Counting (Total - Odd)
590. Complementary Counting + Binary Exponentiation.
591. Component Labeling (Island ID) + Hash Map.
592. Concurrency: 3 Semaphores
593. Concurrency: Mutexes / Semaphores / Promises
594. Concurrency: Two Alternating Semaphores
595. Condition Variables / Semaphores
596. Conditional Aggregation (SUM + CASE)
597. Conditional Aggregation (SUM + IF/CASE)
598. Connected Components (BFS/DFS) + Global Totals
599. Connected Components (DFS/BFS) + Combinatorics
600. Constrained Backtracking / DFS
601. Constructive Algorithm / Greedy Pattern Filling.
602. Constructive Pattern / Mathematical Observation
603. Contribution of Each Character (Index tracking).
604. Contribution Technique (Position-wise Counting)
605. Controlled Stack-based DFS (Lazy Loading).
606. Convex Hull + 3 Pointers (Shoelace Formula).
607. Convex Hull + Rotating Calipers.
608. Coordinate Compression + Segment Tree / Sparse Map.
609. Coordinate Compression / Line Sweep
610. Coordinate Compression / Sweep Line Map
611. Coordinate Geometry + Single Pass Accumulation.
612. Coordinate Mapping (Direct Indexing).
613. Coordinate Mapping (Precomputation)
614. Coordinate Math (i, j) Simulation
615. Coordinate Math (Manhattan Distance Pathing)
616. Coordinate Min-Max Comparison
617. Coordinate Rotation (Chebyshev Transformation)
618. Coordinate Sorting + Monotonic Optimization
619. Corner Count Set + Area Summation
620. Count Factors of 5 (Legendre's Formula).
621. COUNT(DISTINCT customer_id)

622. COUNT(DISTINCT) with GROUP BY
623. Counting & Filling (Stable Parity). O(N)
624. Counting + Combinatorics (Three Cases)
625. Counting / Frequency (Linear Scan)
626. Counting 1s + Suffix Matching
627. Counting Sort (Frequency Map)
628. Counting Sort / Frequency Array
629. CROSS JOIN
630. CROSS JOIN (Cartesian Base) + LEFT JOIN
631. Cross Join (Master Template) + Left Join
632. CROSS JOIN (Template) + LEFT JOIN
633. CROSS JOIN / SELF JOIN
634. Cross Join with Duplicate Filtering (Upper Triangular Check).
635. Cross Product (Orientation) For every three consecutive points, calculate the cross product. All cross products must have the same sign (all positive or all negative).
636. Cross-Multiplication (Slope Equality)
637. CTE + Interval Merging (Sweep Line SQL)
638. CTE + Window Functions (RANK)
639. CTE / UNION for Bidirectional Friends + JOIN
640. CTE / Window Functions
641. CTE Aggregation + Window Function (LEAD)
642. CTE with `COUNT(*) OVER(PARTITION BY salary)` + `DENSE_RANK()`.
643. CTEs + Not In / Except (Set Difference)
644. Custom Comparator / Lambda Sorting.
645. Custom Comparator / Partitioning
646. Custom Comparator with __builtin_popcount
647. Custom Comparator with Bitwise Manipulation
648. Custom Greedy Sorting
649. Custom Greedy Sorting (Comparator: S1+S2 vs S2+S1).
650. Custom Sort (Length then Lexicographical)
651. Custom Sort + Two Pointers (Subsequence Check).
652. Cycle Decomposition (Permutation Graph).
653. Cycle Decomposition (Permutation Theory).
654. Cycle Detection (Hash Map)
655. Cycle Detection (Pigeonhole) Record the `index` in s_2 after each s_1. If the same index repeats, a cycle is found. Calculate remainder math.
656. Cyclic Sort (Index as Map)
657. Date Filtering + Grouping
658. Datetime library or Formula-based difference. `(Date2 - Date1).days * rate`.
659. Days Since Epoch (1971) Calculation
660. Deep Call Stack (Visualize memory frames for every nested level of the JSON structure).
661. Deep Call Stack Grid (Showing massive stack frames for O(N) depth and O(2^N) repeated states).

662. Deep Recursion Stack (Visualizing maximum stack depth O(N) but an exploding number of string copies across states).
663. Degree Array + Edge Adjacency Matrix
664. Denary Tree Traversal Visualize numbers as a 10-ary tree (1, 10, 100...). Calculate "steps" between prefixes to skip whole subtrees.
665. Depth-First Search (DFS) / BFS
666. Depth-First Search (DFS) or Breadth-First Search (BFS)
667. Deque + HashSet (Constant-Time Movement and Collision)
668. Descending Order Sorting or Single-Pass Mapping.
669. Deterministic Finite Automaton (DFA)
670. Deterministic One-Pass (DFA)
671. df.drop_duplicates(subset=['id'])
672. df.dropna(subset=['name'])
673. df.head(3) (Slicing)
674. df.melt(id_vars=['product'], ...)
675. df.pivot(index='month', columns='city', values='temperature')
676. df.rename(columns={...})
677. df.shape Attribute Access
678. df['grade'].astype(int)
679. df['quantity'].fillna(0)
680. df['salary'] *= 2 (Vectorized Math)
681. DFS (Leaf-only Extraction).
682. DFS (Parent Tracking) + Pointer Manipulation
683. DFS (Post-order Traversal) with Existence Flags
684. DFS (Post-order) + Target Set
685. DFS (Pre-order Traversal of a 10-ary Tree)
686. DFS (Pre-order) with target subtraction (passing `target - node.val` down the recursion).
687. DFS (to Map) + BFS (to Find Shortest Path).
688. DFS + Ancestor Hash Map
689. DFS + Ancestor Value Tracking (Value-based Map).
690. DFS + Backtracking (Parent Tracking)
691. DFS + Bit Manipulation
692. DFS + Combinatorics (Product of Subtree Counts)
693. DFS + Degree Counting
694. DFS + Fenwick Tree (BIT) / Merge Sort Tree.
695. DFS + Frequency Array (Counting)
696. DFS + Hash Set.
697. DFS + Memoization (or Sorting + DP)
698. DFS + Memoization (Topological Sort Pattern)
699. DFS + Memoization + Pruning Use string compression (remove 3+ matches) and a Map to store "State: MinBalls". Skip redundant placements.
700. DFS + Memoization / DP (State: r, c, dir, turn_taken, next_val). O(N Â· M)
701. DFS + Path Serialization.
702. DFS + Precomputation (Level-wise Max Depths)

703. DFS + Sliding Window on Path + Hash Map for Last Depth. O(N)
704. DFS + Subtree Size Math
705. DFS + Trie (Bitwise)
706. DFS / BFS (Parental Context)
707. DFS / BFS + Grid Traversal (Visited tracking)
708. DFS / BFS from the click point with strict boundary/number checks.
709. DFS / BFS or Union-Find (Disjoint Set Union).
710. DFS / BFS with Connectivity Logic.
711. DFS / BFS with Node Mapping (Memoization)
712. DFS / Preorder Traversal + Hash Set
713. DFS mapping (to reveal grid) + Dijkstra's Algorithm (Shortest Path).
714. DFS Post-order Traversal + Tracking Node with Value 1
715. DFS Recursion (Post-Order)
716. DFS Subtree Precomputation + Ancestor Check
717. DFS Tree Flattening + Manacher's Algorithm
718. DFS with Arithmetic Accumulation.
719. DFS with Backtracking
720. DFS with Backtracking / String Builder
721. DFS with Backtracking. Maintain a single shared path list: `push` node, recurse, `pop` node.
722. DFS with Height-Based Grouping
723. DFS with In-place Sink (or BFS/Union-Find).
724. DFS with In-Place Sinking (Visited marking).
725. DFS with Local Set Use backtracking. At each recursive level, use a `HashSet` to skip duplicate numbers to avoid redundant subsequences.
726. DFS with Max-So-Far Parameter
727. DFS with Memoization / Dependency Graph Resolve. O(Final Length)
728. DFS with Path Tracking (Bottom-Up Logic)
729. DFS with Probability Propagation
730. DFS with Pruning / Bitmask DP Sort matchsticks descending. Try placing in a side; if it exceeds `target`, backtrack immediately.
731. DFS with Range Tracking (Top-Down).
732. DFS with Running Sum/State
733. DFS with Sliding Window on Tree Paths + Hash Map. O(N)
734. DFS with State Marking (3-Coloring).
735. DFS with Subtree Size Precomputation
736. DFS with Timestamping (Kahnâs or Visited Array)
737. DFS/BFS Traversal (Same as Brute force, as you must visit every node to check).
738. DFS/BFS with a Hash Map (Mapping Original Nodes to Cloned Nodes).
739. DFS/BFS with Symmetrical Dual-Pointers
740. Diagonal Indexing (i - j) + HashMap/Bucket Sort. O(NÂ²)
741. Diagonal Iteration + Hash Set
742. Diagonal Point Matching + Hashset
743. Difference Array (Line Sweep)
744. Difference Array (Prefix Sum of Queries)

745. Difference Array (Prefix Sum of Shifts)
746. Difference Array (Prefix Sum Optimization)
747. Difference Array (Sweep Line)
748. Difference Array (Sweep Line) or Fenwick Tree (Binary Indexed Tree) to mathematically track shifts.
749. Difference Array + Greedy (Sweep Line)
750. Difference Array + Greedy Peak Tracking.
751. Difference Array / Greedy
752. Difference Array / Greedy Peaks
753. Difference Array / Lazy Propagation (Segment Tree).
754. Difference Array / Line Sweep.
755. Difference Array / Prefix Sum
756. Difference Array / Prefix Sum (Line Sweep).
757. Difference Array / Prefix Sums
758. Difference Array / Prefix Sums.
759. Difference Array / Sweep Line
760. Difference Array / Sweep Line Algorithm
761. Difference Array String Hashing + Frequency Map.
762. Digit DP (Combinatorial Counting).
763. Digit DP (Combinatorial Math)
764. Digit DP (Dynamic Programming)
765. Digit DP (Dynamic Programming) / Combinatorics.
766. Digit DP + KMP (Automata State)
767. Digit DP + Precomputed Sieve
768. Digit DP / Arithmetic Progression Formula.
769. Digit DP / Combinatorics.
770. Digit DP / Combinatorics. Precompute valid target "set bit counts", then build valid numbers bit by bit using combinations (nCr).
771. Digit DP / Constructive Greedy
772. Digit DP / Fibonacci Pattern.
773. Digit DP / Mathematical Counting (Digit-by-Digit)
774. Digit DP with Remainder State
775. Digit Dynamic Programming
776. Digit Dynamic Programming (Bitmask)
777. Digit Dynamic Programming (Digit DP)
778. Digit Dynamic Programming (State: index, tight, productMod, sum). O(digits Â· sum Â· prod)
779. Digit Dynamic Programming.
780. Digit Extraction (Modulo Arithmetic)
781. Digit Extraction (Pop & Push)
782. Digit Frequency Counting (Anagram Check).
783. Digit Manipulation + Hash Set
784. Digit-based DFS / Backtracking.
785. Digit-by-Digit Math w/ Carry
786. Digital Root (Mathematical Modulo 9)

787. Dijkstra (from Node N) + DAG DP (Topological Sort/Memoization).
788. Dijkstra + Auxiliary "Teleport Hub" nodes. Instead of V x V edges, connect all nodes to one "Hub" with specific entry/exit costs.
789. Dijkstra + DP (Counting)
790. Dijkstra / BFS + A* Search or Dynamic Programming.
791. Dijkstra / Binary Search + BFS
792. Dijkstra with State (City, DiscountsUsed)
793. Dijkstra with String Priority Use a Priority Queue. Store `(distance, path_string, x, y)`. Only update if distance is smaller OR distance is equal but string is lexicographically smaller.
794. Dijkstra's (Bi-directional) + Edge Relaxation
795. Dijkstra's Algorithm
796. Dijkstra's Algorithm (with Priority Queue)
797. Dijkstra's Algorithm + Parity Waiting Logic.
798. Dijkstra's Algorithm with Bitmask DP (State Space Search). State = (Current_Node, Visited_Mask) .
799. Dijkstra's on a Coordinate Graph
800. Dijkstra's x3 (Steiner Tree on 3 Nodes variant).
801. Dijkstraâs Algorithm (Min-Priority Queue)
802. Dijkstraâs Algorithm + Edge Coverage Math.
803. Dijkstraâs Algorithm + Prime Sieve
804. Dimension Reduction + Interval Merging
805. Dimension Sorting + LIS (Dynamic Programming)
806. Direct Accumulation (Math optional)
807. Direct Array Placement / Cyclic Sort
808. Direct Comparison / O(1) Access
809. Direct Digit Extraction (Modulo/Div)
810. Direct Hash Map / ASCII Array Indexing.
811. Direct Index Mapping (Modulo Arithmetic).
812. Direct Index Mapping (One-Pass)
813. Direct Indexing Compare `words[i].charAt(j)` with `words[j].charAt(i)` with boundary checks.
814. Direct Integer Accumulation
815. Direct Linear Scan with Set Lookup.
816. Direct Mapping + Pairwise Check
817. Direct Math (Division & Min). O(1)
818. Direct Math (Positional Notation).
819. Direct Math Operations / ALU
820. Direct SQL Filtering (Math inside WHERE clause)
821. Direct SQL WHERE Clause + ORDER BY.
822. Direct String Transformation + Regex
823. Directed Acyclic Graph (DAG) - Count In-Degrees
824. Directed Acyclic Graph (DAG) with Topological Updates.
825. Direction Array + Matrix Simulation
826. Directional Ray Casting + Primality Sieve

827. Disjoint Set Union (DSU)
828. Disjoint Set Union (DSU) + Sorting by Value
829. Disjoint Set Union (DSU) / Connected Components
830. Disjoint Set Union (DSU) / Connected Components + Hash Maps.
831. Disjoint Set Union (DSU) / Connected Components + Sorting
832. Disjoint Set Union (DSU) / Connected Components.
833. Disjoint Set Union (DSU) / Kruskal's Variant with DP.
834. Disjoint Set Union (DSU) / Union-Find
835. Disjoint Set Union (DSU) or Validated BFS
836. Disjoint Set Union (DSU) with Path Compression. For every session connection, union the two nodes and decrement the total cluster count.
837. Disjoint Set Union (DSU) with Restrictions Check
838. Disjoint Set Union (Union-Find)
839. Disposable HashSets (Draw hundreds of temporary Sets being created and garbage-collected for every individual cell).
840. Distance Array + Priority Queue (Drawing the standard Dijkstra memory footprint).
841. Distance Sorting / Hash Set of Lengths.
842. Distinct Counting + Null Handling.
843. Divide & Conquer + Combinatorics (Pascal's Triangle)
844. Divide & Conquer Recursively check if a quadrant is "leaf". If all 4 children are leaves and same value, merge.
845. Divide and Conquer (Odd/Even Separation)
846. Divide and Conquer (Quadtree Decomposition)
847. Divide and Conquer (Recursive Splitting).
848. Divide and Conquer (Split on Invalid Characters)
849. Divide and Conquer + Memoization (Top-Down DP)
850. Divide and Conquer / DFS. Find mid, make root, recurse left half, recurse right half.
851. Divide and Conquer / Sweep Line
852. Divide and Conquer or Segment Tree with "Monotonic Property" optimization. Focus on boundaries where the XOR value changes (at most 32 times).
853. Divisor Counting + Inclusion-Exclusion
854. Divisor Iteration + Freq Map Comparison
855. Divisor Search + Post-order DFS
856. Double Array Concatenation + Monotonic Queue/DP
857. Double BFS (or DFS with Height Tracking)
858. Double BFS / DFS
859. Double Binary Search
860. Double DFS / Max Flow Min Cut (Path Neutralization).
861. Double Ended Queue (Deque) / Two Pointers
862. Double Hash Maps (Count of Counts)
863. Double In-place Reversal (Reverse Whole + Reverse Words).
864. Double JOIN + CASE Statement
865. Double Logarithmic Loop (Logarithmic Bounds)

866. Double Loop with Square Root Check
867. Double Topological Sort (Kahn's Algorithm)
868. Double-Ended Queue (Deque) + Running Sum
869. Doubly Linked List + TreeMap (or Lazy Deletion PQ)
870. Doubly Linked List w/ Dummy Nodes
871. DP (1D Calendar / Days-based)
872. DP (1D/2D) `dp[i][j]` stores max score diff from subarray i...j. `dp[i][j] = max(nums[i]-dp[i+1][j], nums[j]-dp[i][j-1])`.
873. DP (Difference as State)
874. DP (Hash Map of Sets) Map each stone to a Set of jump lengths that reached it.
875. DP (Index, Remaining K, Last Parity)
876. DP (Interval 3D)
877. DP (Max Length per Char) Track the longest "consecutive" sequence ending at each character ('a' through 'z'). The sum of these 26 values is the result.
878. DP (Sorted Robots & Factories)
879. DP (Stirling Numbers of Second Kind)
880. DP (Subset Sum) Transform to finding a subset with sum (S + target)/2. Solve using 1D Knapsack DP.
881. DP + BFS / Optimized Recursion
882. DP + Binary Search (Weighted Interval Scheduling).
883. DP + Bitmask
884. DP + Bitset (Bit-Level Parallelism)
885. DP + Character Frequency Pre-computation
886. DP + Combinatorics (Digits, Count, Sum)
887. DP + Freq Map + Two Pointers
888. DP + Generating Functions / Polynomial Interpolation or NTT.
889. DP + Global Max Tracking (O(Nk))
890. DP + Largest Histogram (Q84)
891. DP + LCP (Suffix) + Prefix Sums
892. DP + Longest Common Prefix (LCP) with Rolling Hash or Suffix Array
893. DP + Median Distance Pre-computation
894. DP + Median Properties / Slopes
895. DP + Memoization (State: zeros, ones, last_bit)
896. DP + Memoization + Bitwise AND
897. DP + Memoization / Combinatorics
898. DP + Memoization or Monotonic Stack Sort
899. DP + Monotonic Deque (Sliding Window Maximum)
900. DP + Monotonic Deque / Sliding Window.
901. DP + Monotonic Queue (Deque)
902. DP + Monotonic Queue (Sliding Window)
903. DP + Prefix Sum Optimization
904. DP + Prime Factor Bitmasking
905. DP + Segment Tree or Fenwick Tree. Use a coordinate-compressed Segment Tree to query the sum of `Up/Down` counts for all values smaller/larger than the current element in O(log N).

906. DP + Sliding Window (Prefix Sums)
907. DP + Sorting + Binary Search
908. DP + Ternary Bitmask (State Compression)
909. DP / BFS (Visited Grid)
910. DP / Bitset Optimization
911. DP / Center Expansion + Greedy
912. DP / Central Expansion (Mismatch Count)
913. DP / Contribution of Each Character
914. DP / Middle-Element Counting
915. DP / Pre-calculated Jumps Join sentence with spaces. At each screen column start, pre-calculate which word the next row will start with.
916. DP on Strings / Bitmasking
917. DP with Bitmask
918. DP with Bitmask (Prime Representation)
919. DP with Bitmask (State: Mask of completed tasks)
920. DP with Bitmask / Memoization.
921. DP with Bitmasking.
922. DP with Bitset (Prefix/Suffix ORs)
923. DP with Bitset Optimization
924. DP with Convex Hull Trick (CHT) or prefix sum optimization. O(N^2)
925. DP with Frequency & Sum Mapping
926. DP with Frequency Map + Sliding Window Optimization
927. DP with Frequency Partitioning (i pmod K).
928. DP with Hash Map + Sorting.
929. DP with Interval Grouping (Bucket Sort style)
930. DP with Interval Precomputation
931. DP with Left/Right Prefix Max
932. DP with Lexicographical Greedy Path. O(26 x N)
933. DP with Map Matrix Use an array of HashMaps `dp[i]`, where `dp[i][diff]` stores count of slices ending at i with difference `diff`.
934. DP with Memoization (Suffix Sums)
935. DP with Minimum Tracking (Two-Minima Optimization)
936. DP with Obstacle Guard
937. DP with Prefix Sum Optimization + "Ongoing" State. O(n Â· k)
938. DP with Pruning (Capped Set size).
939. DP with Remainder State for each Divisor (1-9). O(10 Â· 10 Â· N)
940. DP with Sliding Window / Prefix-Suffix Max.
941. DP with Sliding Window Sum.
942. DP with State (lastRemainingIndex, nextIndex). O(N^2)
943. DP with Two Minimums (Optimization)
944. DSU (Disjoint Set Union) + Backtracking + Sorting
945. DSU + In-degree Tracking.
946. DSU + Multiples (Sieve-like)
947. Dual Frequency Array Comparison
948. Dual Frequency Maps (Visualizing two HashMaps being fully rebuilt for every

possible split point).
949. Dual Hash Maps (Count of Counts)
950. Dual Hash Maps (Frequency Tracking)
951. Dual Hash Maps (In-Transit & Completed)
952. Dual-Map Bijection or Pattern Normalization.
953. Dual-Pass Dijkstra (Target Distance Matching)
954. Dummy Node + Linear Traversal
955. Dutch National Flag (3-Pointer)
956. Dynamic BFS / Incremental DP
957. Dynamic Prefix Maximum & Max-Difference Tracking
958. Dynamic Programming (0/1 Knapsack variation).
959. Dynamic Programming (1D Array).
960. Dynamic Programming (1D)
961. Dynamic Programming (1D) or BFS
962. Dynamic Programming (1D).
963. Dynamic Programming (1D/2D)
964. Dynamic Programming (2D Range DP)
965. Dynamic Programming (2D Table or 1D Optimized)
966. Dynamic Programming (2D Table)
967. Dynamic Programming (2D)
968. Dynamic Programming (2D) or Two-Pointer Slide
969. Dynamic Programming (2D/3D).
970. Dynamic Programming (3-State Machine)
971. Dynamic Programming (4-Directional Prefix)
972. Dynamic Programming (4-state variables) or Prefix/Suffix Profit Arrays.
973. Dynamic Programming (Base-X Representation)
974. Dynamic Programming (Bellman-Ford variant).
975. Dynamic Programming (Bi-directional Kadane's)
976. Dynamic Programming (Bottom-Up 1D)
977. Dynamic Programming (Bottom-Up)
978. Dynamic Programming (Catalan Number Variation)
979. Dynamic Programming (Catalan Number)
980. Dynamic Programming (Column by Column)
981. Dynamic Programming (Column State Transitions)
982. Dynamic Programming (Divide and Conquer with Memoization)
983. Dynamic Programming (DP Table with State).
984. Dynamic Programming (DP with Hash Table)
985. Dynamic Programming (DP) tracking the minimum and maximum possible sum of `1s - 0s` at each cell.
986. Dynamic Programming (DP). `dp[i][direction]` = number of zigzags ending at index i going "up" or "down".
987. Dynamic Programming (Ending-character Tally)
988. Dynamic Programming (Frequency Transitions)
989. Dynamic Programming (Generating Functions Reconstruction).
990. Dynamic Programming (Histogram Heights) + Greedy Sorting.

991. Dynamic Programming (House Robber Variation)
992. Dynamic Programming (In-place or 1D)
993. Dynamic Programming (In-place)
994. Dynamic Programming (In-Place)
995. Dynamic Programming (Interval DP).
996. Dynamic Programming (Interval DP). `dp[i][j]` stores whether the subarray from i to j can be transformed into the target.
997. Dynamic Programming (Iterative Space Optimized)
998. Dynamic Programming (Kadane's Algorithm Variant)
999. Dynamic Programming (Kadane's variation with Min/Max tracking).
1000. Dynamic Programming (Knapsack on Log-Sums) / Prime Factorization.
1001. Dynamic Programming (Knapsack variant) or Combinatorial Math. Since capping is a uniform rule, use the frequency of elements to calculate contributions.
1002. Dynamic Programming (Knapsack variant) or Meet-in-the-Middle if N is around 30-40.
1003. Dynamic Programming (LCS Variant)
1004. Dynamic Programming (LCS Variation).
1005. Dynamic Programming (Left & Right Prefix/Suffix Arrays)
1006. Dynamic Programming (Linear House Robber Variant)
1007. Dynamic Programming (Linear State)
1008. Dynamic Programming (Linear).
1009. Dynamic Programming (LIS Variant).
1010. Dynamic Programming (LIS Variation)
1011. Dynamic Programming (Longest Common Subsequence Variation)
1012. Dynamic Programming (Longest Common Subsequence).
1013. Dynamic Programming (Longest Palindromic Subsequence)
1014. Dynamic Programming (Matrix Chain Multiplication style).
1015. Dynamic Programming (Matrix Exponentiation)
1016. Dynamic Programming (MCM - Matrix Chain Multiplication Style).
1017. Dynamic Programming (MCM Variant)
1018. Dynamic Programming (Min/Max Tracking)
1019. Dynamic Programming (Most Significant Bit / Least Significant Bit)
1020. Dynamic Programming (O(N^2 * k))
1021. Dynamic Programming (Partition DP) / Generating Functions.
1022. Dynamic Programming (Pascal's Triangle Variation)
1023. Dynamic Programming (Pick/Skip State)
1024. Dynamic Programming (Pre-computed Distances)
1025. Dynamic Programming (Prefix Sum Opt)
1026. Dynamic Programming (Prefix Sum Transition)
1027. Dynamic Programming (Range DP).
1028. Dynamic Programming (Remainder DP). State = `dp[i][remainder]`. Find the maximum sum we can delete that leaves a remainder of R.
1029. Dynamic Programming (Remainder State)
1030. Dynamic Programming (Right-to-Left).
1031. Dynamic Programming (Row & Column Pre-computation)

1032. Dynamic Programming (Sort + Longest Path in DAG)
1033. Dynamic Programming (Space Optimized HashMap).
1034. Dynamic Programming (Space Optimized)
1035. Dynamic Programming (State Compression)
1036. Dynamic Programming (State Independence)
1037. Dynamic Programming (State Machine Kadane's).
1038. Dynamic Programming (State Machine)
1039. Dynamic Programming (State Machine) / Greedy.
1040. Dynamic Programming (State Machine): 2 states per index (Swapped vs Not Swapped).
1041. Dynamic Programming (State Transitions)
1042. Dynamic Programming (State: [roads][skips]).
1043. Dynamic Programming (State: City x Fuel)
1044. Dynamic Programming (State: i, j, diff)
1045. Dynamic Programming (State: Index + Prev Value)
1046. Dynamic Programming (State: index, last_color_left, last_color_right). O(N)
1047. Dynamic Programming (State: mile, lane, switchesLeft). O(N)
1048. Dynamic Programming (State: Value & Direction)
1049. Dynamic Programming (Stirling Numbers of the First Kind).
1050. Dynamic Programming (Subproblem relation).
1051. Dynamic Programming (Suffix Sum with Jump)
1052. Dynamic Programming (Suffix-based)
1053. Dynamic Programming (Tabulation). Building the current row iteratively from the directly previous row.
1054. Dynamic Programming (Top-Down Memoization or 2D Bottom-Up).
1055. Dynamic Programming (Top-Down with Memo)
1056. Dynamic Programming (Two 1D Arrays).
1057. Dynamic Programming (Two Passes)
1058. Dynamic Programming (Two-State: Ends in 0, Ends in 1)
1059. Dynamic Programming (Two-State: Same/Different)
1060. Dynamic Programming (Viterbi-like)
1061. Dynamic Programming + Bitmasking.
1062. Dynamic Programming + Freq Counting
1063. Dynamic Programming + Hash Map
1064. Dynamic Programming + Hashing (Sorting by Length).
1065. Dynamic Programming + Monotonic Queue (Sliding Window)
1066. Dynamic Programming + Monotonic Queue/Segment Tree.
1067. Dynamic Programming + Precomputed Best Times.
1068. Dynamic Programming + Prefix Sums
1069. Dynamic Programming + Sliding Window (Prefix Sum of Reachable Nodes).
1070. Dynamic Programming / 1D State Tracking.
1071. Dynamic Programming / Backtracking with Memoization
1072. Dynamic Programming / BFS with Memoization & Pruning.
1073. Dynamic Programming / BFS with State. State = `(row, col, direction)`. Since the path is "twisted", we track our entering vector.

1074. Dynamic Programming / Combinatorics Math
1075. Dynamic Programming / Greedy Matching
1076. Dynamic Programming / Greedy Range Extremes
1077. Dynamic Programming / Greedy State
1078. Dynamic Programming / Greedy Tally
1079. Dynamic Programming / Hash Map
1080. Dynamic Programming / Linear Counting
1081. Dynamic Programming / Math (Running Count).
1082. Dynamic Programming / Mathematical Greedy
1083. Dynamic Programming / Monotonic Stack.
1084. Dynamic Programming / Prefix Sums
1085. Dynamic Programming / Prefix Sums.
1086. Dynamic Programming / Prefix-Suffix Min.
1087. Dynamic Programming / State Machine
1088. Dynamic Programming / State Machine. State = `(Day, HasStock)`. Use a simple greedy one-pass if unlimited trades are allowed.
1089. Dynamic Programming on Trees (Tree DP).
1090. Dynamic Programming or Mathematical Optimization (Rule of 3s)
1091. Dynamic Programming with a Fenwick Tree or Segment Tree to find the maximum shipments ending at index `i`.
1092. Dynamic Programming with Bitmask
1093. Dynamic Programming with Bitmask (State Compression).
1094. Dynamic Programming with Bitmask.
1095. Dynamic Programming with Bitmasking
1096. Dynamic Programming with Bitmasking (State Compression)
1097. Dynamic Programming with Bitmasking / Dijkstra.
1098. Dynamic Programming with Counting (Combinatorics). Group elements by their remainder when divided by the "beauty" difference K, then solve as a "House Robber" variant.
1099. Dynamic Programming with Frequency Maps or Combinatorics. Calculate combinations of each element frequency F independently and sum them.
1100. Dynamic Programming with Mathematical Contribution (Prefix Arrays).
1101. Dynamic Programming with Memoization
1102. Dynamic Programming with State (Changed vs. Unchanged).
1103. Dynamic SQL / Information Schema
1104. Early Exit Iterator
1105. Early Short-Circuit Simulation. If any digit is '0', the product becomes 0 instantly (cannot divide by zero), so return false immediately.
1106. Early-Exit Linear Comparison
1107. Edge-Centric + Top 3 Neighbors
1108. Enumeration by Zero-Count (O(NâN))
1109. Euclidean Algorithm (Recursive/Iterative)
1110. Euclidean Algorithm for GCD. `gcd(a, b)` where you repeatedly take the remainder until one reaches zero.
1111. Event-Driven Simulation using 4 Priority Queues (Heaps).

1112. Execution Context Overhead (Visualizing the engine creating and destroying complex scopes for every iteration).
1113. Exhaustive Heatmap Search (Brute Force)
1114. Expand Around Center
1115. Expand Around Center + Difference Count.
1116. Expand Around Center.
1117. Exponential Expansion + Binary Search
1118. Exponential State Memory (Visualizing the storage of all valid subarray combinations).
1119. Exponential State Search (Visualizing all possible combinations of powers being summed).
1120. Exponential State Tracking (Visualizing memory holding all subsets simultaneously or deep recursion tree).
1121. Exponential String Allocation (Visualize the heap filling with modified versions of the string).
1122. Extreme Frequency Identification. O(N)
1123. Factorial Number System (Math)
1124. Fast & Slow Pointers (One Pass)
1125. Fast and Slow Pointers
1126. Fast and Slow Pointers (Tortoise and Hare)
1127. Fast Fourier Transform (FFT) or Karatsuba.
1128. Fast/Slow Pointers + Reverse Linked List.
1129. Fenwick Tree (Binary Indexed Tree)
1130. Fenwick Tree (BIT) / Index Mapping
1131. Fenwick Tree (BIT) / Segment Tree
1132. Fenwick Tree (BIT) / Segment Tree + Coordinate Compression
1133. Filtered Inner Join Pipeline
1134. Filtered Select with Null Handling.
1135. Find Dip, Swap, Reverse
1136. Find Middle + Reverse Second Half + Interweave.
1137. Finding Bound Indices (Last 1, First 3) + Linear locked-state count. O(N)
1138. First-Occurrence String Search / Math
1139. First/Last Index Pre-computation
1140. First/Last Occurrence Hash Map.
1141. Fisher-Yates Shuffle Algorithm
1142. Fixed Size Sliding Window
1143. Fixed Sliding Window (No Set).
1144. Fixed Sliding Window + Frequency Map
1145. Fixed Sliding Window + Persistent Hash Map (or Array Cache) for O(1) incremental score updates.
1146. Fixed Sliding Window on a Concatenated Array.
1147. Fixed-Size Max-Heap
1148. Fixed-Size Sliding Window
1149. Fixed-Space Iteration (100..998) + Frequency Map Tally. O(N + 450)
1150. Fixed-Window Sliding (Multiple Passes)

1151. Floating Closure Memory (The callback and its lexical environment stay in memory until the timer naturally finishes).
1152. Flood Fill (DFS/BFS) + Edge Trimming
1153. Floyd-Warshall (All-Pairs Shortest Path)
1154. Floyd-Warshall (Precomputation) or Weighted Disjoint Set Union (DSU).
1155. Floyd-Warshall (Transitive Closure)
1156. Floyd-Warshall Algorithm (All-Pairs Shortest Path)
1157. Floydâs Cycle Finding (Tortoise and Hare)
1158. Floydâs Cycle-Finding Algorithm (Fast & Slow Pointers)
1159. Floydâs Cycle-Finding Algorithm (Tortoise and Hare).
1160. Fractional Conversion (Math/String)
1161. Frequency Array (Character Tally)
1162. Frequency Array (Counting Sort Pattern)
1163. Frequency Array (Counting Sort).
1164. Frequency Array (Hash Buckets)
1165. Frequency Array (Hash Map Pattern)
1166. Frequency Array + Greedy Consolidation.
1167. Frequency Array + Prefix Sums + Math (Harmonic Series block processing).
1168. Frequency Array + Triple Loop (Values 1-100).
1169. Frequency Array / Counting (Rows are Sorted)
1170. Frequency Array / Hash Map
1171. Frequency Array / Hashing (Long Int)
1172. Frequency Array / Hashing (Modulo 24)
1173. Frequency Array / Math (Sum & Squares)
1174. Frequency Array / Matrix Exponentiation
1175. Frequency Bucket Counting
1176. Frequency Counter (Fixed-size Array)
1177. Frequency Counting (Greedy)
1178. Frequency Counting (Hash Map)
1179. Frequency Counting (Hash Map/Array)
1180. Frequency Counting (Odd Count Limit)
1181. Frequency Counting (Since arrays are sorted & distinct)
1182. Frequency Counting (Y vs. Non-Y)
1183. Frequency Counting + Combinatorics
1184. Frequency Counting + Greedy Math
1185. Frequency Counting + Iterative Value Check
1186. Frequency Counting + Last Occurrence Mapping
1187. Frequency Counting + Permutation Invariance.
1188. Frequency Counting + Top 2 Candidates.
1189. Frequency Counting Map / Parity Math
1190. Frequency Counting of Top 2 Elements (Even/Odd).
1191. Frequency Dominance / Median Comparison
1192. Frequency Hash Map (Value -> List of Indices).
1193. Frequency Hash Map + Greedy Matching.
1194. Frequency Hash Map + Sum Equation

1195. Frequency Hash Map or Bitmasking on an Array
1196. Frequency Hash Map or Sorted Two-Pointers.
1197. Frequency Hash Map.
1198. Frequency Intersection (Min-Tally)
1199. Frequency Map (Counting Sort Style)
1200. Frequency Map (Hash Table)
1201. Frequency Map + Binary Search or Sliding Window.
1202. Frequency Map + Divisibility Check
1203. Frequency Map + Greedy Construction
1204. Frequency Map + Greedy Sort
1205. Frequency Map + Greedy Sorting
1206. Frequency Map + Map of Stacks (Grouped by Freq).
1207. Frequency Map + Multiples / Divisors
1208. Frequency Map + Sorted Greedy
1209. Frequency Map + Sorted Unique Elements.
1210. Frequency Map / Bitset (Single Pass)
1211. Frequency Map / Bottleneck Analysis
1212. Frequency Map / Bucket Sort
1213. Frequency Map / Counting Sort
1214. Frequency Map / Hash Table
1215. Frequency Map / Index Array
1216. Frequency Map Compression (Maximum Requirements)
1217. Frequency Map or Sort + Two Pointers
1218. Frequency Map over Range (Visualizing a large array tracking every possible integer value).
1219. Frequency Map Ratio
1220. Frequency Mapping (Coordinate Transformation)
1221. Frequency Mapping + Greedy Pairing + Min-Cost Observation.
1222. Frequency Mapping by Index Parity
1223. Frequency Maps + Character Set Comparison
1224. Frequency Maps / Discrepancy Counting. (Compare counts of what you have vs. what you need).
1225. Frequency of Frequencies (Count Map)
1226. Frequency Sorting + Sliding Threshold
1227. Frequency Tracking with Contribution Counting. For each element, mathematically calculate how many subarrays it serves as the mode.
1228. Frequency-Based Math.
1229. Frobenius Coin Problem (Math)
1230. FULL OUTER JOIN (or UNION of LEFT JOINs)
1231. Function with Local Variable Offset (N-1) and LIMIT .
1232. Function.prototype.apply / Symbol Key
1233. Functional Graph Decomposition (Cycles & Tails)
1234. Game Theory (Minimax) + 3D Memoization (Topological Sort from End States).
1235. Game Theory (Sprague-Grundy/Nim) & Modulo Math Pattern
1236. Gaps and Islands (Row Number Difference)

1237. Gaps and Islands (SQL Window Functions) / Sequence Hashing.
1238. Gauss' Sum Formula OR XOR Gates
1239. GCD + Factor Counting
1240. GCD + Sorting
1241. Generators (`yield`)
1242. Geometry / Gradient Descent (or Weiszfeld's)
1243. Geometry Math (Column-based Calculation).
1244. Global Bounding Box Scan
1245. Global State Buffer (Persistent Pointer/Counter).
1246. Global Sum Maintenance (Delta Tracking)
1247. Graph (Functional Graph / Cycle Detection + Topological Sort).
1248. Graph (Prim's or Kruskal's MST)
1249. Graph + BFS/DFS or Disjoint Set Union (DSU) with Weights
1250. Graph / DFS with Simultaneous Verification
1251. Graph / Dijkstra's (Max-Heap)
1252. Graph Adjacency List + Degree Finding + DFS/Linear Traversal.
1253. Graph BFS / DFS with Visited Array
1254. Graph Coloring (3-Color DFS) OR Kahn's Algorithm (Topological Sort on reversed edges).
1255. Graph Coloring (BFS/DFS)
1256. Graph Coloring (Bipartite Graph Check).
1257. Graph Connectivity (BFS/DFS or Union-Find)
1258. Graph connectivity (Union Find / BFS on Circles)
1259. Graph Connectivity (Unique Source Tracking)
1260. Graph Conversion (Parent Pointers) + BFS.
1261. Graph Conversion + BFS
1262. Graph Cycle Decomposition
1263. Graph Cycles + Median Math
1264. Graph Degree Analysis + BFS
1265. Graph Degree Counting (Array-based Frequency Map).
1266. Graph Degree Logic (Out-degree vs In-degree) / Slot Counting
1267. Graph In-Degree Count + BFS/DFS
1268. Graph In-Degree Counting / Row Sum Optimization
1269. Graph Precomputation (Anchored Weights) / Component Ratios.
1270. Graph Reachability / SCC (Tarjan's)
1271. Graph Theory (Articulation Points / 2-Day Rule)
1272. Graph Theory (Degree counting). Odd-degree nodes must total 0, 2, or 4. Check specific adjacency rules.
1273. Graph Theory (In-Degree Zero)
1274. Graph Theory (Out-degree & Cycles)
1275. Graph Theory (Topological Sort)
1276. Graph Theory / Degree Sorting (Constructive).
1277. Graph Traversal (BFS or DFS) with Visited Set.
1278. Graph Traversal (BFS) / Collaborative Filtering with Inverted Index.
1279. Graph Traversal (DFS/BFS) + Reachability

1280. Graph Traversal (Directed Graph) + BFS/DFS.
1281. Graph Tree Diameter (2-Pass BFS/DFS)
1282. Gray Code Generation + Bitwise XOR
1283. Gray Code Reversal / Bit Manipulation
1284. Greatest Common Divisor (GCD)
1285. Greatest Common Divisor (GCD) Filter
1286. Greatest Common Divisor (GCD) of the displacements. Find the distance each element needs to move and find the GCD of all those distances.
1287. Greedy "Flip-and-Carry" or Difference Array. Start from the left; if the current bit doesn't match the target, flip the remainder of the string.
1288. Greedy "Largest to Back"
1289. Greedy "Unbalanced" Counter
1290. Greedy (Adjacent Pair Checking)
1291. Greedy (Adjacent Zero-Segment Merging). O(N)
1292. Greedy (Always pick two largest) or Math.
1293. Greedy (BFS-Style Window)
1294. Greedy (Bit Manipulation)
1295. Greedy (Complementary Minimum Tracking). O(N)
1296. Greedy (Count Even Numbers)
1297. Greedy (Dual Sorting)
1298. Greedy (Extreme Finding)
1299. Greedy (Fill from Back)
1300. Greedy (First Mismatch).
1301. Greedy (Heavy-Letter Priority)
1302. Greedy (Huffman Coding / Min-Heap)
1303. Greedy (Independent Max Gaps)
1304. Greedy (Interval Scheduling / Jump Game logic).
1305. Greedy (Interval Sorting)
1306. Greedy (Kruskal's) or Tree DP (if constrained).
1307. Greedy (Last Occurrence Map)
1308. Greedy (Math-based Calculation).
1309. Greedy (Max Reach)
1310. Greedy (Maximum Digit)
1311. Greedy (Min-Heap / Sorting)
1312. Greedy (Min/Max Balance Tracking).
1313. Greedy (modulo matches) + DP with Bitmask / Memoization for remainders.
1314. Greedy (Monotonic Suffix Max)
1315. Greedy (One-Pass Peak-Valley Counting)
1316. Greedy (Pairs Comparison)
1317. Greedy (Priority Queue / BFS) or Binary Search + DFS.
1318. Greedy (Right-to-Left Scan).
1319. Greedy (Right-to-Left Traversal)
1320. Greedy (Right-to-Left)
1321. Greedy (Right-to-Left) + Math Division
1322. Greedy (Rightward free, Leftward paid).

1323. Greedy (Running Offset)
1324. Greedy (Skipping Alternate Stones)
1325. Greedy (Smallest Even Strategy).
1326. Greedy (Sort + Boundary Cap)
1327. Greedy (Sort and Gap-Check).
1328. Greedy (Sort by Difference)
1329. Greedy (Sort by End Time) Sort intervals by their end times. Always pick the one that ends earliest to leave maximum room for others.
1330. Greedy (Sort by End Time).
1331. Greedy (Sort by End) Sort balloons by their end coordinates. Shoot one arrow at the end of the first balloon; skip all that overlap.
1332. Greedy (Sort Neighbors per Node)
1333. Greedy (Sorting + Linear Grouping)
1334. Greedy (Sorting + Prefix Sums)
1335. Greedy (Sorting by Combined Value)
1336. Greedy (Sorting by Fractional Part).
1337. Greedy (Sorting by Opportunity Cost).
1338. Greedy (Sorting)
1339. Greedy (Sorting).
1340. Greedy (Sum of Unique Positive Numbers). O(N)
1341. Greedy (Target Candidates).
1342. Greedy (Two Pointers / Multi-Pass).
1343. Greedy (Two Pointers).
1344. Greedy (Two Scenarios)
1345. Greedy (Two Scenarios: Even vs Odd indices)
1346. Greedy (Two-Candidate Check).
1347. Greedy (Two-Pass Scanning).
1348. Greedy (Work Backwards from Target).
1349. Greedy + Arithmetic Progression
1350. Greedy + Backtracking + Prime Factorization
1351. Greedy + Bitmask (or HashSet)
1352. Greedy + Case Analysis Categorize by length: <6 (Insert), 6-20 (Replace), >20 (Delete then Replace). Priority on deleting from repeating triplets.
1353. Greedy + Coordinate Modulo Counting
1354. Greedy + Counting / Bucket Sort
1355. Greedy + Decay Offset
1356. Greedy + Delta Sorting (or Simple DP)
1357. Greedy + Fenwick Tree / Segment Tree
1358. Greedy + Frequency Array (Counting Sort).
1359. Greedy + Frequency Counting (Bucket Sort).
1360. Greedy + Frequency Fill (Even/Odd Interleaving).
1361. Greedy + Frequency Sorting / Hash Set
1362. Greedy + Greedy with Binary Search (TreeSet)
1363. Greedy + Max-Heap + Difference Array
1364. Greedy + Min-Heap (Bounded Priority Queue)

1365. Greedy + Min-Heap (Priority Queue)
1366. Greedy + Monotonic Stack + Frequency Counting.
1367. Greedy + Priority Queue (Min-Heap of negative values)
1368. Greedy + Remainder DP
1369. Greedy + Set Intersection (Filtering Friendships).
1370. Greedy + Sorting
1371. Greedy + Sorting (Two-Pointer logic)
1372. Greedy + Sorting.
1373. Greedy + Stack (Two Passes).
1374. Greedy + Stack + Suffix Minimums
1375. Greedy + Suffix Matching Precomputation
1376. Greedy + Two Pointers (Rolling Hash)
1377. Greedy / Array Math
1378. Greedy / Bit Manipulation
1379. Greedy / Bubble Sort Logic
1380. Greedy / Bucketing by Remainder
1381. Greedy / Case Analysis.
1382. Greedy / Cycle Counting with Costs.
1383. Greedy / Cycle Decomposition (DSU)
1384. Greedy / Event Simulation (State tracking).
1385. Greedy / Fixed Window Flip
1386. Greedy / Frequency Counting
1387. Greedy / Independent Sorting
1388. Greedy / Last-Position Tracking.
1389. Greedy / Linear Scan
1390. Greedy / Majority Element
1391. Greedy / Math (Reverse Operations)
1392. Greedy / Math (Zero Counting).
1393. Greedy / Math Accumulator
1394. Greedy / Math Analysis
1395. Greedy / Math Observation
1396. Greedy / Math-based Cycle Counting
1397. Greedy / One-Pass Simulation
1398. Greedy / One-Pass State Machine
1399. Greedy / Parity based on Depth
1400. Greedy / Parity Counting
1401. Greedy / Post-Order DP (State Signaling)
1402. Greedy / Prefix Counters
1403. Greedy / Preprocessing
1404. Greedy / Relative Order Tracking
1405. Greedy / Running Balance Counter
1406. Greedy / Running GCD
1407. Greedy / Set Intersection
1408. Greedy / Simulation (Peak Tracking)
1409. Greedy / Single-Source Shortest Path (Dijkstra) or Prefix Min.

1410. Greedy / Sorting
1411. Greedy / Sorting Mirror Elements
1412. Greedy / Stack Logic (Counter)
1413. Greedy / State-Flip Parity
1414. Greedy / Two Pointers
1415. Greedy / Two Pointers / Preprocessing
1416. Greedy / Virtual Deletion
1417. Greedy Absolute Displacement
1418. Greedy Accumulation (Target Sum Matching).
1419. Greedy Accumulation / Prefix Sum tracking.
1420. Greedy Algorithm (Coin Change variation)
1421. Greedy Algorithm (Pointer Jumping)
1422. Greedy Algorithm (Price-per-Weight Ratio).
1423. Greedy Algorithm (Rearrangement Inequality): Sort one ascending, one descending.
1424. Greedy Algorithm (Summing Positive Slopes).
1425. Greedy Algorithm + Sorting
1426. Greedy Algorithm + Sorting.
1427. Greedy Algorithm + Sorting. Sort the array. To maximize medians, pair the largest available, the *next* largest (the median), and the smallest available.
1428. Greedy Algorithm / Single Pass Array Scan.
1429. Greedy Algorithm / Sorting. Match the longest land rides with the shortest water rides (or vice versa, depending on exact problem constraints) to minimize the maximum bottleneck.
1430. Greedy Algorithm / String Parsing with continuous accumulation.
1431. Greedy Algorithm with a Max-Priority Queue. At any step, activate the item that provides the highest immediate gain/value based on current state.
1432. Greedy algorithm with a Queue or Monotonic Stack, focusing on the nearest available character that satisfies the activation rule.
1433. Greedy Alternation
1434. Greedy Analysis of Parity. An alternating sum is (EvenSum - OddSum) . Swapping only helps if you move a large number from an "Odd" (negative) position to an "Even" (positive) position.
1435. Greedy AND-Reset Logic
1436. Greedy Array Compression
1437. Greedy Array Processing (Jump Game II Pattern)
1438. Greedy Assignment + Frequency Sorting
1439. Greedy Assignment + Hash Map of Max-Heaps + Special Worker Greedy choice. O((N+M) log M)
1440. Greedy Billing (Counter-based).
1441. Greedy Bit Alignment
1442. Greedy Bit Construction
1443. Greedy Bit Manipulation (Low-bit Analysis).
1444. Greedy Bit Manipulation (Trailing Zeros)
1445. Greedy Bit Manipulation + Min-Priority Queue (or Frequency Array)

1446. Greedy Bit-by-Bit Construction (from MSB to LSB). Check if a partition can satisfy the current bit mask using a linear scan.
1447. Greedy Bit-Construction + Trie.
1448. Greedy Bitwise Construction (MSB to LSB).
1449. Greedy Bitwise Masking (MSB to LSB)
1450. Greedy Block Counting (Run-Length)
1451. Greedy Boundary Detection (Top-Left to Bottom-Right)
1452. Greedy Candidate Maintenance (Two-Variable Sweep)
1453. Greedy Case-Based Logic.
1454. Greedy Chunking (Math)
1455. Greedy Coloring.
1456. Greedy Column-wise Scan
1457. Greedy Comparison (2 Target States).
1458. Greedy Component Rounding
1459. Greedy Constraint Matching
1460. Greedy Constructive Construction.
1461. Greedy Coordinate Accumulation (Math)
1462. Greedy Counting (Compare with two possible targets: "0101..." and "1010...").
1463. Greedy Counting + Sorting lengths
1464. Greedy Counting / Cumulative Sum
1465. Greedy Counting / Frequency Analysis
1466. Greedy Degree-Based Assignment
1467. Greedy Difference Tracking + Parity Check.
1468. Greedy Digit Allocation
1469. Greedy Digit Manipulation
1470. Greedy Digit Substitution
1471. Greedy Distance Calculation (Total Distance - Max Saving).
1472. Greedy DP + Prefix Sums + Monotonic Queue
1473. Greedy Elimination (Two Pointers)
1474. Greedy Extremes Check
1475. Greedy Factorization (Divisors 9 down to 2).
1476. Greedy Filtering (One Pass).
1477. Greedy First-Digit Replacement.
1478. Greedy Frequency Counting + Min-Heap
1479. Greedy Frequency Distribution. Sort elements by frequency. Place the most frequent elements into different groups first.
1480. Greedy Frequency Sorting
1481. Greedy Frequency Sorting + Combinatorics
1482. Greedy Game State Simulation (Two Pointers)
1483. Greedy Gap Filling / Pigeonhole Principle
1484. Greedy Grouping (HashMap)
1485. Greedy Half-Scan Mutation
1486. Greedy Hash Map / Frequency Array
1487. Greedy Interval Scheduling (Earliest Deadline First).
1488. Greedy Interval Scheduling (End-Point Prioritization).

1489. Greedy Left-to-Right / Math
1490. Greedy Left-to-Right Minimization / Two Pointers
1491. Greedy Linear Scan
1492. Greedy Local Correction.
1493. Greedy Math (Arithmetic Progression)
1494. Greedy Math (Integer Break logic) + Fast Modular Exponentiation.
1495. Greedy Math (Single Pass).
1496. Greedy Math + Case Analysis
1497. Greedy Math / Frequency Map
1498. Greedy Math / Max Priority Queue
1499. Greedy Math / Modulo Arithmetic
1500. Greedy Math / Total Sum
1501. Greedy Mathematical Balancing
1502. Greedy Matrix Allocation
1503. Greedy Min/Max Tracking.
1504. Greedy Modular Distance
1505. Greedy One-Pass
1506. Greedy One-Pass (The Total Net Gain Rule).
1507. Greedy One-Pass Comparison
1508. Greedy Overlap Selection.
1509. Greedy Pairing + Modular Exponentiation
1510. Greedy Palindrome Generation Start from the largest possible palindrome (e.g., 999999) and work downwards. For each, check if it has two n-digit factors.
1511. Greedy Parity Construction
1512. Greedy Parity Property
1513. Greedy Peak Finding (Single Element Identifier).
1514. Greedy Pointer / Linear Jump
1515. Greedy Pointer Matching.
1516. Greedy Positional Logic.
1517. Greedy Positioning (Two-Target Comparison).
1518. Greedy Pre-reversal + Optimized Split Scan.
1519. Greedy Prefix Matching
1520. Greedy Prefix Maximum Tracking (O(N x M)).
1521. Greedy Preorder Simulation
1522. Greedy Prime Factorization (Reverse Digits)
1523. Greedy Property (Row Head + Column Max).
1524. Greedy Quadrant Invariance. O(N)
1525. Greedy Range Expansion
1526. Greedy Range Extension (Patching)
1527. Greedy Range Removal (Set/Segment Tree)
1528. Greedy Reverse Engineering (Work Backwards)
1529. Greedy Reverse Matching (Work Backwards)
1530. Greedy Selection (Math/Sorting)
1531. Greedy Selection + Boolean Array/Set
1532. Greedy Selection + Difference Array / Sorting.

1533. Greedy Selection + Hash Set Lookup.
1534. Greedy Selection + Hash Sets
1535. Greedy Selection + Smallest-Odd/Even Swap
1536. Greedy Selection on Sorted Array. O(N log N)
1537. Greedy Selection with Math Constraints
1538. Greedy Set Partitioning
1539. Greedy Simulation
1540. Greedy Simulation (Two Cases)
1541. Greedy Simulation (While Loop)
1542. Greedy Simulation with BFS/DFS
1543. Greedy Single Pass / State Machine
1544. Greedy Single Pass with Hash Set.
1545. Greedy Single-Pass / Max-Tracker.
1546. Greedy Sliding Window
1547. Greedy Sorting (Descending Order).
1548. Greedy Sorting (Largest Cost First)
1549. Greedy Sorting (Smith's Rule)
1550. Greedy Sorting + Last Digit Swap
1551. Greedy Sorting + Stack-based Substitution
1552. Greedy Sorting / Hash Map counting. Pair cards based on their first and second letters to maximize your "counter-plays".
1553. Greedy Sorting / Two Pointers
1554. Greedy Sorting Alignment
1555. Greedy Sorting by "Time-to-Arrival"
1556. Greedy Sorting.
1557. Greedy Sorting. Sort the array descending and pick the largest values until you hit the K distinct limit.
1558. Greedy Stack (Simulation).
1559. Greedy Strategy
1560. Greedy Strategy (Tian Ji's Horse Racing).
1561. Greedy Strategy + Sorting.
1562. Greedy Strategy: Earliest Deadline First (EDF) or Shortest Processing Time (SPT).
1563. Greedy String Matching (Split by wildcard, search Prefix then Suffix). (O(N))
1564. Greedy String Mutation (Right-to-Left)
1565. Greedy Subtraction
1566. Greedy Subtraction (Coin Change)
1567. Greedy Sum Matching
1568. Greedy Summation / Math
1569. Greedy Swapping / Parity-Based Reordering. O(N)
1570. Greedy Tally + Remainder Math
1571. Greedy Template Filling + Contradiction Check (KMP/Suffix). O(n Â· m)
1572. Greedy Time Calculation (Hash Map)
1573. Greedy Traversal (Right-to-Left) + Smallest Prime Factor (SPF)
1574. Greedy Traversal / Peak Finder

1575. Greedy Two-Pointer Placement
1576. Greedy Virtual Deletion (Two Pointers)
1577. Greedy with Difference Array / Sliding Window
1578. Greedy with Frequency & Vacancy Maps.
1579. Greedy with Frequency Maps or Dynamic Programming. Count the excess of each item type and discard only the necessary surplus.
1580. Greedy with Max-Heap (Priority Queue).
1581. Greedy with Max-Priority Queue.
1582. Greedy with Min-Heap (Priority Queue).
1583. Greedy with Modular Arithmetic
1584. Greedy with Priority Queue (Max-Heap)
1585. Greedy with Set Cover or Bitmask DP (if dimensions are small). Focus on covering the "bottleneck" (uncovered cells with the fewest neighbors).
1586. Greedy with Sortedness Tracking
1587. Greedy with Suffix Comparison.
1588. Greedy Word Packing + Round Robin Spacing
1589. Greedy Zero-ing (Place Value Increment)
1590. Grid Expansion + DFS/BFS (or Union-Find)
1591. Grid Partitioning + Bounding Box
1592. Group By + Aggregate COUNT(DISTINCT)
1593. Group By + Aggregate Ratio + Order By.
1594. GROUP BY + Aggregation (SUM)
1595. Group By + CASE WHEN + ROUND
1596. GROUP BY + COUNT
1597. GROUP BY + COUNT(DISTINCT)
1598. GROUP BY + COUNT(DISTINCT) + AVG
1599. GROUP BY + DATE_FORMAT
1600. GROUP BY + DATEDIFF / BETWEEN
1601. GROUP BY + GROUP_CONCAT / STRING_AGG
1602. Group By + Having (Conditional Sums).
1603. Group By + Having (Min/Max Dates).
1604. Group By + Having (Set Membership)
1605. Group By + Having (Subquery Max).
1606. Group By + Having Clause.
1607. Group By + Having Count (Distinct).
1608. Group By + Having Count Clause
1609. GROUP BY + HAVING COUNT()
1610. GROUP BY + HAVING COUNT(DISTINCT)
1611. Group By + Having.
1612. GROUP BY + MAX() + EXTRACT/LIKE/YEAR().
1613. Group By + Order By Count + Limit 1.
1614. Group By + Scalar Subquery / Cross Join
1615. GROUP BY + SUM(CASE WHEN)
1616. Group By / Sort by ID & Date + Linear Scan. O(N log N)
1617. Group by First Letter + Venn Diagram Intersection

1618. Group by Length/Last Char + Trie for Longest Common Prefix (LCP).
1619. GROUP BY with HAVING COUNT(*) > 1 .
1620. Group By.
1621. Group Consecutive Counts.
1622. Group line segments by slope using a Hash Map. A trapezoid is formed by picking 2 segments with the same slope (bases) and ensuring they don't share points.
1623. Grouping + `COUNT(DISTINCT)` with `HAVING`
1624. Grouping + Sliding Window/Case Analysis
1625. Grouping by Vowel Count + Reverse Pointers
1626. Hamiltonian Path (Backtracking + Pruning)
1627. Hash Aggregation / Index Join
1628. Hash Anti-Join (LEFT JOIN + IS NULL)
1629. Hash Join / Hash Map Filtering
1630. Hash Map (Aggregate & Track Max)
1631. Hash Map (Counter) + Custom Stable Sort.
1632. Hash Map (Counting) + Hash Set (Uniqueness)
1633. Hash Map (Dictionary) for O(1) Access.
1634. Hash Map (Dictionary) for O(1) CRUD.
1635. Hash Map (First Word Mapping)
1636. Hash Map (First/Last/Count).
1637. Hash Map (Frequency Count) / Single Pass
1638. Hash Map (Frequency Counter) + Second Pass
1639. Hash Map (Frequency Counter) with GCD reduction for fractions
1640. Hash Map (Frequency Counting)
1641. Hash Map (Frequency Counting) + Math Combinatorics.
1642. Hash Map (Frequency Counting).
1643. Hash Map (Frequency Map) + Two Sum variant for 22 static targets.
1644. Hash Map (Frequency) + String Prefix/Suffix Splitting
1645. Hash Map (ID to Rank) or Tuple Sorting. Store (Time, ID) pairs and sort them directly.
1646. Hash Map (Inventory Counter).
1647. Hash Map (Meet-in-the-Middle logic)
1648. Hash Map (Meet-in-the-Middle) Calculate all A+B and store in a Map `sum: count`. Then iterate C+D and check for -(C+D) in the Map.
1649. Hash Map (Object Literal) Accumulator
1650. Hash Map (One Pass)
1651. Hash Map (Parent Pointer) + Set Intersection
1652. Hash Map (Piece-Start Mapping)
1653. Hash Map (Remainder Frequency)
1654. Hash Map (Sorted Key / Char Count)
1655. Hash Map (Sparse Storage) + Direct Formula Parsing. O(1) set / O(L) get.
1656. Hash Map (Stringified Rows)
1657. Hash Map (Timestamp -> Price) + Two Heaps (or TreeMap)
1658. Hash Map (User to Task-List) + Sorting/Filtering.

1659. Hash Map (Val -> Node) + Set (Child Tracking).
1660. Hash Map (Value rightarrow Frequency Count).
1661. Hash Map + Balanced BST (or TreeMap/SortedSet)
1662. Hash Map + Bitmask (Sparse Row Processing)
1663. Hash Map + clearTimeout
1664. Hash Map + Custom Comparator Sorting
1665. Hash Map + DFS/BFS.
1666. Hash Map + Digit Perturbation
1667. Hash Map + Doubly Linked List (DLL).
1668. Hash Map + Doubly Linked List of Buckets Map strings to nodes. Nodes are buckets of strings with the same count. Buckets are linked in order of count.
1669. Hash Map + Frequency Counters
1670. Hash Map + Grouping (Run-Length Encoding)
1671. Hash Map + Linked List of DLLs Map keys to nodes. Have a second "Frequency Map" that points to a Doubly Linked List of all keys with that frequency.
1672. Hash Map + Max-Heap (Priority Queue) with Lazy Deletion.
1673. Hash Map + Max-Heap/SortedList with Lazy Deletion. (O(log N))
1674. Hash Map + Max-Priority Queue (Lazy Removal)
1675. Hash Map + Min-Heap (for ID reuse)
1676. Hash Map + Min-Heap (size K).
1677. Hash Map + Prefix Sum
1678. Hash Map + Result Linked List Construction
1679. Hash Map + Sieve of Eratosthenes.
1680. Hash Map + Target XOR Decomposition
1681. Hash Map + Word Split
1682. Hash Map / Array Grouping by Max Digit
1683. Hash Map / Array Substitution Cipher
1684. Hash Map / Chunk Frequency
1685. Hash Map / Combinatorics
1686. Hash Map / Dictionary Counting
1687. Hash Map / Fixed Array (First Occurrence)
1688. Hash Map / Frequency Array
1689. Hash Map / Frequency Array (Modulo Arithmetic).
1690. Hash Map / Frequency Array Count char frequencies. Add all even counts. Add (odd-1) for odd counts. Add 1 if any odd exists.
1691. Hash Map / Frequency Array.
1692. Hash Map / Frequency Count (Modulo Arithmetic).
1693. Hash Map / Frequency Counting
1694. Hash Map / Frequency Indexing
1695. Hash Map Adjacency List + BFS/DFS.
1696. Hash Map Aggregation
1697. Hash Map Aggregation / Group By (if SQL). Accumulate counts in a single pass, then compare against the precomputed average.
1698. Hash Map Aggregation / Sorting by User
1699. Hash Map Aggregation / SQL Group By with HAVING clause. Compute

statistics for all books in one pass.
1700. Hash Map Aggregation / SQL-style GroupBy.
1701. Hash Map Binding / Array of Objects + Sort
1702. Hash Map Count + Sentinel Node.
1703. Hash Map Counting (Frequency of Arrays)
1704. Hash Map Counting + Single-Pass Max Tracking.
1705. Hash Map for Index Lookup.
1706. Hash Map for List A (Value -> Index). Scan List B and look up values in the Map in O(1).
1707. Hash Map for lookups + Single Pass StringBuilder.
1708. Hash Map for O(1) inorder index lookup + Recursive DFS (Process postorder backwards: Root, Right, Left).
1709. Hash Map for O(1) inorder index lookup + Recursive DFS.
1710. Hash Map for Row Patterns.
1711. Hash Map for Slopes (per starting point).
1712. Hash Map Frequencies + Greedy Math Downward Search
1713. Hash Map Frequencies + Mathematical Chaining
1714. Hash Map Frequency + Sorting
1715. Hash Map Frequency Counting
1716. Hash Map Frequency Tracking.
1717. Hash Map grouping (Content -> List of Paths).
1718. Hash Map Grouping + Circular Maximum Gap Calculation
1719. Hash Map Grouping + Single Pass Validation.
1720. Hash Map Grouping + Sort & Sweep.
1721. Hash Map Grouping + Sorting
1722. Hash Map Grouping by `Row + Col`
1723. Hash Map Indexing
1724. Hash Map mapping User ID to a Hash Set of minutes.
1725. Hash Map of Bitmasks. For each unique bitmask, store the maximum integer found. Then compare all pairs of bitmasks.
1726. Hash Map of Distances For each point i, calculate distances to all other points j. Store counts in a Map `dist: count`. Result is sum count x (count - 1).
1727. Hash Map of Edge Frequencies.
1728. Hash Map of Hash Maps & Tree Sets
1729. Hash Map of Hash Sets. O(N)
1730. Hash Map of Indices + Sliding Window
1731. Hash Map of Lists (Greedy Accumulation)
1732. Hash Map of Lists (Indexed Access)
1733. Hash Map of Lists + Sorting + Sliding Window (Size 3)
1734. Hash Map of Max Values
1735. Hash Map of Sets (Callback Registry)
1736. Hash Map of Sorted Indices + Binary Search
1737. Hash Map of Sorted Sets (TreeMap)
1738. Hash Map Point Frequencies + Diagonal Anchoring
1739. Hash Map Pre-processing

1740. Hash Map Pre-processing + Two Pointers
1741. Hash Map Remapping (Virtual Array)
1742. Hash Map Root Lookup + DFS Validation
1743. Hash Map State Tracking + DFS Pruning
1744. Hash Map Traversal / Parent-Child Mapping.
1745. Hash Map with Canonical Form Key
1746. Hash Map with Expiration Timestamps
1747. Hash Map with Index Lists / Sliding Window of Size 3
1748. Hash Map with Math Ratio (GCD) and Middle-element Fixing (O(N^2)).
1749. Hash Map with Random Base62 Encoding.
1750. Hash Map with Size-3 Deque (Sliding Window of Indices)
1751. Hash Map with Suffix Tracking
1752. Hash Map with Tuple Keys / Bitsets
1753. Hash Set (Backward Scan)
1754. Hash Set (Coordinate Encoding).
1755. Hash Set (Impossible Value Filtering).
1756. Hash Set (Intelligent Starting Points).
1757. Hash Set (Seen Filter)
1758. Hash Set (Single Pass)
1759. Hash Set (Target Difference math).
1760. Hash Set + Extremity Midpoint
1761. Hash Set + Greedy Growth
1762. Hash Set + Min/Max Math (No Sorting)
1763. Hash Set + Pre-processing.
1764. Hash Set + Single BFS/DFS
1765. Hash Set + While Loop
1766. Hash Set / Bitmask + Math.min
1767. Hash Set / Bitmasking
1768. Hash Set / Boolean Array
1769. Hash Set / Boolean Array (Bitmask)
1770. Hash Set / Boolean Array Lookup
1771. Hash Set / Boolean Arrays
1772. Hash Set / Boolean Lookahead
1773. Hash Set / Direct Indexing
1774. Hash Set / Frequency Counting
1775. Hash Set / Sorting + Binary Search.
1776. Hash Set / Two Pointers (after Sort)
1777. Hash Set Aggregation. For each customer, collect `transaction_month` into a Set. Filter customers where `Set.size == 12`.
1778. Hash Set for O(1) word lookup + Custom Comparator Sorting (or Min-Heap for top K selection).
1779. Hash Set Intersection (Difference Checking)
1780. Hash Set Lookup
1781. Hash Set Lookup + Single Pass Traversal.
1782. Hash Set Lookup / Single Pass Tokenization.

1783. Hash Set Lookups
1784. Hash Set of Pair Strings
1785. Hash Set of Visited Points
1786. Hash Set or Boolean Array Check
1787. Hash Set or Boolean Frequency Array.
1788. Hash Set Validation (or Boolean Array Tracking).
1789. Hash Set.
1790. Hash Sets / Set Difference
1791. Hash Table + Alphabetical Sorting
1792. Hash Table / Array Mapping (Single Pass)
1793. Hashing / Rolling Hash
1794. HashMap + Dynamic Array (Swap with Last)
1795. HashMap + Sorted List (Binary Search)
1796. HashMap of Sets + Dynamic Array
1797. HashMaps for Line Counting
1798. HashSet + Dummy Node 1-Pass
1799. HashSet Lookup
1800. Havel-Hakimi Algorithm. Sort the degree sequence, then repeatedly "consume" the highest degree by connecting it to the next highest degrees.
1801. Head Re-linking (Two-Pointer Shift)
1802. Heap Allocation Churn (Visualizing the heap filling with intermediate string fragments: "a", "aa", "aaa"...).
1803. Heap Allocation Overload (Visualize multiple intermediate string objects piling up in memory due to immutable string slicing).
1804. Heap Array Allocation (Visualizing thousands of string objects being stored simultaneously).
1805. Heap Fragment Memory (Visualizing thousands of temporary string objects created during slicing).
1806. Heap memory allocation map (Drawing separate object allocations in memory for every closure context created).
1807. Hierholzer's Algorithm (Eulerian Path via DFS)
1808. Hierholzerâs Algorithm (Eulerian Path)
1809. Higher-Order Function (Factory)
1810. History Tracking (Binary Search on Versions)
1811. Horizontal Shrinking
1812. Horner's Method for Polynomial Evaluation. Process the string character by character, multiplying the current result by the base and adding the new digit.
1813. Implicit Treap (Randomized Binary Search Tree) / Splay Tree with Lazy Propagation.
1814. IN Clause with Multi-Column Key
1815. In-Degree Accumulator (Array)
1816. In-order DFS + Two-Pointer Merge
1817. In-Order DFS Traversal with Level Grouping.
1818. In-Order Flattening + Midpoint Reconstruction
1819. In-Order Traversal (natively sorted) + Track `prev` Node.

1820. In-Order Traversal (Sorted Property)
1821. In-order Traversal + Binary Search (Sorted Array)
1822. In-order Traversal with In-place Re-linking.
1823. In-Place "Leapfrog" Reversal
1824. In-place array mutations (Unnecessary write operations to memory).
1825. In-place Array Mutations (Visualizing the CPU repeatedly writing to memory for every range update).
1826. In-place Array Transformation.
1827. In-Place Encoding (Math / Modulo)
1828. In-Place Extraction + Sorting.
1829. In-Place Group Iteration & Re-linking
1830. In-Place In-order Maintain a `first` and `last` pointer. Connect `last.right = current` and `current.left = last` during traversal.
1831. In-Place Iterative Reduction
1832. In-Place Link Manipulation
1833. In-place Marking For each number x, go to index |x|-1 and flip the value to negative. If already negative, it's a duplicate.
1834. In-place Matrix Rotation (Transpose + Reverse) or Index Mapping.
1835. In-place modification of Character Array.
1836. In-place Negation For each value x, treat |x|-1 as an index and make the value at that index negative. Positives remaining are the gaps.
1837. In-Place Pointer Reversal
1838. In-Place Pointer Splicing + Euclidean Algorithm
1839. In-place row swapping within the specified submatrix boundaries.
1840. In-Place Simulation (Two Pointers/Copying)
1841. In-Place State Encoding (Bit Manipulation)
1842. In-place State Flagging & Two-Pointer Drop
1843. In-Place State Marking
1844. In-Place Two Pointers / Link Manipulation.
1845. In-Place Two-Pointer Swap
1846. Inclusion-Exclusion (Math)
1847. Inclusion-Exclusion Principle (Area1 + Area2 - Overlap)
1848. Inclusion-Exclusion Principle (Math)
1849. Inclusion-Exclusion Principle with Modular Exponentiation
1850. Inclusion-Exclusion Principle. (Total Trapezoids = All shapes with at least 1 pair of parallel sides MINUS Parallelograms which have 2 pairs).
1851. Incremental Accumulation + Upper Bound Pruning (O(N^2)).
1852. Independent Precomputed BFS
1853. Independent Row/Column Counting
1854. Independent Subqueries / Common Table Expressions (CTE)
1855. Index Mapping + Linear Scan.
1856. Index-as-Hash / Negation Marker.
1857. Information Theory (N-Dimensional Hypercube) Each pig represents a dimension. The number of states a pig can have is (Tests + 1). Solve (States)^{Pigs} ge Buckets.

1858. Inner Join or Group By with IN clause.
1859. INNER JOIN with GROUP BY and HAVING
1860. INNER JOIN with Inequality Filtering
1861. Inner Join.
1862. Inorder Simulation (DFS). Calculate list length. Build left subtree, assign current list node as root, advance list pointer, build right subtree.
1863. Inorder Traversal (Morris Traversal) with State Tracking variables.
1864. Inorder Traversal to Hash Set (or Two Pointers on Arrays)
1865. Insertion Sort with Dummy Head and Predecessor Pointers.
1866. Integer Modulo Sum + Frequency Bucket
1867. Integer State Mutation (Visualizing the variables main and additional changing over time).
1868. Interactive Bit Manipulation
1869. Intermediate Array Allocation (Visualize a potentially massive array of strings being created in memory just for a boolean check).
1870. Intermediate Array Reallocations (Visualizing the heap creating new arrays for every merge pass).
1871. Intermediate Subarray Slices (Visualizing the heap filling with transient array segments).
1872. Interval Consolidation + Greedy Activity Selection. O(N)
1873. Interval DP (Matrix Chain Multiplication) + HashSet
1874. Interval DP (Top-Down Memoization)
1875. Interval DP + Partition DP
1876. Interval DP `dp[i][j]` stores the shortest encoded string for `s[i...j]`. Check for patterns using `(s+s).indexOf(s, 1)`.
1877. Interval Dynamic Programming
1878. Interval Dynamic Programming (Minimax)
1879. Interval Expansion + Greedy Search
1880. Interval Merging
1881. Interval Merging / Prefix Sums
1882. Interval Merging / Segment Tree (TreeMap)
1883. Interval Sorting + Monotonic Scan
1884. Interweaving Nodes (In-place).
1885. Invariance & Hamming Distance
1886. Inverse Sliding Window (Two Pointers) seeking the maximum middle subarray to *leave behind*.
1887. Inverse Transform (Polar) Generate theta in [0, 2Ï] and r = R â{rand()}. The square root is crucial for uniform density.
1888. Inverted Index (Hash Map)
1889. Inverted Index + Bitsets / Matrix Multiplication.
1890. Iterative Accumulator Update
1891. Iterative Bottom-Up Dynamic Programming (State variables).
1892. Iterative Brute Force (Optimized Range)
1893. Iterative Cleanup + Block Processing
1894. Iterative Comparison

1895. Iterative Digit Extraction (Modulo & Stack).
1896. Iterative Division (Prime Squeezing)
1897. Iterative DP (Fibonacci)
1898. Iterative DP / Simulation
1899. Iterative Dynamic Programming (1D).
1900. Iterative Dynamic Programming (Bottom-Up). `dp[i] = dp[i-1] + dp[i-2] + dp[i-3]`.
1901. Iterative Dynamic Programming (Substring Check)
1902. Iterative GCD + Greedy Expansion
1903. Iterative Grid Simulation.
1904. Iterative In-order Traversal with Early Stop
1905. Iterative In-Place Reversal (Three Pointers: prev, curr, next)
1906. Iterative Mapping (New Array)
1907. Iterative Mirroring / Bitwise Formula
1908. Iterative Modulo (%) and Division (/)
1909. Iterative Modulo and Division.
1910. Iterative New Array Push
1911. Iterative Pointer Manipulation (Bottom-Up Logic).
1912. Iterative Pointer Mutation (Visualizing the CPU cycle overhead for millions of small increments).
1913. Iterative Simulation with Window Functions.
1914. Iterative Slicing (Step = Size)
1915. Iterative Stack (Depth Tracking).
1916. Iterative Stack / Pointer Weaving When you hit a `child`, "insert" the child branch between `current` and `current.next`.
1917. Iterative StringBuilder / Two-Pointer
1918. Iterative Traversal using Two Stacks (or One Stack + Reverse).
1919. Iterative Traversal with a Stack.
1920. Iterative with Stack (or Morris)
1921. JavaScript Generators & Promises
1922. JavaScript Prototype Chain Traversal
1923. JavaScript Proxy (`get` trap)
1924. JOIN + Aggregation (HAVING Count Match)
1925. JOIN + Conditional Aggregation
1926. JOIN + Distinct Filtering
1927. Join + Group By (Install Date Logic).
1928. Join + Group By + AVG.
1929. JOIN + GROUP BY + CASE/IF logic
1930. JOIN + GROUP BY + HAVING
1931. Join + Group By + Limit.
1932. Join + Group By with Filtered Aggregation
1933. JOIN + SUM Aggregation
1934. JOIN + Window Function (RANK)
1935. Join Filtering + Order By Clause
1936. Join with a GROUP BY subquery or Window Function.

1937. K-way Merge (Dynamic Programming with Multiple Pointers)
1938. Kadane's (Min/Max Complement)
1939. Kadane's + Total Sum Logic
1940. Kadane's Algorithm
1941. Kadane's Algorithm on Difference Array
1942. Kadane's Algorithm or Sliding Window with Prefix Sums. If Value = Sum, it's strictly Kadane's.
1943. Kadaneâs Algorithm (Greedy/DP)
1944. Kadaneâs Algorithm Variation
1945. Kahn's Algorithm (Topological Sort) + Multi-source BFS
1946. Key Normalization + Frequency Map
1947. KMP (LPS Array) + Three-Pointer Greedy Slide. O(N + M)
1948. KMP (String Matching) + Matrix Exponentiation (DP)
1949. KMP / String Matching (or O(N^2) if constraints allow)
1950. KMP / Z-Algorithm on Trend Array
1951. KMP Algorithm (LPS Array)
1952. KMP Algorithm (Prefix Function)
1953. KMP or Optimized Built-in Search
1954. KMP Partial Match Table (LPS Array)
1955. KMP Pattern Matching on Tree
1956. KMP Prefix Function (LPS Array)
1957. KMP Prefix Function (LPS) + Monotonic Two Pointers
1958. KMP String Matching + Binary Search / Two Pointers
1959. KMP with Stream State
1960. Knuth-Morris-Pratt (KMP) Automaton
1961. Kruskal's / Prim's with Virtual Root
1962. Kruskal's Algorithm (Greedy + DSU)
1963. Kruskal's Algorithm with Disjoint Set Union (DSU) or Tarjan's Bridge Finding Algorithm.
1964. Kruskal's MST + LCA (Lowest Common Ancestor) with Binary Lifting.
1965. Kruskalâs + Edge Exclusion/Inclusion
1966. Language-native sort with a Custom Key/Comparator.
1967. Large Intermediate Join (Visualizing the DB loading every single booking record into memory before counting).
1968. Largest Suffix Algorithm (Two Pointers) + Substring Truncation. (O(N))
1969. Lazy Evaluation / Two Pointers / State Machine.
1970. Lazy Flipping (Flag + Two Arrays).
1971. Lazy Iteration + Stack/List Cache
1972. Lazy Iterator with Stack (Depth-First Search)
1973. Lazy Propagation (Difference Array Logic)
1974. LCA (Lowest Common Ancestor) + Binary Lifting + Prefix Bitmasks.
1975. LCA (Lowest Common Ancestor) + Binary Lifting + Prefix Frequencies
1976. LCA (Lowest Common Ancestor) + Path Comparison
1977. LCA + Distance Formula
1978. LCP Array (Longest Common Prefix) / Z-Algorithm

1979. LCRS (Recursive Pointer Re-assignment) Use a recursive DFS to transform pointers without creating new nodes.
1980. LEFT JOIN (Hash Join)
1981. LEFT JOIN + Aggregation
1982. Left Join + CASE Statement + Aggregate Functions
1983. LEFT JOIN + COALESCE (Conditional Logic)
1984. LEFT JOIN + COALESCE / IFNULL
1985. LEFT JOIN + Conditional Aggregate
1986. Left Join + Filtering + Having.
1987. Left Join + Group By (SQL)
1988. LEFT JOIN + GROUP BY + COALESCE
1989. LEFT JOIN + GROUP BY + DISTINCT
1990. LEFT JOIN + GROUP BY + IFNULL/COALESCE
1991. Left Join + Group By.
1992. LEFT JOIN + NULL Filtering
1993. LEFT JOIN / WHERE IS NULL
1994. LEFT JOIN on Interval + GROUP BY
1995. LEFT JOIN where Orders.Id IS NULL (or NOT IN with subquery).
1996. LEFT JOIN with COALESCE / IFNULL
1997. LEFT JOIN with COALESCE / IFNULL.
1998. LEFT JOIN with Conditional Aggregation
1999. LEFT JOIN with NULL check / EXCEPT operator
2000. Left Join with Null/Value Filtering.
2001. LEFT JOIN with time range condition + IS NULL check.
2002. Length Counting + Mathematical Chunking
2003. Length Sort + KMP (or Suffix Tree)
2004. Level Order Traversal (BFS) + Sorting/Min-Heap.
2005. Level-by-Level Linked List Traversal (using existing `next` pointers to traverse horizontally).
2006. Level-Order BFS (Null Detection)
2007. Level-Order BFS (Same Level, Different Parent)
2008. Level-Order BFS + Frequency Map Sort
2009. Lexical Scope / Closures
2010. Line Sweep + Segment Tree (or Coordinate Compression).
2011. Line Sweep / Prefix Sums / 2D Difference Arrays.
2012. Line Sweep Algorithm (Prefix Sum over time) OR Min-Heap
2013. Line Sweep Algorithm + DP with Segment Tree.
2014. Line Sweep Algorithm + TreeMap (Difference Array)
2015. Line Sweep Algorithm + TreeMap (Event Sorting)
2016. Line Sweep Algorithm / Difference Array
2017. Linear Algebra (System of Equations)
2018. Linear Array Scan with Modulo Arithmetic (comparing absolute distance vs circular wrap-around distance).
2019. Linear Array Scanning
2020. Linear Basis (Gaussian Elimination for XOR). Find a set of basis vectors that

can represent any possible XOR sum of the array.
2021. Linear Boundary Comparison
2022. Linear Count Tracker
2023. Linear Counter + Math
2024. Linear DP (Climbing Stairs Variant)
2025. Linear DP + Monotonic Optimization
2026. Linear Dynamic Programming (DP) - Staircase variation.
2027. Linear Expression Reduction (Parsing).
2028. Linear Filter Pipeline + Custom Sort
2029. Linear Iteration (Math/Simulation)
2030. Linear loop checking `nums[i]-nums[i-1]` + one explicit edge check for `nums[N-1]-nums[0]`. O(N)
2031. Linear Memory Scan (Visualizing the CPU reading the entire marbles array for every single instruction).
2032. Linear One-Pass Update
2033. Linear Row Scan + Tracker
2034. Linear Scan (Manual Digit Check)
2035. Linear Scan (Max Property Observation)
2036. Linear Scan (Min/Max Tracking)
2037. Linear Scan + Digit Math.
2038. Linear Scan / Adjacent Identical Pairs Math
2039. Linear Scan / Counter
2040. Linear Scan / Fixed Sliding Window
2041. Linear Scan / String Splitting.
2042. Linear Scan / Substring Search.
2043. Linear Scan of Adjacent Elements.
2044. Linear Scan with Consecutive Counter. O(N)
2045. Linear Scan with Mathematical Digit Extraction.
2046. Linear Scan with State Tracker
2047. Linear Search Overhead (Visualizing indexOf being called repeatedly).
2048. Linear Search Pointers (Drawing the search range for every individual query).
2049. Linear Simulation / Iteration
2050. Linear Strike Counting (Math)
2051. Linear Traversal (Bidirectional)
2052. Linked Hash Map / Queue + Hash Map
2053. Linked List Traversal with a Dummy Head (tracking the start of the next level).
2054. LIS (Longest Increasing Subsequence) + LDS (Decreasing)
2055. LIS-style DP + Coordinate Compression + Fenwick Tree
2056. List Maintenance + Pointer Tracking
2057. List of Coordinates + Delta Frequency Map.
2058. Local Incremental Update
2059. Logarithmic Digit Counting / Range Check
2060. Logical Comparison of First Two Edges.
2061. Logical Invariance (Existence of '1').
2062. LogTrick / Hash Map (Monotonic Bitwise AND)

2063. Long Division + Hash Map (Remainder Tracking).
2064. Longest Common Substring (LCS Variant)
2065. Longest Consecutive Sequence (Greedy)
2066. Longest Non-Decreasing Subsequence (Patience Sorting / Binary Search).
2067. Look-Ahead (Caching)
2068. Loop pointers footprint (Draw i and j pointers constantly jumping back and forth across an array).
2069. Lowest Common Ancestor (LCA) + Node Depth.
2070. Lowest Common Ancestor (LCA) via integer division (node / 2) in a Complete Binary Tree.
2071. Lowest Common Ancestor (LCA) with Binary Lifting.
2072. Manacher's Algorithm + Prefix/Suffix Max
2073. Manacher's Algorithm + Suffix Automaton / Hash.
2074. Manhattan Distance Comparison
2075. Map of Min-Heaps (Group by i - j)
2076. Massive NxN Grid Matrix (Highlighting the memory waste of storing every direct edge).
2077. Massive Set Allocation (Visualizing thousands of HashSets being created and destroyed in memory).
2078. Massive Sort Buffer Allocation (Drawing the memory used to sort N^2 subarrays).
2079. Massive String Heap (Visualizing huge strings being copied and stored at every level of recursion).
2080. Massive String Reallocations (Immutable strings mean creating a brand new string in memory for every deletion).
2081. Math
2082. Math (Algebraic Equation).
2083. Math (Arithmetic Progression Series).
2084. Math (Arithmetic Progression)
2085. Math (Combinatorics + Stars and Bars)
2086. Math (Combinatorics) + Filtering.
2087. Math (Contribution to Sum) + Combinatorics
2088. Math (Cross Product / Slope Comparison).
2089. Math (Digit Extraction and Remainder).
2090. Math (Direct Search is already efficient given N <= 10^5)
2091. Math (Distance Squared Comparison).
2092. Math (Distributive Law): (a^b) AND (c^d).
2093. Math (Euclidean GCD Logic).
2094. Math (Euclidean-like division).
2095. Math (Expression Expansion / 8 Scenarios)
2096. Math (FOIL Method) + String Split.
2097. Math (Median Finding) + Divisibility Check
2098. Math (Modular Inverse) + Global Modifiers
2099. Math (Modulo 3 Categorization) & Case Analysis
2100. Math (Modulo Arithmetic / Bitwise)

2101. Math (Modulo Arithmetic / Pigeonhole Principle).
2102. Math (Modulo Arithmetic)
2103. Math (Modulo with Negative Base).
2104. Math (Observation)
2105. Math (Parity check): Sum of ASCII value of letter and integer value of number.
2106. Math (Parity tracking). Count negative numbers and check for zeros.
2107. Math (Perfect Square Property)
2108. Math (Running Modulo).
2109. Math (Sieve-like approach). Check if a target GCD 'x' can be formed by iterating its multiples.
2110. Math (Stars and Bars) / DP
2111. Math (Suffix Length Precalculation)
2112. Math (Summation / XOR)
2113. Math (Symmetry / Bit Manipulation).
2114. Math (The Josephus Problem) / Bottom-Up DP.
2115. Math (Trailing Zeros + Modulo/Logarithms).
2116. Math (Triangular Numbers) or DP.
2117. Math + Prefix Sums of "Division Counts". O(Q log(Range)) or O(Q Â· 1).
2118. Math / Binary Pattern Recognition.
2119. Math / Binomial Coefficients
2120. Math / Combinatorics (Stirling Numbers of the Second Kind)
2121. Math / Contiguous Counting
2122. Math / Contribution Technique
2123. Math / Digit DP.
2124. Math / Edge-Delta Optimization
2125. Math / Geometry (Closest Point)
2126. Math / Geometry (Line of Sight checks)
2127. Math / Greedy / Min-Max
2128. Math / Greedy Distribution (Base + Remainder)
2129. Math / Greedy Observation.
2130. Math / Half-String Mirroring
2131. Math / Hash Set (Min/Max finding)
2132. Math / Index Hopping
2133. Math / Line Intersection (Convert MMDD to Day of Year Integer)
2134. Math / Modular Arithmetic
2135. Math / Modulo Arithmetic.
2136. Math / Number Theory (Global Minimum modulo reduction)
2137. Math / Observation (O(1)).
2138. Math / Pair Divisors up to Square Root of N.
2139. Math / Parity Analysis.
2140. Math / Permutation Pruning (Binary Search style)
2141. Math / Prefix Sum (Tracking flow and bottlenecks).
2142. Math / Simulation
2143. Math / Simulation (Repeated Division and Modulo).
2144. Math / Square Root Optimization

2145. Math Bounds + Hash Set
2146. Math Formula
2147. Math Observation (Bubblesort Property).
2148. Math Optimization + Prefix Map (Advanced)
2149. Math Rearrangement + Hash Map: Check nums[i] - rev(nums[i]) == nums[j] - rev(nums[j]).
2150. Math-Based Accumulator
2151. Math-based column parity (Odd/Even columns)
2152. Math-based digit extraction (Modulo/Division).
2153. Math/Graph (Cycle Detection). Track only the movement of index 1.
2154. Math/Index Mapping (i / n, i % n)
2155. Math/Modulo Zero-Check Loop
2156. Mathematical Base Conversion (N % K, N // K) using a StringBuilder/Byte Array, followed by a single Reverse operation.
2157. Mathematical Binning (Floor Division)
2158. Mathematical Bitwise Cancellation (Symmetry). The expression simplifies to just XORing all elements in the array.
2159. Mathematical Ceiling Division.
2160. Mathematical Constant Division (Base-10 Log or Max Int)
2161. Mathematical Digit Decomposition
2162. Mathematical Digit Extraction
2163. Mathematical Digit Extraction (Modulo & Division)
2164. Mathematical Digit Extraction + Hash Set
2165. Mathematical Digit Extraction.
2166. Mathematical Digit Summing + Array Frequency Map.
2167. Mathematical DP (Linear Recurrence)
2168. Mathematical Greedy + Parity Check
2169. Mathematical Greedy Partitioning
2170. Mathematical Induction / Constant Observation
2171. Mathematical Insight (Constant Return)
2172. Mathematical Insight (Elimination)
2173. Mathematical Interval Binary Search
2174. Mathematical Invariant / Game Theory
2175. Mathematical Least Common Multiple (LCM).
2176. Mathematical Modulo (Game Theory)
2177. Mathematical Modulo-10 Digit Extraction.
2178. Mathematical Observation (Cascade Effect).
2179. Mathematical Observation (Parity) OR 2D DP.
2180. Mathematical Observation (Powers of 2)
2181. Mathematical Offset Calculation (f(R) - f(L-1)).
2182. Mathematical Parity / Bottleneck Check
2183. Mathematical Pattern (Square Root)
2184. Mathematical Pattern Reduction.
2185. Mathematical Progression (Sum of Arithmetic Series).
2186. Mathematical Proof / Bit Manipulation (XOR properties).

2187. Mathematical Property (Perfect Squares).
2188. Mathematical Range Summation.
2189. Mathematical Reduction Incrementing n-1 elements is mathematically equivalent to decrementing 1 element. Sum up the distance of each element from the minimum.
2190. Mathematical Relative Position
2191. Mathematical Remainder Reversal
2192. Mathematical Series (Tetrahedral Numbers) + Greedy.
2193. Mathematical Simulation (Step & Head Tracking)
2194. Matrix Exponentiation
2195. Matrix Exponentiation / 1D DP.
2196. Matrix Marking / State Grid
2197. Matrix Traversal (Local Constraint)
2198. Matrix Traversal + O(âN) Prime Check
2199. Max Heap (Priority Queue)
2200. Max Heap + Absolute Value Sorting
2201. Max Heap / Binary Search on Sums
2202. Max Heap / Priority Queue (Greedy)
2203. MAX() with Conditional Aggregation (CASE/IF)
2204. Max-Heap (Priority Queue) of "Potential Gain".
2205. Max-Heap (Priority Queue) of Intervals.
2206. Max-Heap (Priority Queue) to maintain the largest element dynamically.
2207. Max-Heap (Priority Queue).
2208. Max-Heap (Size K) or QuickSelect
2209. Max-Heap + Cooling Queue (Greedy)
2210. Max-Heap / Greedy with Interleaving
2211. Max-Heap / K-way Merge (Pull Model)
2212. Max-Priority Queue (Heap)
2213. Max-Value Tracking (Greedy)
2214. Maximum Flow / Bipartite Matching. Model robots as one set of nodes, walls as another, and find the maximum matching/coverage.
2215. Median + Palindrome Generation
2216. Median Property (Independent Dimensions)
2217. Median Target (Greedy) The point that minimizes the sum of absolute differences is the median. Sort and find the middle.
2218. Median-Based Greedy approach. The total cost is minimized when all elements are changed to the median of the array.
2219. Median-based Greedy.
2220. Meet in the Middle + Binary Search
2221. Meet-in-the-Middle + Binary Search / Trie.
2222. Meet-in-the-Middle + Dynamic Programming / Math Pruning.
2223. Meet-in-the-Middle + Two Pointers.
2224. Memoized Recursion / Dynamic Programming on (n, firstPlayer, secondPlayer).
2225. Merge Intervals + Binary Search
2226. Merge Intervals + Combinatorics

2227. Merge Intervals / Segment Tree / Jump Pointers (DSU).
2228. Merge Sort (Bottom-Up Iterative for O(1) space, or Top-Down Recursive).
2229. Merge Sort (Divide & Conquer) During the merge step, both halves are sorted. Use two pointers to count reverse pairs before merging.
2230. Merge Sort (Divide & Conquer) or Fenwick Tree after transforming to (nums1[i]-nums2[i])
2231. Merge Sort (Divide & Conquer) tracking original indices
2232. Merge Sort / Quick Sort / Heap Sort (Divide & Conquer)
2233. Merkle Hashing or String Serialization (KMP).
2234. Middle Character Extraction
2235. Middle Element Fix + Left/Right Max Search
2236. Min Heap (Priority Queue)
2237. Min-Cost Flow / Hungarian Algorithm
2238. Min-Element Comparison / Math
2239. Min-Heap (K-Way Merge)
2240. Min-Heap (Priority Queue)
2241. Min-Heap (Priority Queue) of End Times
2242. Min-Heap (Priority Queue) tracking `[rot_day, count]`.
2243. Min-Heap (Priority Queue) with Dijkstra-like Expansion
2244. Min-Heap (Size K)
2245. Min-Heap + BFS Push perimeter to min-heap. Pop lowest boundary, process neighbors, add to heap. "Shrinking perimeter".
2246. Min-Heap + Boolean Visited Array
2247. Min-Heap + Hash Set (or Min_Tracker)
2248. Min-Heap + List of Stacks
2249. Min-Heap + Map of Sets
2250. Min-Heap / Priority Queue (Multi-way Merge)
2251. Min-Max Coordinate Tracking
2252. Min-Priority Queue (Heap)
2253. Min-Priority Queue (Min-Heap) sorted by processing time and original index.
2254. Min-Priority Queue (Min-Heap).
2255. Min/Max Tracking.
2256. Minimal Memory (Only tracking unique category representatives).
2257. Minimal Variable State (Only index i).
2258. Minimax / Information Theory (Heuristic Guessing).
2259. Minimax / Topological Sort on States
2260. Minimum Boundary Tracking.
2261. Mirroring + Middle-Digit Increment. Construct the palindrome from the first half of N, then adjust based on "Special" digit constraints.
2262. Mismatch Index Comparison.
2263. Modified BFS (Dijkstra-like) with Top-2 Tracking + Traffic Light Math
2264. Modified Binary Search
2265. Modified Dijkstra (or 2D DP)
2266. Modified Dijkstra (State-Space Expansion)
2267. Modified Dijkstra / BFS with State

2268. Modified Dijkstra / Min-Heap BFS
2269. Modified Dijkstra's Algorithm (Min-Heap)
2270. Modified Dijkstra's Algorithm (Shortest Path on weighted states).
2271. Modified Kadane's Algorithm for each distinct value T. O(50 Â· N)
2272. Modified Merge Sort (Divide & Conquer)
2273. Modular Arithmetic & Hash Map (Prefix Modulos).
2274. Modular Arithmetic (Cycles).
2275. Modular Arithmetic (Iterative Remainder).
2276. Modular Arithmetic + Hash Map (Digit Length Bins).
2277. Modular Exponentiation + Recursive Property
2278. Modulo Arithmetic (%)
2279. Modulo Arithmetic (Circular Indexing)
2280. Modulo Arithmetic Indexing
2281. Modulo Backtracking (Reverse Engineering)
2282. Modulo Digit Extraction
2283. Modulo Frequency Mapping (Remainders).
2284. Modulo Math + Sliding Window on Doubled Array
2285. Modulo Simulation
2286. Modulo Sliding Window (Size 3)
2287. Modulo-based Frequency Array. Use `n % 10` to strip digits and an array of size 10 to store counts.
2288. Monotone Chain Algorithm (Sorting + Cross Product).
2289. Monotonic Decreasing Stack
2290. Monotonic Decreasing Stack (Looping 2N times).
2291. Monotonic Deque (Double-Ended Queue)
2292. Monotonic Deque (Sliding Window Max)
2293. Monotonic Deque + Prefix Sums.
2294. Monotonic Deque / Two Pointers + Sliding Window Minimum.
2295. Monotonic Stack
2296. Monotonic Stack (Decreasing)
2297. Monotonic Stack (Greedy)
2298. Monotonic Stack (Increasing)
2299. Monotonic Stack (Next Greater Element)
2300. Monotonic Stack (Next Smaller Element)
2301. Monotonic Stack (O(N)).
2302. Monotonic Stack (Pair of Price and Span).
2303. Monotonic Stack (Parent Tracking)
2304. Monotonic Stack (Right-to-Left) + Collision Time Comparison.
2305. Monotonic Stack (Right-to-Left) Keep a stack of potential "2" values (middle magnitude) and track the largest "3" value found so far.
2306. Monotonic Stack (Row/Col wise)
2307. Monotonic Stack (Simulation of DFS)
2308. Monotonic Stack (Time-to-Die)
2309. Monotonic Stack + 1D DP
2310. Monotonic Stack + Contribution of Each Element + Math (Counting valid

subarrays). O(N)
2311. Monotonic Stack + DP (Suffix Jump Mapping)
2312. Monotonic Stack + Dynamic Programming
2313. Monotonic Stack + Frequency Map + Visited Array.
2314. Monotonic Stack + Greedy (Frequency Map).
2315. Monotonic Stack + Greedy + Frequency Map
2316. Monotonic Stack + HashMap Pre-process nums2 using a stack to find all "Next Greater" pairs. Store in a Map `num: next_greater`.
2317. Monotonic Stack + Lexicographical Merge
2318. Monotonic Stack + Prefix Sum of Prefix Sums
2319. Monotonic Stack + Prefix/Suffix DP
2320. Monotonic Stack + Range Extension
2321. Monotonic Stack + Sieve of Eratosthenes + Greedy (Contribution Technique)
2322. Monotonic Stack + Two Pointers
2323. Monotonic Stack + Two Pointers (Right-to-Left)
2324. Monotonic Stack / Greedy
2325. Monotonic Stack / Greedy Suffix Max
2326. Monotonic Stack / Histogram
2327. Monotonic Stack Keep digits in increasing order. Pop if current digit < top of stack and K > 0.
2328. Monotonic Stack or Reverse Recursion
2329. Monotonic Stack to find the next/previous greater/smaller elements in O(N), then Dijkstra/DP on those specific edges.
2330. Monotonic Stack with Budget Constraints
2331. Monotonic Stack.
2332. Morris Inorder Traversal
2333. Multi-Dimensional Dynamic Programming (State Compression).
2334. Multi-Level LEFT JOIN with Grouping
2335. Multi-Level Sorting (Frequency Matrix)
2336. Multi-pass Two Pointers (In-Place).
2337. Multi-Queue Interleaving
2338. Multi-Source BFS
2339. Multi-source BFS (Water) + Standard BFS (Player)
2340. Multi-Source BFS + Arrival Time Comparison
2341. Multi-source BFS + Max-Heap/Dijkstra (or Binary Search + BFS)
2342. Multi-source BFS + Priority Queue (Dijkstra Variant).
2343. Multi-Source BFS / Dijkstra with Coordinate Mapping.
2344. Multi-Source BFS or 2-Pass DP.
2345. Multi-Source BFS.
2346. Multi-source BFS/DFS Start from the Pacific edges and "flow uphill." Do the same for Atlantic. Find the intersection.
2347. Multi-source Breadth-First Search (BFS).
2348. Multi-source DFS/BFS from Boundaries.
2349. Multi-Source Dijkstra / Graph Reversal
2350. Multiple Boolean Flags (Visualizing all the manual state variables needed for

date math).
2351. Multiple Statement Contexts (Each call is a separate statement in the execution stack).
2352. Multiple TreeSets / Priority Queues + HashMaps
2353. Native Pandas Constructor (Vectorized Internal Logic)
2354. Nested BFS / State BFS (Box Pos + Person Pos)
2355. Nested Char Loops.
2356. Nested Loop Sliding Window with Running Set
2357. Nested Loop with Running Frequency Array.
2358. Nested Maps / Prefix Tree (Trie)
2359. Net Displacement Counter.
2360. Net Shift Calculation + Slicing
2361. Network Socket Overload (Visualizing multiple open connections consuming system file descriptors).
2362. New Grid Mapping.
2363. Next Permutation (K times) + Greedy Swap Alignment.
2364. Next Permutation Algorithm (Linear Scan).
2365. Next Permutation Algorithm (Suffix Reversal)
2366. Next Permutation Algorithm on the first half.
2367. Next Permutation on Half-String
2368. Number Theory (GCD + Frequency Map).
2369. Number Theory (GCD + Power of Two).
2370. Number Theory (GCD / BÃ©zout's Identity)
2371. Number Theory (Greatest Common Divisor)
2372. Number Theory / Sieve of Eratosthenes
2373. O(1) Combinatorics (Parity Math)
2374. O(1) State Variables (Draw one sum and one i).
2375. O(M^2) Simulation (M = instruction length).
2376. O(MÂ· N) Matrix Allocation (Massive memory waste for grids like 10^5 x 10^5).
2377. O(N^2) 2D Array Allocation (Massive memory waste for large N).
2378. Object-Oriented Design (Polymorphism) + Postfix Stack Tree
2379. Object.entries() + Array.reduce()
2380. Odd Frequency Character Counting
2381. Offline Queries (Sort Queries) + Min-Heap (Priority Queue) + BFS to prevent re-processing nodes.
2382. Offline Queries + Bitwise Trie.
2383. Offline Queries + DFS + Binary Trie
2384. Offline Queries + DSU with Sorting.
2385. Offline Queries + Monotonic Stack / Priority Queue
2386. Offline Queries + Sorting + Balanced BST (TreeSet).
2387. Offline Queries + Sorting + Min-Priority Queue (Heap).
2388. Offline Queries + Union-Find (DSU)
2389. On-the-fly Trial Division + Hash Set deduplication.
2390. One-Pass DP / Sliding Window (Arithmetic Progressions)
2391. One-Pass Greedy (Peak-Valley).

2392. One-pass Greedy / Kadaneâs-like Sliding Window.
2393. One-Pass Greedy / Simulation
2394. One-Pass Greedy Iterate through attacks. If the gap between attacks < duration, add the gap. Else, add the full duration.
2395. One-Pass Greedy Swapping
2396. One-Pass Mathematical Summation
2397. One-Pass Min-Max Tracking
2398. One-Pass Neighbor Comparison
2399. One-Pass Parity Tally
2400. One-Pass Precomputation
2401. One-Pass Running Maximum
2402. One-pass Sliding Window / Greedy.
2403. One-Pass String Comparison.
2404. One-Pass with Toggle/Previous Tracking
2405. Optimal Brute (Observation: GCD only decreases)
2406. Optimised Monotonic Stack + Dynamic Programming
2407. Optimized BFS / DFS on Digit States
2408. Optimized Brute Force with LCM Early Exit
2409. Optimized Linear Scan (Early Exit)
2410. Optimized Sliding Window (Variable Frequency)
2411. ORDER BY Multiple Columns
2412. Ordered Map (TreeMap) / Balanced BST
2413. Ordered Map / TreeMap with Granularity Masking.
2414. Palindrome Construction (Math/Generator)
2415. Palindrome Generation (Length-based)
2416. Palindrome Generation + Combinatorial Counting
2417. Palindrome Generation + Primality Test.
2418. Parallel StringBuilder Appending
2419. Parity Counting (Math/Greedy)
2420. Parity Counting (Row/Col Arrays)
2421. Parity Logic (Difference is even iff Total Sum is even). O(N)
2422. Parity Summation
2423. Parity-Based Character Comparison
2424. Parity-Based Math (Even/Odd Geometry)
2425. Partition Memory Sets (Visualizing two massive separate heaps for every single partition attempt).
2426. Path Frequency Maps (Visualizing thousands of HashMaps created to check character counts for every path).
2427. Patience Sorting (Binary Search + Tails Array)
2428. Patience Sorting (LIS Variation)
2429. Patience Sorting / Segment Tree (LIS variant)
2430. Pattern Canonicalization / Hashing.
2431. Pattern Categorization (6-Segment Window)
2432. Pattern Conversion + KMP Algorithm
2433. Pattern Searching (Presence of "01").

2434. pd.concat([df1, df2])
2435. Perimeter Modulo Math
2436. Perimeter Unwrapping + Binary Search on Answer + Greedy. O(N log N + N log side)
2437. Permutation Validation (Fixed Size)
2438. Persistent Pointer (Amortized Analysis)
2439. Pigeonhole Principle (Frequency Counting)
2440. Pigeonhole Principle + Iterative Modulo
2441. Pivot Table (CASE WHEN + GROUP BY).
2442. Pointer Manipulation + Dummy Node
2443. Pointer Simulation on Compressed Pairs.
2444. Pointer-based Simulation
2445. Polar Angle Sorting + Sliding Window
2446. Polynomial Class / Map of Term-to-Coefficient
2447. Polynomial Rolling Sum (Base-26 conversion).
2448. Post-Order DFS (Bottom-Up Flagging)
2449. Post-Order DFS (Bottom-Up Size Tally)
2450. Post-Order DFS (Bottom-Up State)
2451. Post-Order DFS (Bottom-Up)
2452. Post-order DFS (Bottom-Up)
2453. Post-order DFS (Bottom-Up).
2454. Post-order DFS (Depth-Pair Return)
2455. Post-Order DFS (Depth-Pair Return).
2456. Post-Order DFS (Tree DP)
2457. Post-Order DFS Sum Tracking
2458. Post-order DFS Traversal + Hash Set
2459. Post-Order Recursive Reconstruction
2460. Post-order Subtree Sums + Hash Set.
2461. Post-order Traversal (Bottom-Up DFS) + Hash Map.
2462. Post-Order Traversal (Bottom-Up DFS).
2463. Post-order Traversal (Bottom-Up DFS). 1 + max(left, right)
2464. Post-order Traversal (Bottom-Up DP)
2465. Post-order Traversal (Bottom-Up Information Passing)
2466. Post-order Traversal (Bottom-Up Pruning).
2467. Post-Order Traversal (Bottom-Up Summing)
2468. Post-order Traversal (Bottom-Up Summing).
2469. Post-order Traversal (Bottom-Up)
2470. Post-Order Traversal (Bottom-Up)
2471. Post-Order Traversal (DFS)
2472. Post-Order Traversal (Excess/Deficit Flow)
2473. Post-order Traversal + Serialization + Hash Map.
2474. Power of Two Check (Bit Manipulation).
2475. Pre-allocation + Linear Fill
2476. Pre-calculation + Stable Sorting.
2477. Pre-computation + Binary Search/Suffix Sum

2478. Pre-computation with Hash Set
2479. Pre-computed Array + Leap Year Math
2480. Pre-evaluated Boolean Variables mapping to strict logic combinations.
2481. Pre-mapped Char-to-Row Create a Map or Array of size 26 where `row[char]` is 1, 2, or 3. Check if all chars in a word return the same row ID.
2482. Pre-order + Bounds Use Pre-order without null markers. Rebuild using BST property (current value must be within [min, max]).
2483. Pre-order / In-order / Post-order Coordination.
2484. Pre-order Traversal (Bitwise Accumulation).
2485. Pre-Order Traversal (DFS) with Sentinel Markers
2486. Pre-order with Child Count Serialize as `Value, ChildCount`. E.g., `1,3,2,0,3,0,4,0` (Node 1 has 3 children...).
2487. Pre-processing (Index Collection)
2488. Precomputation (2D Grid Greedy).
2489. Precomputation (Prefix Products)
2490. Precomputation (Row/Col Sum Arrays)
2491. Precomputation + 2D Partition Dynamic Programming
2492. Precomputation + Binary Search
2493. Precomputation / Backtracking with Pruning
2494. Precomputation / Bitmasking
2495. Precomputation / Combinatorics
2496. Precomputation of All Substrings (up to 30 bits) + Hash Map.
2497. Precompute "Longest Decreasing" from left and "Longest Increasing" from right for every index.
2498. Precompute dependencies and apply a Greedy "Earliest Start" calculation.
2499. Precomputed Frequency Map + Single Pass. O(N)
2500. Precomputed HashMaps (Priority Tiering)
2501. Precomputed Pairwise Frequency (Hashmap)
2502. Precomputed Prefix/Suffix Counts (DP)
2503. Precomputed Row-Mins and Col-Maxes
2504. Prefix & Suffix Arrays
2505. Prefix & Suffix Arrays (State Tracking)
2506. Prefix & Suffix Products (Flattened 2D to 1D)
2507. Prefix and Suffix Accumulators
2508. Prefix and Suffix Counts. Count existing '0's and '1's. Inserting '0' at the start pairs with all existing '1's. Inserting '1' at the end pairs with all existing '0's.
2509. Prefix and Suffix Precomputation Arrays.
2510. Prefix and Suffix Running Totals.
2511. Prefix and Suffix Sums / Hash Map of Lists.
2512. Prefix and Suffix Sums. Precalculate even/odd sums. When you "remove" an element, the suffix even/odd sums simply swap roles!
2513. Prefix Expansion / Direct String Construction
2514. Prefix Hash Set / Trie
2515. Prefix Manipulation + Candidate Testing.
2516. Prefix Max & Suffix Min

2517. Prefix Max & Suffix Min Pass
2518. Prefix Max / Suffix Max Arrays
2519. Prefix Max and Suffix Min Arrays
2520. Prefix Maximum & Suffix Maximum (or Pre-calculation)
2521. Prefix Min & Suffix Min Arrays
2522. Prefix Min & Suffix Min Arrays (Identical to 2908)
2523. Prefix Parity State + Sliding Window on Pair Combinations (20 pairs). O(20 Â· N)
2524. Prefix Product Array with Zero-Reset
2525. Prefix Sum & Suffix Sum Arrays (Minimax pattern)
2526. Prefix Sum (In-place)
2527. Prefix Sum (Parity Violations)
2528. Prefix Sum (Running Total).
2529. Prefix Sum (Total Sum Balance)
2530. Prefix Sum (Treat 0 as -1) + Hash Map.
2531. Prefix Sum + Binary Indexed Tree (Fenwick)
2532. Prefix Sum + Binary Indexed Tree (Fenwick) / Merge Sort
2533. Prefix Sum + Binary Search (or Sliding Window).
2534. Prefix Sum + Binary Search Calculate areas of all rectangles. Create a prefix sum array. Randomly pick a "total area" value and binary search for the rect.
2535. Prefix Sum + DFS Use a Hash Map to store "Prefix Sum -> Frequency". If (CurrentSum - Target) exists in the map, a path is found.
2536. Prefix Sum + Dynamic Programming (Suffix Max).
2537. Prefix Sum + Hash Map (Diff Tracking)
2538. Prefix Sum + Hash Map (First Occurrence)
2539. Prefix Sum + Hash Map (Math Restructuring)
2540. Prefix Sum + Hash Map (Minimum Prefix Tracking)
2541. Prefix Sum + Hash Map (Modular Arithmetic)
2542. Prefix Sum + Hash Map (Modulo Tracking)
2543. Prefix Sum + Hash Map (One/Two Pass)
2544. Prefix Sum + Hash Map (Two-Sum Variation)
2545. Prefix Sum + Hash Map / Monotonic Stack
2546. Prefix Sum + Hash Map.
2547. Prefix Sum + Hash Set + Greedy
2548. Prefix Sum + Hash Set for Mid-Point Optimization.
2549. Prefix Sum + Hashmap (or Sliding Window)
2550. Prefix Sum + Math (Parity)
2551. Prefix Sum + Modulo Arithmetic + Binary Search.
2552. Prefix Sum + Modulo Hash Map.
2553. Prefix Sum + Monotonic Stack (Next Smaller Element).
2554. Prefix Sum + Range Math.
2555. Prefix Sum / Minimum Running Sum
2556. Prefix Sum / Running Count.
2557. Prefix Sum / Running Total
2558. Prefix Sum / Total Sum Tracking

2559. Prefix Sum Array
2560. Prefix Sum Array + Binary Search.
2561. Prefix Sum Array + Leap Year Logic
2562. Prefix Sum Array.
2563. Prefix Sum Array. O(N)
2564. Prefix Sum Average (Greedy Ceiling)
2565. Prefix Sum Balance
2566. Prefix Sum Difference + Hash Map
2567. Prefix Sum Hash Map / Two-Sum Pattern
2568. Prefix Sum Modulo K + Trie/String Hashing
2569. Prefix Sum on Trees (Parity)
2570. Prefix Sum Range (Max - Min).
2571. Prefix Sum Simulation
2572. Prefix Sum Transformation + Fenwick Tree / Sliding Window.
2573. Prefix Sums (Brute Force is expected for N=1000)
2574. Prefix Sums (Cost Optimization)
2575. Prefix Sums (Even/Odd Tracking)
2576. Prefix Sums (Mathematical Simplification)
2577. Prefix Sums + Binary Search (or Two Pointers).
2578. Prefix Sums + Bitmasking
2579. Prefix Sums + Combinatorics (nC2)
2580. Prefix Sums + First Occurrence Hash Map.
2581. Prefix Sums + Frequency Counting (HashMap).
2582. Prefix Sums + Frequency Map (Balance Tracking)
2583. Prefix Sums + Frequency Map (Remainder Counting)
2584. Prefix Sums + Frequency Maps.
2585. Prefix Sums + Hash Map
2586. Prefix Sums + Hash Map (Frequency of remainders)
2587. Prefix Sums + Hash Map (Frequency Tracking)
2588. Prefix Sums + Hash Maps for Difference Tracking
2589. Prefix Sums + Last Occurrence Tracking
2590. Prefix Sums + Nearest Candle Precomputation
2591. Prefix Sums + Parity Symmetry Check
2592. Prefix Sums + Range Intersection (Inequality Check).
2593. Prefix Sums + Relative State Tracking (+1/-1)
2594. Prefix Sums / Difference Array (Line Sweep)
2595. Prefix Sums / Frequency Mapping.
2596. Prefix Sums / Running Totals
2597. Prefix Sums / Running Totals.
2598. Prefix Sums / Single Pass Logic
2599. Prefix Sums of Coordinates. Calculate (x, y) for all prefixes and suffixes. Endpoint = Prefix[i-1] + (Total - Prefix[j]) .
2600. Prefix Sums of Skill + Gap Analysis Formula. O(N Â· M)
2601. Prefix Sums on 2D Coordinates / Hash Map caching.
2602. Prefix Sums on Doubled Array + Median of Differences

2603. Prefix Sums over Frequency Array
2604. Prefix Sums. Calculate all cumulative sums once. Sum of right half = TotalSum - PrefixSum[i] .
2605. Prefix XOR + Binary Trie with Range Counting.
2606. Prefix XOR + Bitwise Complement.
2607. Prefix XOR + Dynamic Programming with HashMap.
2608. Prefix XOR + Hash Map (First Occurrence)
2609. Prefix XOR + Hash Map (Frequency Counting).
2610. Prefix XOR + Property of XOR
2611. Prefix XOR + Target Search.
2612. Prefix XOR / Greedy Mathematical Observation
2613. Prefix XOR Array (Cumulative State Cache)
2614. Prefix XOR Array combined with a Binary Trie (Prefix Tree) to count valid pairs in O(N log(Max_Value)) time.
2615. Prefix/Suffix Counts
2616. Prefix/Suffix DP (Length 5)
2617. Prefix/Suffix Dynamic Programming (Precomputation)
2618. Prefix/Suffix OR + Greedy Power of 2
2619. Preorder DFS with Conditional Appending.
2620. Prepared Statements / Dynamic SQL
2621. Preprocessing (Row/Col Sums)
2622. Prim's/Kruskal's + Greedy / DP with Knapsack.
2623. Prime Factorization (Greedy/DP).
2624. Prime Factorization + Range Merging / Interval Coverage.
2625. Prime Factorization + Stars and Bars (Combinatorics).
2626. Prime Product Hashing or Frequency Array
2627. Priority Queue (Max-Heap) + Greedy Shrinking
2628. Priority Queue (Slope Trick)
2629. Priority Queue / Array of Stacks
2630. Priority Queue Heap State (Visualizing the heap re-balancing for every element used in a group).
2631. Probabilistic Multi-Level Linked List
2632. Promise.allSettled Polyfill
2633. Promise.race() with Timer
2634. Promises + setTimeout
2635. Proxy + Structural Sharing
2636. Proxy Object / Recursive Object.freeze
2637. Pruned DFS (Early Exit)
2638. Pruning + Constant-Time Symmetry Check.
2639. Query/Pair Stack (Drawing the search range for every individual query).
2640. Queue + 3 TreeMaps (Multisets for Bottom K, Middle M-2K, Top K) + Running Sum.
2641. Queue + HashSet (Available Pool)
2642. Queue Memory (Visualizing the BFS queue filling with millions of possible intermediate numbers).

2643. Queue Memory (Visualizing the BFS queue growing linearly with K, potentially millions of strings).
2644. Queue of Iterators
2645. Queue-based Level Order State
2646. Quickselect (Hoare's Partition) OR Min-Heap
2647. Quickselect + Virtual Indexing (3-way Partition)
2648. QuickSelect Partitioning (nth_element)
2649. Radix Sort Logic / Precomputed Sorted States
2650. Range Narrowing (Iterative Intersection). O(N)
2651. Rank Mapping (Hash Map / 2D Array)
2652. Re-rooting DP (Post-order + Pre-order DFS).
2653. Recursion + Bitmask Memoization Use an integer bitmask to represent which numbers are used. Cache result in a Map `mask: win/loss`.
2654. Recursion + String Sorting (Divide & Conquer)
2655. Recursion / Divide and Conquer
2656. Recursion / Stack with HashMaps
2657. Recursion with Backtracking
2658. Recursion with Memoization
2659. Recursion with Memoization (Divide and Conquer).
2660. Recursion with Memoization (Top-Down DP)
2661. Recursion with Range (Min/Max)
2662. Recursion with Valid Range (Upper/Lower Bounds).
2663. Recursive Backtracking (Implicit Stack)
2664. Recursive Backtracking (Inside-Out Construction)
2665. Recursive Backtracking (Post-Order Traversal)
2666. Recursive Backtracking with Frequency Map (or House Robber DP).
2667. Recursive Branch Pruning.
2668. Recursive Call Stack (Visualize memory frames for every nested level of the object).
2669. Recursive Call Stack (Visualizing deep paths and redundant calculations of the same sub-problems).
2670. Recursive Call Stack (Visualizing depth-first search for the longest possible valid string path).
2671. Recursive Call Stack (Visualizing the depth-first search through all valid and invalid partition combinations).
2672. Recursive Call Stack (Visualizing the massive overhead of re-calculating the best path for the same index multiple times).
2673. Recursive Canonical Comparison
2674. Recursive Chain / Worker Pool
2675. Recursive Closure / Function Arity
2676. Recursive CTE (Common Table Expression)
2677. Recursive CTE (Date Generation) + Left Join
2678. Recursive CTE (Month Generator) + LEFT JOIN + Window Functions
2679. Recursive CTE (Range Generation) + Left Join
2680. Recursive CTE (SQL) / DFS with Aggregate Accumulation. O(N)

2681. Recursive CTE (SQL) / Graph Cycle Detection with Visited Set.
2682. Recursive CTE (WITH RECURSIVE)
2683. Recursive CTE (Year-Level Pivot)
2684. Recursive CTE + LEFT JOIN
2685. Recursive CTE + LEFT JOIN + SUM/COUNT Window Functions
2686. Recursive CTE + LEFT JOIN.
2687. Recursive CTE + Window Function (ROWS BETWEEN)
2688. Recursive Descent Parser + Scoped Symbol Table
2689. Recursive Descent Parser / Stack
2690. Recursive Descent Parsing / Stack.
2691. Recursive Descent with Operator Precedence
2692. Recursive DFS (Object Traversal)
2693. Recursive DFS (Root arrow Right arrow Left).
2694. Recursive DFS (Structural Comparison)
2695. Recursive DFS Flattening + Sorted Key Set
2696. Recursive DFS or Iterative "Reverse Preorder".
2697. Recursive DFS or Iterative Stack.
2698. Recursive DFS Serialization
2699. Recursive DFS Type Checking
2700. Recursive DFS with Depth Tracking
2701. Recursive DFS with Depth Tracking.
2702. Recursive Divide and Conquer (Sign Partitioning)
2703. Recursive Generation with Range Pruning
2704. Recursive In-Place Merge.
2705. Recursive In-place Pointer Reversal
2706. Recursive Length-Based Search
2707. Recursive Logical AND
2708. Recursive Matrix Filling (Height-based).
2709. Recursive or Iterative In-Place Swap (DFS)
2710. Recursive Pattern / Head-Pointer Tracking
2711. Recursive Pointer Re-assignment.
2712. Recursive Post-Order Split
2713. Recursive Pruning (BST Property)
2714. Recursive Range Splitting (Divide and Conquer).
2715. Recursive Re-linking Find node. If it's a leaf, remove. If 1 child, replace with child. If 2 children, replace with In-order Successor (min in right subtree).
2716. Recursive setTimeout via Closure
2717. Recursive Symmetry / Divide & Conquer
2718. Recursive Tree Merging (Divide & Conquer).
2719. Recursive Triplet Grouping (Base 1000)
2720. Recursive Yield* (Delegation)
2721. Redundant Array Rotation Buffers (Storing the state of the array for every single shift).
2722. Redundant Decision Paths (Visualizing same wall counts being re-calculated in different branches).

2723. Redundant Min/Max Scans (Drawing a search arrow within every window).
2724. Redundant String Slicing (Visualizing thousands of temporary substring copies being checked against a HashSet).
2725. Redundant Traversal Stack (Visualizing the CPU re-visiting the same bottom nodes for every ancestor).
2726. Regex (Regular Expressions) or SQL Pattern Matching. O(N)
2727. Regex Extraction + Group By + Order By
2728. Regex Matching or Structured Single-Pass Scan
2729. Regex or Modular Parsing Use pre-defined Regex patterns for IPv4 and IPv6 to handle boundaries and valid character ranges in one pass.
2730. Regex Pattern Matching (grep -P or awk).
2731. Regex/String Parsing + Hash Set + Hash Map.
2732. REGEXP_SUBSTR + Window Aggregation
2733. Regular Expression (Regex) Matching. O(N)
2734. Regular Expression (Regex) Parsing. O(N)
2735. Regular Expressions (Regex)
2736. Rejection Sampling (7x7 Matrix) Create a range of 1-49 using `7 * (rand7() - 1) + rand7()`. If the result is > 40, reject it. Otherwise, return `result % 10 + 1`.
2737. Relative Parity / Math
2738. Remainder Frequency Counting
2739. Repeated Digit Extraction (Drawing the logic to pull digits from numbers inside the nested loops).
2740. Repeated Function Execution Stack (Multiple redundant calls to the selector function).
2741. Repeated String Concatenation (Visualizing memory churn from adding one char at a time to a string).
2742. Reservoir Sampling
2743. Reservoir Sampling (Memory Efficient)
2744. Resource Hierarchy / Semaphores
2745. Reverse Encryption Mapping (Trie/Hash)
2746. Reverse Graph Traversal or Topological Sort.
2747. Reverse In-order Traversal (Right -> Root -> Left).
2748. Reverse In-Order Traversal (Right â Root â Left).
2749. Reverse Linear Scan
2750. Reverse Linear Scan (Right-to-Left)
2751. Reverse Post-order Traversal (Right, Left, Root) maintaining a global `prev` pointer.
2752. Reverse Processing + Union Find (DSU)
2753. Reverse Recursive Math / Divide and Conquer
2754. Reverse Scan + Hash Set
2755. Reverse Search (King outward to Queens)
2756. Reverse Simulation with Max-Heap
2757. Reverse Time Union-Find (Disjoint Set).
2758. Reverse Traversal + 32-Bit Position Tracking
2759. Reverse Traversal + HashSet

2760. Reverse Traversal / String Builder
2761. Reverse Traversal Start from the end of the string. Collect K characters, add a dash, and repeat. Reverse the result at the end.
2762. Reverse Trie (Suffix Trie) with Precomputed Min
2763. Reverse Work-Backwards with Modulo.
2764. Revert Half the Number
2765. Right Max Array + Sorted Set
2766. Right-Spine Insertion
2767. Right-to-Left Array Reduce
2768. Right-to-Left Carry Simulation (Greedy)
2769. Right-to-Left Monotonic Scan.
2770. Rolling Hash (Rabin-Karp) + Hash Set
2771. Rolling Hash (Rabin-Karp) - Reversed.
2772. Rolling Hash (Rolling Frequencies) + Trie.
2773. Rook-Centric Scan
2774. Row Max, Column Max, and Non-Zero Count.
2775. Row Numbering / Window Functions.
2776. Row Sorting + Column Maximums
2777. Row Sorting + Global Priority Queue. O(N Â· M log M + NM log K)
2778. Row-based Parity Processing + Array Slicing. O(M Â· N)
2779. Row-by-Row Comparison Buffer (Visualizing the SQL engine re-scanning the bookings table for every single record).
2780. Row-by-Row Memory Copy (Intermediate tables filling up Cartesian space).
2781. Row-Pair Comparison (DP-lite)
2782. Row-wise Iteration (Simple Scan)
2783. Row/Column Frequency Counting
2784. Row/Column Tally Arrays.
2785. Row/Column/Diagonal Counters
2786. Row/Column/Diagonal Sum Counters
2787. Rule-Based Combinatorics (Independent Slots)
2788. Running Max + Prefix Sum (Single Pass)
2789. Running Maximum Comparison
2790. Running Maximum Property
2791. Running Total (Cumulative Frequency) Analysis.
2792. Schoolbook Addition (Right-to-Left)
2793. sed or tail + head .
2794. Segment Tree
2795. Segment Tree (DP State Merging)
2796. Segment Tree (Dynamic) / Churning Map
2797. Segment Tree (Max-Range) + Binary Search for First Index. O(N log N)
2798. Segment Tree (Point Updates) tracking Max Subarray Sum or DP Prefix mapping. (O(N log N))
2799. Segment Tree (Range Sum & Max)
2800. Segment Tree + Balanced BST (Sorted Set)
2801. Segment Tree / Balanced BST (TreeMap)

2802. Segment Tree / Fenwick Tree
2803. Segment Tree / Fenwick Tree (Range Maximum Query)
2804. Segment Tree or Binary Indexed Tree (Fenwick Tree)
2805. Segment Tree or Fenwick Tree (BIT) for O(N log N) LIS.
2806. Segment Tree or Fenwick Tree with Lazy Propagation. Store XOR values in nodes and apply multiplications as "Lazy" updates.
2807. Segment Tree or Square Root Decomposition where each "node" or "bucket" stores partial Trionic counts.
2808. Segment Tree where each node stores a "Bit Count" array (size 32 or 64). Multiplication by a power of 2 becomes a "Bit Shift" on the array.
2809. Segment Tree with Boyer-Moore Voting or Random Sampling + Range Binary Search (using a map of indices).
2810. Segment Tree with Lazy Propagation
2811. Segment Tree with Lazy Propagation + Sweepline
2812. Segment Tree with Lazy Propagation.
2813. SELECT DISTINCT + ORDER BY
2814. Self Join with absolute difference = 1 OR Window Functions (LAG/LEAD).
2815. Self-Anti-Join / WHERE NOT IN
2816. Self-Balancing BST or Linked List (Ideal)
2817. Self-Join / Aggregate CASE WHEN
2818. Self-Join Delete.
2819. Self-Join on Common Friends
2820. Self-Join on Follower Column
2821. Self-Join on ID = Reports_To + Group By.
2822. Self-Join on ManagerId = Id .
2823. Self-Join with Anti-Join (NOT IN)
2824. Self-Join with DATEDIFF .
2825. Self-Join with Inequality Constraints
2826. Self-Join with ROW_NUMBER()
2827. Semaphores + CyclicBarrier / Phasers
2828. Sentinel Node + Two Pointers
2829. Serialized Memory Wait (The closure for the addition waits twice).
2830. Set Difference / Hash Map
2831. Set Size vs. Half-Length Comparison.
2832. Set-Based Adjacency List + DFS/BFS for Components. O(N^2 Â· M) (Optimized constant factor).
2833. Set-Based Unique Value Counting
2834. setTimeout + Promise Wrapper
2835. Shrinking Bitwise Set
2836. Shunting Yard Algorithm / Two Stacks (Operands & Operators)
2837. Shunting-yard Algorithm / Stack
2838. Sieve + Sliding Window / Prefix Sums.
2839. Sieve of Eratosthenes
2840. Sieve of Eratosthenes (Precomputation) + Linear Array Scan.
2841. Sieve of Eratosthenes (precompute primes up to N) + Single Pass Prefix/Suffix

sums.
2842. Sieve of Eratosthenes (precompute primes) + Unbounded Knapsack Dynamic Programming (Coin Change variant).
2843. Sieve of Eratosthenes + BFS with a "furthest reached" pointer to strictly avoid re-scanning identical prime gaps.
2844. Sieve of Eratosthenes + Binary Search (Greedy)
2845. Sieve of Eratosthenes + Combinatorics
2846. Sieve of Eratosthenes + Digit Mask
2847. Sieve of Eratosthenes + Disjoint Set Union (DSU) or DFS
2848. Sieve of Eratosthenes + Math
2849. Sieve of Eratosthenes + Prefix Prime Frequencies.
2850. Sieve of Eratosthenes + Rolling Hash / DP.
2851. Sieve of Eratosthenes + Sliding Window / Prefix Sums
2852. Sieve of Eratosthenes + Two Pointers
2853. Sieve of Eratosthenes / Harmonic Series Mapping. O(M log M + N)
2854. Simple conditional logic: IF or CASE WHEN.
2855. Simple DFS Recursion.
2856. Simple Greedy Simulation
2857. Simple Iteration (Small Constraints)
2858. Simple Linear Scan
2859. Simple WHERE clause with OR.
2860. Simulation (Direction Flag) Use a direction flag (Up/Down). Calculate new i, j based on boundaries (e.g., if at last column, move down and flip direction).
2861. Simulation (due to small N=100). O(N^2)
2862. Simulation (Two-Pointer Logic)
2863. Simulation + 3D State Tracking (r, c, direction)
2864. Simulation + Hash Set
2865. Simulation + Modular Exponentiation
2866. Simulation + Prime Factorization
2867. Simulation / Greedy
2868. Simulation / Minute-by-Minute Increment.
2869. Simulation or Mathematical Arithmetic Progression.
2870. Simulation with Condition Validation
2871. Simulation with Leftmost Greedy Search. O(N^2) is acceptable given N=100.
2872. Simulation with Queue (Work Backwards/Index Mapping)
2873. Simulation with Real Stack
2874. Simulation with Step Increment (Greedy Walking).
2875. Single Counter Tracking
2876. Single Loop Math
2877. Single Pass (Hash Sets)
2878. Single Pass (Head Counting) Iterate through the grid. Only count an 'X' if it doesn't have an 'X' above it or to its left.
2879. Single Pass (In-Place)
2880. Single Pass (Running Sum and Count)
2881. Single Pass (Shared Edge) Count land cells x 4. Subtract 2 for every shared

edge (horizontal or vertical neighbor).
2882. Single Pass (Space Detection)
2883. Single Pass + Prev Pointer
2884. Single Pass / Linear Indexing
2885. Single Pass / Offset Comparison
2886. Single Pass / State Tracking (Current Streak)
2887. Single Pass Boolean Flag Scan.
2888. Single Pass Bound Check. O(N)
2889. Single Pass Bucketing (or SQL UNION/GROUP BY)
2890. Single Pass Comparison (Pythagorean Theorem)
2891. Single Pass Contraction Check.
2892. Single Pass Count Iterate through characters. Count a segment whenever you see a non-space character that follows a space (or is at index 0).
2893. Single Pass Counter (Greedy).
2894. Single Pass Counter Iterate through the array. Increment `count` if 1; if 0, update `max` and reset `count`.
2895. Single Pass Counting (Independent Moves)
2896. Single Pass First-Occurrence Map
2897. Single Pass Greedy Tracking
2898. Single Pass Greedy Training
2899. Single Pass Greedy Violation Detection
2900. Single Pass Linear Scan
2901. Single Pass Linear Scan / Prefix Sums.
2902. Single Pass Matrix Traversal
2903. Single Pass Max Tracking
2904. Single Pass Max/Min Tracker (Greedy).
2905. Single Pass Minimum Tracking.
2906. Single Pass State Toggle
2907. Single Pass State Tracking.
2908. Single Pass Streaming Maximum (King of the Hill).
2909. Single Pass Tokenization / On-the-fly Parsing
2910. Single Pass Tracking (Max/Min Greedy)
2911. Single Pass with Hash Map/Frequency Array.
2912. Single Pass with Lookahead (In-Place)
2913. Single Pass with Running Minimum (Prefix Min)
2914. Single Pass with Three-Node Window + Edge Point Tracking
2915. Single Pointer Iteration (count spaces and substring once).
2916. Single Queue Rotation
2917. Single-Pass Accumulation with Modulo Digit Extraction.
2918. Single-Pass Accumulator / State Machine
2919. Single-Pass Aggregation with CASE/IF statements
2920. Single-Pass Array Mapping
2921. Single-Pass Bottom-Up / Top-Down DFS
2922. Single-Pass Character Counting.
2923. Single-Pass Constant Space Scan

2924. Single-Pass Early Exit
2925. Single-Pass Filtering
2926. Single-Pass Flagging
2927. Single-Pass Frequency Array
2928. Single-pass Frequency Array (size 26) + Max Frequency tracking.
2929. Single-Pass Frequency Map.
2930. Single-Pass Greedy Accumulation (Kadane's Variant).
2931. Single-Pass Greedy Accumulation / Simulation.
2932. Single-Pass Greedy Counter
2933. Single-Pass Greedy Scan
2934. Single-pass Hash Map / Frequency Array. Count all frequencies first, then perform a single pass to filter.
2935. Single-Pass Index Mapping
2936. Single-Pass Index Traversal (Step K)
2937. Single-Pass Iteration / In-Place Bit Manipulation.
2938. Single-pass iteration with a running bitwise OR variable and a parity check.
2939. Single-Pass Level Accumulation
2940. Single-Pass Linear Scan
2941. Single-Pass Mathematical Extraction
2942. Single-Pass Max & Second Max Tracking.
2943. Single-Pass Parity Check
2944. Single-Pass Pointer Distance
2945. Single-Pass Pointer Scan
2946. Single-Pass Range Intersection
2947. Single-Pass Reverse Scan (Right-to-Left Pointer)
2948. Single-Pass Scalar Tracking.
2949. Single-pass Simulation using a Stack or a dynamically sized Array/ StringBuilder.
2950. Single-Pass Simulation with ASCII Math. O(N)
2951. Single-Pass Stack (Precedence Handling)
2952. Single-pass state machine or boolean flag tracking within a single iteration.
2953. Single-Pass Two-Pointer (In-place if bits allow)
2954. Single-Pass Two-Pointer / StringBuilder
2955. Single-pass: Find the first and last occurrence of different words. The max distance is either LastIndex - FirstIndex or LastIndex - SecondFirstIndex .
2956. Single-Pivot Circularity Check
2957. Single/Double Pass Hash Map (Frequency Array)
2958. Sliding Boundary Pointers
2959. Sliding Window
2960. Sliding Window (At Most K) Transformation.
2961. Sliding Window (atLeast(k) - atLeast(k+1))
2962. Sliding Window (Constant Space)
2963. Sliding Window (Exact = AtMost(K) - AtMost(K-1))
2964. Sliding Window (Expand/Contract)
2965. Sliding Window (Expand/Contract) Use two pointers. Keep track of `maxFreq`

in current window. If `windowSize - maxFreq > k`, shrink left.
2966. Sliding Window (Expandable) Maintain a window with at most one '0'. If zeros > 1, shrink left pointer.
2967. Sliding Window (Expanding Right, Shrinking Left)
2968. Sliding Window (Fixed 2x2) / Counting
2969. Sliding Window (Fixed Size)
2970. Sliding Window (Fixed Size) / Prefix Sums
2971. Sliding Window (Fixed Size).
2972. Sliding Window (Fixed)
2973. Sliding Window (Flexible)
2974. Sliding Window (Hash Set)
2975. Sliding Window (Inverse approach)
2976. Sliding Window (Monotonic Property)
2977. Sliding Window (Observation: minSize only)
2978. Sliding Window (Optimised Summing)
2979. Sliding Window (Queue)
2980. Sliding Window (Shrinkable)
2981. Sliding Window (String or Math)
2982. Sliding Window (Target = TotalSum - x)
2983. Sliding Window (Two Pointers + Map)
2984. Sliding Window (Two Pointers)
2985. Sliding Window (Two Pointers) Run Twice
2986. Sliding Window (Two Pointers) with Hash Map.
2987. Sliding Window (Two Pointers) with Sub-problem Logic
2988. Sliding Window (Two Pointers).
2989. Sliding Window (Two-Pointer / At-Most)
2990. Sliding Window (Two-Pointers)
2991. Sliding Window (Variable Size)
2992. Sliding Window + 1D DP (Prefix Min)
2993. Sliding Window + 2D Data Structure (Segment Tree/Fenwick Tree) or Sorted Set.
2994. Sliding Window + Binary Trie
2995. Sliding Window + Bit Count Array
2996. Sliding Window + Deficit Counter
2997. Sliding Window + Dynamic Programming (Prefix Max).
2998. Sliding Window + Fenwick Tree / Segment Tree
2999. Sliding Window + Frequency Array (Counting Sort)
3000. Sliding Window + Frequency Array/Map.
3001. Sliding Window + Frequency Array/Set.
3002. Sliding Window + Frequency HashMap.
3003. Sliding Window + Frequency Map
3004. Sliding Window + Frequency Map (Fixed Size).
3005. Sliding Window + Frequency Map (or Set)
3006. Sliding Window + Frequency Map / Bitmask.
3007. Sliding Window + Frequency Map Use an array of size 26. As the window

moves, increment the "entering" char and decrement the "leaving" char.
3008. Sliding Window + Hash Map (Frequencies)
3009. Sliding Window + Hash Map (Two Pointers)
3010. Sliding Window + Hash Map of Sets
3011. Sliding Window + Hash Maps
3012. Sliding Window + Hash Set
3013. Sliding Window + Hash Set (or Rolling Hash)
3014. Sliding Window + Hash Set (or Rolling Hash).
3015. Sliding Window + Mathematical Logic.
3016. Sliding Window + Median Distance + Prefix Sums
3017. Sliding Window + Monotonic Deque
3018. Sliding Window + Monotonic Deques
3019. Sliding Window + Monotonic Deques (Min/Max Tracking)
3020. Sliding Window + Ordered Set / Hash Map
3021. Sliding Window + Prefix Sums
3022. Sliding Window + Rabin-Karp / Trie
3023. Sliding Window + Running Medians (Two Heaps).
3024. Sliding Window + Segment Splitting
3025. Sliding Window + Two Balanced BSTs (Heaps)
3026. Sliding Window + Two Heaps (Min/Max) to maintain dynamic median. O(N log K)
3027. Sliding Window + Two Heaps (or Balanced BST)
3028. Sliding Window / DP
3029. Sliding Window / Greedy.
3030. Sliding Window / Hash Map
3031. Sliding Window / Hash Set.
3032. Sliding Window / Monotonic Queue.
3033. Sliding Window / Prefix Sum Array
3034. Sliding Window / Prefix Sum Hashing
3035. Sliding Window / Randomized Search
3036. Sliding Window / Regex Pattern Matching. O(N)
3037. Sliding Window / Sweep Line
3038. Sliding Window / Two Pointers with Bitmask Running OR
3039. Sliding Window DP.
3040. Sliding Window Maximum
3041. Sliding Window on concatenated string (s + s).
3042. Sliding Window on Digit String "123456789"
3043. Sliding Window over an array of indices + Median greediness.
3044. Sliding Window over Gap Arrays. O(N)
3045. Sliding Window with a single counter for "duplicate pairs".
3046. Sliding Window with Frequency Map.
3047. Sliding Window with Hash Map (Frequency Counter).
3048. Sliding Window with Min-Heap.
3049. Sliding Window with Monotonic Deque OR Segment Tree (depending on exact stability constraint).

3050. Sliding Window: exact(k) = atLeast(k) - atLeast(k+1)
3051. Slow/Fast Pointers + In-Place Reversal of Second Half
3052. Sort & Interleave (Wiggle Sort)
3053. Sort + Anchor & Squeeze
3054. Sort + Backtracking (Sibling Skip)
3055. Sort + Backtracking (Skip Dupes)
3056. Sort + Backtracking (Used Array / Skip)
3057. Sort + Binary Search (Two Pointers)
3058. Sort + Binary Search LIS
3059. Sort + Binary Search on Answer + Greedy
3060. Sort + De-duplicate + Hash Map
3061. Sort + Deduplicate + Sliding Window (Two Pointers)
3062. Sort + Greedy (Adjacent Triplet)
3063. Sort + Greedy Linear Scan
3064. Sort + Insert Sort descending by height (h), then ascending by (k). Insert people into a list at index k.
3065. Sort + Longest Increasing Subsequence (LIS) via Binary Search
3066. Sort + Prefix Check (or Trie)
3067. Sort + Tracking Next Available
3068. Sort + Two Anchors + Squeeze
3069. Sort + Two Pointers (Greedy)
3070. Sort + Two Pointers (Left/Right)
3071. Sort + Two Pointers.
3072. Sort Both Arrays + Sequential Pairing. O(N log N)
3073. Sort Buffer (Visualize the O(N) or O(log N) extra space used by sorting).
3074. Sort Buffer (Visualizing the O(N) memory used to rearrange the array).
3075. Sort Buffer (Visualizing the O(N) or O(log N) extra space used by the sorting algorithm).
3076. Sort by End Time + Binary Search + 2D Dynamic Programming (dp[i][quota]). (O(N log N))
3077. Sort by Start (Asc) and End (Desc) + Greedy Scan
3078. Sort Dictionary + Two Pointers.
3079. Sort edges by time, use Disjoint Set Union (DSU). Start with C=V components. Decrement C for every successful union. Stop when C == K.
3080. Sort Intervals + Sliding Window snapped to Interval Boundaries (Two passes: Left/Right). (O(N log N))
3081. Sort OR Linear Scan for Top 3 and Bottom 2.
3082. Sort the array + Binary Search on the Answer Space + Greedy Checking.
3083. Sorted Time Processing + DSU with Reset
3084. Sorting & Median Finding
3085. Sorting (Attack Desc, Defense Asc) + 1-Pass Greedy
3086. Sorting (by Ratio) + Max-Heap (for Quality).
3087. Sorting + 1D Dynamic Programming (Knapsack variant)
3088. Sorting + 2D Dynamic Programming (or Max-Heap Greedy).
3089. Sorting + Adjacent Comparison

3090. Sorting + Adjacent Comparison OR Trie Insertion. O(N log N Â· L)
3091. Sorting + Adjacent Swaps
3092. Sorting + Arithmetic Progression.
3093. Sorting + Binary Search
3094. Sorting + Binary Search (By Height)
3095. Sorting + Binary Search (Prefix Sums).
3096. Sorting + Binary Search (Range Check)
3097. Sorting + Binary Search + Suffix Max
3098. Sorting + Binary Search on Answer Space.
3099. Sorting + Combinatorics + Hash Map
3100. Sorting + Comparison Math
3101. Sorting + Contribution of Each Element.
3102. Sorting + Contribution Technique + Combinatorics. O(N log N)
3103. Sorting + Cross-Multiplication
3104. Sorting + DP (Longest Increasing Subsequence Variant)
3105. Sorting + DP on Chains
3106. Sorting + Dynamic Programming
3107. Sorting + Dynamic Programming (Prefix Sum)
3108. Sorting + Dynamic Programming + Binary Search
3109. Sorting + Dynamic Programming / Two Pointers
3110. Sorting + Fenwick Tree (Binary Indexed Tree)
3111. Sorting + Fixed-Size Sliding Window
3112. Sorting + Frequency Counting
3113. Sorting + Frequency Map (or Queue)
3114. Sorting + Gap Calculation
3115. Sorting + Greedy (or 1D DP)
3116. Sorting + Greedy (Sliding Window)
3117. Sorting + Greedy / Two-Pass Max Search.
3118. Sorting + Greedy Accumulation
3119. Sorting + Greedy Accumulation.
3120. Sorting + Greedy Allocation
3121. Sorting + Greedy Alternating Distribution.
3122. Sorting + Greedy Boundary Check
3123. Sorting + Greedy Candidate Validation.
3124. Sorting + Greedy Counting.
3125. Sorting + Greedy Extremes
3126. Sorting + Greedy Matching
3127. Sorting + Greedy Pairing.
3128. Sorting + Greedy Pass (or Counting Sort for O(N)).
3129. Sorting + Greedy Row/Col Max Tracking
3130. Sorting + Greedy Subsequences
3131. Sorting + Greedy Swapping
3132. Sorting + Greedy Target Matching
3133. Sorting + Greedy Two-Pointer (Split Search).
3134. Sorting + Greedy Two-Pointer.

3135. Sorting + Grouping (Disjoint Connectivity)
3136. Sorting + Linear Interval Merge
3137. Sorting + Linear Scan
3138. Sorting + Math (Prefix Sum Logic).
3139. Sorting + Merge Intervals + Modular Exponentiation.
3140. Sorting + Merging Intervals
3141. Sorting + Min-Heap (Sliding Window of Top K).
3142. Sorting + Monotonic Arrival Times.
3143. Sorting + Parity Segregation
3144. Sorting + Prefix Maximum + Binary Search
3145. Sorting + Prefix Sum + Binary Search
3146. Sorting + Prefix Sum + Two Pointers/Binary Search
3147. Sorting + Prefix Sums + Binary Search
3148. Sorting + Prefix Sums + Binary Search (Two-Pointer Optimization)
3149. Sorting + Single Pass Optimization
3150. Sorting + Sliding Window (Fixed Size 3)
3151. Sorting + Sliding Window (Fixed Size K)
3152. Sorting + Sliding Window (Queue)
3153. Sorting + Sliding Window + Prefix Sums
3154. Sorting + Sliding Window with Min-Heap of size K. O(N log N + N log K)
3155. Sorting + Sliding Window.
3156. Sorting + Small Constant Search
3157. Sorting + Sweeping Max-Y
3158. Sorting + Two Pointers
3159. Sorting + Two Pointers (Greedy)
3160. Sorting + Two Pointers (Greedy).
3161. Sorting + Two Pointers (or Binary Search).
3162. Sorting + Two Pointers + Backtracking
3163. Sorting + Two-pass scan (Left-to-Right & Right-to-Left).
3164. Sorting + Two-Pointer
3165. Sorting + Window Functions (Lead/Lag) or Inequality Joins.
3166. Sorting / 1D Greedy Sweep (Assuming 1D coordinate plane).
3167. Sorting / Binary Search on Answer
3168. Sorting / Frequency Map + Suffix Sum
3169. Sorting / Window Functions (LEAD/LAG).
3170. Sorting by Parity / Max Heap
3171. Sorting by Start Time
3172. Sorting by Value + Sorting by Index
3173. Sorting Comparison or Counting Sort.
3174. Sorting Events + Lazy Counter for "ALL" mentions. O(M log M + N)
3175. Sorting or Bucket Sort (Boolean Array).
3176. Sorting Pair Sums (Boundary Logic).
3177. Sorting Rows
3178. Sorting with Index Tracking + Boolean Marking.
3179. Sorting with Index Tracking / Max-Heap.

3180. Sparse Table + Decreasing GCD Property.
3181. Sparse Table / GCD Property (Unique Prefix GCDs)
3182. Spatial Hashing / Grid-based Priority Queues.
3183. Sprague-Grundy Theorem (Bitwise XOR / Nim-Sum)
3184. SQL `CASE` statements with `LIKE` and `REGEXP` for vectorized pattern matching. O(N)
3185. SQL `GROUP BY` with Aggregate Function `MIN()`.
3186. SQL Aggregation / JOIN
3187. SQL Aggregation: `GROUP BY` + `COUNT()`.
3188. SQL Aggregation: `GROUP BY` on multiple columns.
3189. SQL Common Table Expression (CTE) with Filtering and Grouping. O(N)
3190. SQL Common Table Expressions (CTE) + SUM() Aggregate
3191. SQL DATE_FORMAT() function.
3192. SQL GROUP BY + COUNT(DISTINCT)
3193. SQL GROUP BY + String Aggregation + HAVING
3194. SQL IS NULL
3195. SQL JOIN + GROUP BY + HAVING
3196. SQL Join + String Aggregation.
3197. SQL JOINs + Window Functions (RANK)
3198. SQL LEAST() and GREATEST() Functions
3199. SQL Left Outer Join.
3200. SQL Length Function (LENGTH/CHAR_LENGTH)
3201. SQL LIKE Operator / String Matching
3202. SQL Logical Indexing (`AND` condition).
3203. SQL Recursive CTE + Anti-Join
3204. SQL REGEXP_REPLACE / Nested String Functions
3205. SQL Self-Join with Inequality Constraints.
3206. SQL String Aggregation (`GROUP_CONCAT` / `STRING_AGG`).
3207. SQL String Functions (CONCAT , LEFT) running directly on the database engine.
3208. SQL String Functions (INITCAP / Regex)
3209. SQL UNION ALL (Unpivoting).
3210. SQL UPDATE + CASE WHEN
3211. SQL Window Function (Cumulative Sum)
3212. SQL Window Function: `LEAD()` over Partition.
3213. SQL Window Functions (`RANK()` or `DENSE_RANK()`)
3214. SQL Window Functions (Cumulative Sum/Running Total) + CTEs
3215. SQL Window Functions (LAG/LEAD) / Pandas `groupby().rolling()`.
3216. SQL Window Functions (RANK / DENSE_RANK)
3217. SQL Window Functions + Common Table Expressions (CTEs)
3218. Square of Prime Property
3219. Square Root Decomposition (Block separation).
3220. Square Root Decomposition or Fenwick Tree.
3221. Square Root Divisor Hunt
3222. Square Root Factor Pairing

3223. Square Root Factorization + Dynamic Programming. Only check up to âN to find all factors, then use DP to find balanced subsets.
3224. Square Root Search (Greedy)
3225. Square-Free Part Grouping
3226. Stable Sort / Map of Lists
3227. Stack (Greedy Elimination)
3228. Stack (Greedy Simplification).
3229. Stack (Iterative Evaluation)
3230. Stack (LIFO processing).
3231. Stack (LIFO)
3232. Stack (LIFO) / Two Pointers
3233. Stack (Push/Pop on Chunks)
3234. Stack (Right-to-Left) Scan the string backwards. If you hit a `?`, pop the next two values from the stack and push the result based on the condition.
3235. Stack (Simulation of BODMAS/PEMDAS).
3236. Stack / Amortized Character Matching
3237. Stack / Amortized Sliding Window
3238. Stack / Array Depth Tracking
3239. Stack / Integer Counter
3240. Stack / Previous-Word Pointer
3241. Stack / Two-Pass Filtering
3242. Stack Data Structure.
3243. Stack Frame Explosion (Visualizing a call stack N-levels deep repeated billions of times).
3244. Stack or "Core Value" Summation.
3245. Stack Simulation
3246. Stack with Frequency Pairs
3247. Stack-Based Character Matching
3248. Stack-based Evaluation.
3249. Stack-based Expression Parsing + Dynamic Programming (bottom-up costs).
3250. Stack-based Interval Calculation.
3251. Stack-based Iterative Construction.
3252. Stack-Based Iterative Parsing
3253. Stack-based Parsing + State Machine.
3254. Stack-Based Parsing / Grammar state machine
3255. Stack-based Pattern Matching
3256. Stack-Based Reversal Use a stack to push 1, 2, 3 dots If you see 'I', pop everything to the result. If you see 'D', keep pushing.
3257. Stack-Based Segment Reversal
3258. Stack-Based Single Pass with Sign Tracking
3259. Stack-based Single Pass.
3260. Staircase Search (Saddleback)
3261. Staircase Search (Top-Right Start)
3262. Standard BFS/DFS Connected Components. Accumulate a `running_sum` during the traversal. Modulo K at the very end of the island.

3263. Standard Binary Search
3264. Standard Dijkstra's Algorithm using an Adjacency List and a Min-Heap (Priority Queue).
3265. Standard Library formatting (bin/format)
3266. Standard Simulation (Loop) or Math (Square root approach).
3267. Standard Trie with wordCount and prefixCount variables at each node.
3268. Standardization via Pre-filtering + Template Mapping.
3269. Start at Square Root Start W at lfloorâ{N}rfloor and decrement until N % W == 0. This ensures L and W are as close as possible.
3270. State Array Snapshots (Drawing the entire array state for every microscopic time increment).
3271. State Encoding (Bitmask) + Hash Map
3272. State Machine / 1D Dynamic Programming
3273. State Machine / Concurrent Counters
3274. State Machine / Deterministic Parsing
3275. State Machine / Single Counter tracking contiguous valid states.
3276. State Machine DP / Greedy
3277. State Machine DP / Left-Right Precomputation.
3278. State Machine DP / Math
3279. State Tracking (Depth Counter).
3280. State Tracking (Parity / Flip Count)
3281. State Tracking / Greedy
3282. State Transition Counting (Adjacent Differences)
3283. State-Layered Dijkstra (BFS with 2-Layer States).
3284. State-Space Dijkstra's Algorithm. State = (Node, Arriving_Direction).
3285. Stateful Counter + Hash Map
3286. Static Array Memory (Visualizing a large block of heap memory occupied by fixed integers).
3287. String Aggregation (GROUP_CONCAT)
3288. String Builder / Pre-allocated Array
3289. String Builder / Two Pointers (In-place if allowed).
3290. String Chunking / Substring Math.
3291. String Comparison (In-place `startsWith`).
3292. String Comparison / Line Overlap
3293. String Concatenation (Better for scale/many conditions). Append "Fizz" if mod 3, append "Buzz" if mod 5. Output if empty.
3294. String Concatenation (Double Check)
3295. String Concatenation + Substring functions
3296. String Concatenation Buffer (Visualizing the creation of 6 massive new strings in the heap).
3297. String Doubling Trick Concatenate S+S. Remove the first and last characters. If S is still a substring of the result, it has a repeated pattern.
3298. String Functions + Aggregation
3299. String Hashing + Center Expansion.
3300. String Iteration / Character Counting.

3301. String Iteration / Simulation
3302. String Manipulation / Math
3303. String Mutation Buffer (Visualizing repeated string recreations for each swap in languages with immutable strings).
3304. String Parsing + Hash Map (Frequency Counter).
3305. String Parsing + HashSet
3306. String Parsing with Strict Validation Pruning.
3307. String Predicate Filtering
3308. String Slice Memory (Draw many temporary substring copies being created in the heap).
3309. String Slicing / Cyclic Shift
3310. String Split + Bucket Array mapping.
3311. String Splitting (by '#') + Segment Matching
3312. String Splitting + Hash Map
3313. StringBuilder / String Join with Index-based Suffix.
3314. Subarray Buffer (Visualizing the heap filling with transient slices of the input array).
3315. Subquery / CTE with Date Arithmetic.
3316. Subquery / NOT IN clause.
3317. Subquery Grouping with HAVING count = 1.
3318. Subquery with In-Clause or Window Functions.
3319. Subquery with LIMIT 1 OFFSET 1 wrapped in IFNULL .
3320. Substring Slicing + Direct Integer Comparison
3321. SUBSTRING_INDEX / Regex + Group By
3322. Suffix Array Traversal (Right-to-Left Scan).
3323. Suffix Automaton (SAM) or Rolling Hash
3324. Suffix Greedy Search (Targeting "00", "25", "50", "75")
3325. Suffix Sum of Shifts.
3326. Suffix Trie (Inverted Prefix Tree).
3327. SUM(CASE) / AVG(IF)
3328. Sweep Line + Segment Tree
3329. Sweep Line + Segment Tree (with Lazy Propagation/Coverage Count). O(N log N)
3330. Sweep Line / Difference Array
3331. Sweep Line Algorithm (TreeMap)
3332. Sweep Line Algorithm + Max Heap / TreeMap
3333. Symmetric Construction (Pairing)
3334. Symmetry Grouping (2x2 Blocks + Centroids)
3335. Synchronization (Mutex/Locking)
3336. Synchronization (Semaphores / Mutex + Conditions)
3337. Synchronized DFS / BFS
3338. Target Index Mapping + Longest Increasing Subsequence (LIS) via Patience Sorting.
3339. Targeted Filter + Distinct (SQL)
3340. Tarjan's / Bridges (DFS Discovery Time)

3341. Temporal State Snapshots (Visualizing the position of every robot at every tick of time).
3342. Temporary Join Tables (Visualizing the SQL engine creating massive intermediate tables for each join).
3343. Temporary Object Accumulation (Creating massive amounts of intermediate merged objects).
3344. Temporary Property Allocation (Mutation of the target object).
3345. Temporary Reversed String Buffers (Creating a reversed copy for every comparison).
3346. Temporary String Buffers (Visualizing multiple string concatenations and copies).
3347. The Circular Link (Ring)
3348. The Reversal Algorithm.
3349. Thread Pool + Thread-Safe Queue/Set
3350. Three Pointers
3351. Three Pointers (Backward Fill)
3352. Three Pointers (Dynamic Programming)
3353. Three-Case Greedy Analysis
3354. Three-Phase Linear Scan
3355. Three-Pointer Simulation (Single-Pass Result Array).
3356. Three-Pointer Sliding Window (Tracking Indices)
3357. Tokenization + `startsWith`
3358. Top-3 Largest Gaps + Prefix/Suffix Max Gaps. O(N)
3359. Top-3 Length Tracking per Character
3360. Top-3 Row Candidates + Backtracking
3361. Top-3 Variables Keep `first`, `second`, `third` initialized to -Infinity. Update them sequentially as you iterate O(N).
3362. Top-Down DFS (Tree Path Sum)
3363. Top-Down DFS with State Passing
3364. Topological Sort (DFS with 3 States or Kahn's BFS)
3365. Topological Sort (Kahn's Algorithm / BFS)
3366. Topological Sort (Kahn's Algorithm)
3367. Topological Sort (Kahn's Algorithm) / BFS.
3368. Topological Sort (Kahn's or DFS)
3369. Topological Sort (Kahn's) + Dynamic Programming
3370. Topological Sort (Kahn's) + Dynamic Programming.
3371. Topological Sort (Kahn's) Build a graph of dependencies. Check if at every step there is exactly 1 node with an in-degree of 0.
3372. Topological Sort (Kahnâs) or Memoized DFS.
3373. Topological Sort / BFS from Leaves (Peeling the Onion)
3374. Topological Sorting (Peeling leaves like an onion)
3375. Tortoise and Hare (Slow & Fast Pointers).
3376. Total Cost - Max Character Frequency Cost
3377. Transpose + Reverse Rows
3378. Traveling Salesperson (TSP) with Bitmask DP

3379. Tree Centroid Decomposition / Subtree Weight Sums.
3380. Tree DFS/BFS Depth Mapping + Frequency Hashing + Mathematical Convolution/Lookup.
3381. Tree Diameter Endpoints (2/3 DFS Passes)
3382. Tree DP (Bottom-Up State)
3383. Tree DP (Post-order Traversal)
3384. Tree DP (Re-rooting Technique).
3385. Tree DP (Rerooting / 2-Pass DFS)
3386. Tree DP + Centroid Decomposition / Contribution of Edges
3387. Tree DP + Frequency Map (BFS/DFS)
3388. Tree DP + Memoization (State Reduction via Bit Shifts)
3389. Tree DP / Re-rooting Technique
3390. Tree DP with Combinatorics
3391. Tree DP with Greedy Sorting
3392. Tree DP with Rerooting (Change of Root).
3393. Tree Dynamic Programming (Bottom-Up).
3394. Tree Dynamic Programming (House Robber Style)
3395. Tree Dynamic Programming (Kadane's on Trees)
3396. Tree Dynamic Programming (Min Cost to Satisfy)
3397. Tree Dynamic Programming (Post-order Traversal)
3398. Tree Dynamic Programming (Rerooting or Diameters).
3399. Tree Partitioning / Subtree Counting
3400. Treemap (Sorted Dictionary)
3401. TreeMap + Max-Heap (or Sorting).
3402. TreeMap / Frequency Map + Smallest Element Processing.
3403. TreeMap / Ordered Map (Next/Prev Greater Element)
3404. TreeMap of Sorted Lists (Coordinate Grouping)
3405. Trie (Prefix Tree)
3406. Trie (Prefix Tree) + DFS Backtracking for '.'
3407. Trie (Prefix Tree) + DFS.
3408. Trie (Prefix Tree) + DFS/BFS
3409. Trie (Prefix Tree) + Min-Heap/Caching.
3410. Trie (Prefix Tree) / Hash Map
3411. Trie (Prefix Tree) Node Structure
3412. Trie (Prefix Tree) with Frequency Counters
3413. Trie (Prefix Tree) with Reversed Search.
3414. Trie (Prefix Tree) with Wildcard/Parameter Support.
3415. Trie (Prefix Tree).
3416. Trie (Prefix/Suffix) or Hashing with Word Splitting
3417. Trie + Backtracking DFS on Grid
3418. Trie + DFS/Memoization Sort words by length. Insert into a Trie. For each word, use DFS to see if it can be formed by existing Trie words.
3419. Trie + Dynamic Programming
3420. Trie + Floyd-Warshall + DP
3421. Trie + Merkle Hashing (Post-order)

3422. Trie of Combined Strings (suffix + { + prefix)
3423. Trie with DFS / Generalized Neighbors.
3424. Trie with Pairwise Character Keys
3425. Trie with Weight Accumulation.
3426. Trie-Based Backtracking
3427. Trie-Based Backtracking Use a Trie to instantly find all words that start with a specific prefix (the prefix being the current column's characters).
3428. Trie-based Directory Structure.
3429. Trie-Based Dynamic Programming
3430. Trillions of Digit Arrays (Visualizing the CPU bottleneck from converting numbers to strings repeatedly).
3431. Trillions of temporary string allocations for digit extraction.
3432. Trust Score Array (In-Degree vs Out-Degree)
3433. Two Counters (Left/Right Passes)
3434. Two Deques (Balanced Halves)
3435. Two Dummy Nodes (Merging Streams)
3436. Two Hash Maps (Double Tally)
3437. Two Hash Maps + Min-Heap (Sorted Set)
3438. Two Hash Maps + Priority Queues (Sorted Sets)
3439. Two Hash Maps / Frequency-of-Frequency Tracking.
3440. Two Hash Maps / One Map + Indexing
3441. Two Hash Maps / Two Frequency Arrays
3442. Two HashSets (Unique Intersection)
3443. Two Heaps (Balance) / TreeSet Use a Max-Heap for the left half and a Min-Heap for the right half. Use "Lazy Removal" with a Hash Map to handle elements leaving the window.
3444. Two Heaps (Median) + Hash Map & Treap/SortedSet (Mode)
3445. Two Heaps (Min-Heap & Max-Heap)
3446. Two Heaps (Min-Heap / Max-Heap) or Balanced BST.
3447. Two Heaps (Min-Heap for Capital, Max-Heap for Profit) / Greedy.
3448. Two Heaps (Priority Queues) + Prefix/Suffix Sums.
3449. Two Heaps (Priority Queues) / TreeSet
3450. Two Independent BFS/DFS Passes
3451. Two Min-Heaps (Priority Queues)
3452. Two Min-Heaps (Priority Queues): One for unused rooms, one for engaged rooms (endTime, roomIdx).
3453. Two Monotonic Stacks + Priority Queue
3454. Two Passes (Prefix and Suffix) using Max-Heaps of size K.
3455. Two Pointers (Expanding from k).
3456. Two Pointers (Fast & Slow Ruler Slide).
3457. Two Pointers (Generator/Consumer) Use an `int[]` or `char[]` and two pointers: `head` (reading how many) and `tail` (writing what's next).
3458. Two Pointers (Gravity logic) + Standard Matrix Rotation.
3459. Two Pointers (Greedy Matching)
3460. Two Pointers (Greedy Simulation).

3461. Two Pointers (Greedy Squeeze)
3462. Two Pointers (Greedy Subsequence Match)
3463. Two Pointers (Greedy)
3464. Two Pointers (Greedy).
3465. Two Pointers (In-place Swap)
3466. Two Pointers (In-Place Write)
3467. Two Pointers (In-place)
3468. Two Pointers (Inward Convergence)
3469. Two Pointers (Inward Convergence).
3470. Two Pointers (Inward Meeting)
3471. Two Pointers (Inward Search)
3472. Two Pointers (Inward Squeeze)
3473. Two Pointers (Left & Right) converging to center, using a Hash Set for vowels.
3474. Two Pointers (Left and Right).
3475. Two Pointers (Linked List Intersection Style)
3476. Two Pointers (Marker Placement)
3477. Two Pointers (Merge Sort Style)
3478. Two Pointers (Nested Indexing)
3479. Two Pointers (One-Pass).
3480. Two Pointers (Optimal Suffix Comparison)
3481. Two Pointers (Optimized Prefix/Suffix)
3482. Two Pointers (Outside-In) + Middle Checking
3483. Two Pointers (Parabolic Extremity)
3484. Two Pointers (Parallel Pointers).
3485. Two Pointers (Parallel Scan)
3486. Two Pointers (Prefix/Suffix Scan)
3487. Two Pointers (Prefix/Suffix)
3488. Two Pointers (Read & Write)
3489. Two Pointers (Read/Write) Use `read` and `write` pointers. `read` finds the end of a group; `write` updates the array in-place.
3490. Two Pointers (Reader/Writer)
3491. Two Pointers (Relative Order & Constraints)
3492. Two Pointers (Reverse Scanning).
3493. Two Pointers (Right to Left) Traverse both strings backwards. Add digits plus carry. Append to StringBuilder and reverse.
3494. Two Pointers (Sequential Match)
3495. Two Pointers (Sliding Window) + Arithmetic Progression.
3496. Two Pointers (Sliding Window) + Frequency Map.
3497. Two Pointers (Slow/Fast with Offset)
3498. Two Pointers (Slow/Fast) Use Floyd's Cycle-Finding algorithm. A fast pointer moves 2 steps, slow moves 1. Check for direction changes or single-node loops.
3499. Two Pointers (Sort + Shrink).
3500. Two Pointers (Sorted Array Property).
3501. Two Pointers (Sorted) or Hash Map (Frequency)
3502. Two Pointers (Start and End of Run).

3503. Two Pointers (String validation).
3504. Two Pointers (Subsequence Match with Constraints).
3505. Two Pointers (Vowel-Selective Swap)
3506. Two Pointers + Dummy Node
3507. Two Pointers + Greedy Backtrack
3508. Two Pointers + Greedy Sort both arrays. Give the smallest cookie that satisfies the child with the smallest greed factor.
3509. Two Pointers + Hash Set (skip leading zeros directly).
3510. Two Pointers + Mirror Map
3511. Two Pointers + Pre-allocated Array/StringBuilder.
3512. Two Pointers + Prefix Min/Max Tracking
3513. Two Pointers + Prefix Sum
3514. Two Pointers + String Builder
3515. Two Pointers / Cyclic Replacement or Sub-segment Reversal Trick.
3516. Two Pointers / Deque (Match from front, match from back).
3517. Two Pointers / Greedy (Counting 1s)
3518. Two Pointers / Greedy Prefix Sum
3519. Two Pointers / Hash Map (First & Last Occurrence)
3520. Two Pointers / Linear Comparison (Shift checking).
3521. Two Pointers / Math (Sum of N)
3522. Two Pointers / Maximum Overlap
3523. Two Pointers / Mirror Matching
3524. Two Pointers / Run-Length Encoding (RLE) grouping.
3525. Two Pointers / Single-Pass Array Traversal checking alternating non-zero boundary states.
3526. Two Pointers / Single-Pass Sliding Window.
3527. Two Pointers / Single-Pass String Check
3528. Two Pointers / Sliding Window
3529. Two Pointers / Stack-based simulation.
3530. Two Pointers / String Trimming
3531. Two Pointers / StringBuilder / RegEx
3532. Two Pointers / Suffix Analysis / Greedy Char Selection.
3533. Two Pointers / Tracking Min & Max limits
3534. Two Pointers In-Place (Symmetry Swap).
3535. Two Pointers on Compressed Pairs
3536. Two Pointers on compressed segments.
3537. Two Pointers on Difference Array.
3538. Two Pointers on Monotonic Surface
3539. Two Pointers on Sorted Degrees + Shared Edge Subtraction.
3540. Two Pointers on Sorted Intervals
3541. Two Pointers or Fermat's Theorem.
3542. Two Pointers Pointer `i` traverses the word, `j` traverses abbreviation. Parse numbers to skip `i` forward.
3543. Two Pointers with "Skip" Flag. O(N)
3544. Two Pointers with One-Time Divergence.

3545. Two Priority Queues (Available Heap, Busy Heap).
3546. Two Priority Queues (Min-Heap for Sell, Max-Heap for Buy).
3547. Two Queues (Enter and Exit) with State Tracking.
3548. Two Queues (R_index, D_index).
3549. Two separate Binary Searches (Upper Bound for negatives, Lower Bound for positives) leveraging the sorted nature of the array.
3550. Two Stacks (Amortized O(1) - "Lazy Transfer")
3551. Two Stacks (Count Stack & String Stack)
3552. Two Stacks (Data Stack + Min Stack) or One Stack with Pairs.
3553. Two Stacks (Predecessor & Successor)
3554. Two Stacks / Gap Buffer
3555. Two Stacks or Doubly Linked List
3556. Two Stacks Push all nodes of both lists onto two stacks. Pop from both to add from right-to-left.
3557. Two Variables / Counting Consecutive Blocks
3558. Two-Pass / Prefix Tracking
3559. Two-Pass BFS (Level Sums)
3560. Two-Pass DFS (Time Map + Profit DFS)
3561. Two-Pass Distance Arrays OR Two Pointers (Gap Tracking).
3562. Two-Pass Force Calculation or Two-Pointer Segmenting.
3563. Two-Pass Greedy (Left-to-Right and Right-to-Left).
3564. Two-Pass Greedy (Max Tracking)
3565. Two-Pass Hash Set (Prefix/Suffix)
3566. Two-Pass Inclusion/Exclusion (At-Most K)
3567. Two-Pass Linear Traversal (Head-to-Tail)
3568. Two-Pass Min-Sweep (Left-to-Right + Right-to-Left).
3569. Two-Pass Monotonic Logic (Min/Max Tracking).
3570. Two-Pass Precomputation OR One-Pass State Machine.
3571. Two-Pass Prefix and Suffix Sum.
3572. Two-Pass Prefix/Suffix Accumulation
3573. Two-Pass Prefix/Suffix Matching + Sliding Window.
3574. Two-Pointer "Climb"
3575. Two-Pointer (In-Place).
3576. Two-Pointer (Sentinel + Last Non-9)
3577. Two-Pointer / Boundary Tracking (Min and Second Min). O(N + M)
3578. Two-Pointer / Bucket Splitting
3579. Two-Pointer / Greedy Counting
3580. Two-Pointer / Max Tracking
3581. Two-Pointer / Sliding Boundary
3582. Two-pointer approach or Hash Map tracking based on the property target - (arr[i] + arr[j]) .
3583. Two-Pointer Array with Modulo Arithmetic.
3584. Two-Pointer Binary Search
3585. Two-Pointer Bitwise Addition
3586. Two-Pointer Greedy / Sorting.

3587. Two-Pointer Greedy Matching
3588. Two-Pointer Greedy Replacement
3589. Two-Pointer Greedy Scan or Edit Distance variant. If only specific operations are allowed (like moving to front), focus on the "Longest Matching Suffix".
3590. Two-Pointer Greedy Simulation
3591. Two-Pointer Greedy Swapping (Bubble Swaps for target positions).
3592. Two-Pointer In-Place Re-linking
3593. Two-Pointer In-place Reverse (or String Builder).
3594. Two-Pointer In-Place Swap
3595. Two-Pointer In-Place Swap (Jumping by 2k).
3596. Two-Pointer In-Place Swap.
3597. Two-Pointer Index Tracking (One Pass)
3598. Two-Pointer Integer Parsing.
3599. Two-Pointer Iteration + Mathematical Concatenation.
3600. Two-Pointer Iterators (Row & Col)
3601. Two-Pointer Jump (Step of 2)
3602. Two-Pointer Meeting Point + Distance Logic.
3603. Two-Pointer Merging (Sorted Property).
3604. Two-Pointer Position Invariant
3605. Two-pointer recursive DFS (or BFS with queue).
3606. Two-Pointer Ring Traversal
3607. Two-Pointer Sync (Head Switching).
3608. Two-Pointer Technique (Meeting in the center).
3609. Two-way expansion from 'start' index.
3610. Two-Way Summation (Circular Array)
3611. Unbounded Call Stack (Deep recursion for every valid path permutation).
3612. Unbounded Integer Accumulator (Memory usage of the counter variable growing as it fails to terminate).
3613. Unbounded Knapsack + Case Addition
3614. Unbounded Step Counter.
3615. Unified Hash Map (Frequency Counting).
3616. UNION ALL + CTE / Subquery
3617. UNION ALL + GROUP BY.
3618. UNION ALL + Unified GROUP BY (CTEs)
3619. UNION ALL + Window Functions
3620. UNION ALL followed by GROUP BY and ORDER BY LIMIT 1.
3621. UNION ALL for Event Splitting + GROUP BY
3622. UNION ALL within a CTE + Group By.
3623. UNION CTE + Window/Aggregate
3624. Union Find (DSU) / Greedy
3625. Union Find with Path Compression
3626. UNION of two filtered sets.
3627. Union-Find (Connected Components)
3628. Union-Find (Consistency Check)
3629. Union-Find (Disjoint Set Union)

3630. Union-Find (Disjoint Set Union).
3631. Union-Find (Disjoint Set)
3632. Union-Find (Disjoint Set) with Dynamic Group Tracking.
3633. Union-Find (Disjoint Set) with Math Multiples
3634. Union-Find (DSU) with Component Counting.
3635. Union-Find (DSU) with Path Compression.
3636. Union-Find + Prime Factorization
3637. Union-Find + Sieve of Eratosthenes
3638. Union-Find + Topological Sort (Indegree)
3639. Union-Find / Greedy Coloring + LCP Verification.
3640. Union-Find or Boundary Mapping
3641. Union-Find with Component Size Tracking + Adjacency Map.
3642. Union-Find with Optimized Similarity Check.
3643. Union-Find with Weights / Directed Graph DFS
3644. Unique Char Counting Observe unique identifiers: 'z' is only in zero, 'w' in two, 'x' in six. Subtract counts in a specific order.
3645. Unique Positive Integers (Hash Set)
3646. Unix Pipeline (cat | tr | sort | uniq -c | sort -nr).
3647. Update-History Log (Inverse Search)
3648. Upper Bound Estimation + String Search.
3649. Value Overwriting / "Node Identity Theft"
3650. valueOf() and toString() Overriding
3651. Variable Sliding Window / State Counting
3652. Variable-Size Sliding Window (Caterpillar Method).
3653. Vector Math / State Check.
3654. Vectorized Assignment: df['bonus'] = df['salary'] * 2
3655. Virtual 1D Array + Hash Map mapping (Fisher-Yates Shuffle).
3656. Visited Node HashSet (Visualizing extra memory to store every node ID to prevent infinite loops).
3657. Visited Set / In-Place Marking.
3658. Weekday Function (SQL CASE/WEEKDAY)
3659. Weighted Average Formula
3660. Weighted Median / Binary Search on Answer
3661. Welzl's Algorithm (Recursive Smallest Enclosing Circle)
3662. WHERE clause filtering directly in SQL engine.
3663. Win-Condition Invariance Check
3664. Window Function (`DENSE_RANK()`)
3665. Window Function (`FIRST_VALUE`) / `RANK()`
3666. Window Function (`RANK()` / `ROW_NUMBER()`)
3667. Window Function (`RANK()`) + Pivoting
3668. Window Function (AVG over Partition)
3669. Window Function (COUNT OVER PARTITION)
3670. Window Function (DENSE_RANK)
3671. Window Function (LAG/LEAD)
3672. Window Function (LEAD) + CTE

3673. Window Function (LEAD/LAG) or PERIOD_DIFF.
3674. Window Function (MAX OVER)
3675. Window Function (MAX) / GROUP BY
3676. Window Function (MIN) or Grouped Filtering
3677. Window Function (PERCENT_RANK)
3678. Window Function (RANK)
3679. Window Function (RANK/DENSE_RANK) or Join with Max-Group.
3680. Window Function (ROW_NUMBER + COUNT)
3681. Window Function (ROW_NUMBER)
3682. Window Function (ROW_NUMBER) + CTE
3683. Window Function (ROW_NUMBER) + Pivot (CASE WHEN).
3684. Window Function (ROWS BETWEEN).
3685. Window Function / LAG (SQL)
3686. Window Function `ROW_NUMBER() OVER(PARTITION BY player_id ORDER BY event_date)`.
3687. Window Function: `SUM(games_played) OVER(PARTITION BY player_id ORDER BY event_date)`.
3688. Window Function: DENSE_RANK() .
3689. Window Function: DENSE_RANK() OVER(ORDER BY Score DESC) .
3690. Window Function: RANK() or ALL operator.
3691. Window Functions (`RANK()` or `MAX() OVER`)
3692. Window Functions (`RANK()` or `ROW_NUMBER()`).
3693. Window Functions (`ROW_NUMBER`).
3694. Window Functions (AVG ... OVER ROWS BETWEEN 2 PRECEDING)
3695. Window Functions (AVG OVER PARTITION).
3696. Window Functions (COUNT with Range)
3697. Window Functions (Dense_Rank / Max over Partition)
3698. Window Functions (DENSE_RANK)
3699. Window Functions (Gaps and Islands).
3700. Window Functions (id - ROW_NUMBER() grouping).
3701. Window Functions (LAG / Difference)
3702. Window Functions (LAG / LEAD)
3703. Window Functions (LEAD / DENSE_RANK)
3704. Window Functions (Max Over Partition)
3705. Window Functions (RANK / CTE)
3706. Window Functions (RANK / DENSE_RANK)
3707. Window Functions (RANK / MAX OVER)
3708. Window Functions (RANK() / PERCENT_RANK())
3709. Window Functions (RANK, COUNT, CEIL)
3710. Window Functions (ROW_NUMBER) to group consecutive sequences + CTEs.
3711. Window Functions (ROW_NUMBER/RANK)
3712. Window Functions (Sliding Window)
3713. Window Functions (SUM OVER)
3714. Window Functions (SUM OVER) + CASE
3715. Window Functions + CTE Aggregation

3716. Window Functions + Floor Math
3717. Window Functions / CTE (Common Table Expressions)
3718. Window Functions / CTEs
3719. Window Functions / Order By + Limit 1
3720. Window Functions or Common Table Expressions (CTEs) with MAX() and AVG().
3721. Window Functions or Subqueries with Group By.
3722. Window Functions with IGNORE NULLS
3723. Window Functions: LEAD() or LAG() .
3724. Word-by-Word Parsing
3725. Wormhole Teleportation (O(N) Pre-matching)
3726. X-Coordinate Sorting
3727. XOR Inverse Property
3728. XOR Math or Frequency Array
3729. XOR Parity Property (Even/Odd Counts)
3730. XOR Sum Property
3731. Z-Algorithm
3732. Z-Algorithm (Prefix & Suffix Matching)
3733. Z-Algorithm / KMP Prefix Function
3734. Zellerâs Congruence / Sum of Days
3735. Zero-Skipping Sparse Traversal / Compressed Sparse Row (CSR)